

THE ICT UNIVERSITY

Faculty of Information and Communication Technology (FICT)

Department of Cybersecurity

**PhishGuard-AI: Design and Implementation of an AI-Based
Platform for Phishing Detection and Cybersecurity
Awareness in Cameroon**

A dissertation submitted in partial fulfilment of the requirements for the award of a

Bachelor of Science (BSc) Degree in Cybersecurity

By

DJAMBOU NOUBIBOU Lareine Tracy

Registration Number: ICTU20234152

Supervisor: Eng. NEBASI Brayan

Academic Year 2025/2026

Yaoundé, Cameroon

Declaration

I, **DJAMBOU NOUBIBOU Lareine Tracy**, registration number **ICTU20234152**, hereby declare that this dissertation, entitled *"PhishGuard-AI: Design and Implementation of an AI-Based Platform for Phishing Detection and Cybersecurity Awareness in Cameroon"*, is my own work and has not been submitted, in whole or in part, for any other degree or qualification at this or any other university. All sources of information used have been duly acknowledged by means of references.

In the interest of transparency, I declare that AI-based programming assistants were used as development tools during the implementation of the platform, in the same way as compilers, libraries and documentation are used. The problem definition, the requirements, the design decisions, the testing and the interpretation of results presented in this dissertation are my own responsibility.

Student: DJAMBOU NOUBIBOU Lareine Tracy

Signature: Date:

Certification

This is to certify that this dissertation entitled "*PhishGuard-AI: Design and Implementation of an AI-Based Platform for Phishing Detection and Cybersecurity Awareness in Cameroon*" was carried out by **DJAMBOU NOUBIBOU Lareine Tracy** (ICTU20234152) under my supervision, in partial fulfilment of the requirements for the award of the Bachelor of Science degree in Cybersecurity at The ICT University.

Supervisor: Eng. NEBASI Brayan

Signature: Date:

Head of Department: Eng. TANWI

Signature: Date:

Dedication

To my beloved family, for their unconditional love, their sacrifices and their constant encouragement throughout my studies.

Acknowledgements

First and foremost, I thank Almighty God for the strength, health and wisdom that allowed me to complete this work.

I express my deep gratitude to my supervisor, **Eng. NEBASI Brayan**, for his guidance, his availability and his valuable remarks throughout this project.

My thanks go to the Dean of the Faculty of ICT, **Dr. NKANDEU Pascal**, and to the Head of Department, **Eng. TANWI**, as well as to all the lecturers of The ICT University for the quality of the training received.

I thank my family for their moral and financial support, and my classmates and friends for their encouragement and for testing the platform.

Finally, I thank everyone who, directly or indirectly, contributed to the realisation of this work.

Faculty Approval

This dissertation has been read and approved by the undersigned members of the Faculty of Information and Communication Technology as meeting the requirements for the award of the Bachelor of Science degree in Cybersecurity.

Supervisor: Eng. NEBASI Brayan

Signature: Date:

Head of Department: Eng. TANWI

Signature: Date:

Dean, Faculty of ICT: Dr. NKANDEU Pascal

Signature: Date:

Abstract

Phishing is the most common entry point of cyber-attacks, and in Cameroon it has taken a local form: fake mobile-money alerts (MTN MoMo, Orange Money), false electricity bills, fake bank security notices, job and lottery scams, mostly written in French or Cameroonian English and delivered by SMS, WhatsApp and e-mail. Existing protection tools are built for English-language corporate e-mail, and the population has limited access to awareness resources. This dissertation presents **PhishGuard-AI**, a web platform that lets any citizen check a suspicious message, link, screenshot or mailbox, understand why it is dangerous, and learn to recognise such attacks.

The work follows the Design Science Research methodology (Peppers et al., 2007) in four iterations. The detection engine fuses four independent signals: a text classifier (TF-IDF on words and character n-grams with logistic regression), a large-language-model analysis (Claude), a URL-intelligence module (domain age, look-alike brands, blocklists, redirects) and an expert rule set adapted to Cameroon, with deterministic overrides for blocklisted links and brand impersonation. For e-mails, the platform additionally traces the sender's origin from the Received headers (IP geolocation, datacentre/proxy flags, DNS blacklists). The platform is built with Flask, PostgreSQL and Vue 3; it offers read-only Gmail/Outlook scanning through OAuth 2.0, a "forward to PhishGuard" address, a progressive web app with a share target, an awareness quiz, and a security operations centre for administrators (review queue, incident correlation, threat radar, audit log, sudo mode).

A corpus of 6,370 messages was assembled (Cameroonian templates, a hand-written challenge set and the public UCI SMS Spam Collection) and evaluated with a grouped hold-out split. On held-out Cameroonian messages, the full pipeline reaches an F1-score of **0.852** without the AI signal (precision 0.899, false-positive rate 10.6 %) and **0.939** with it (precision 0.92, recall 0.958), against 0.876 F1 but a 31.9 % false-positive rate for the text model alone. Median latency is 10 ms offline and about 7 s with the AI. These results show that combining signals mainly reduces false alarms, which is decisive for user trust. The main limitations are the partly synthetic corpus and the absence of a field acceptance study, which are identified as the next steps.

Keywords: phishing detection, machine learning, large language models, mobile money fraud, cybersecurity awareness, Cameroon, design science research.

Table of Contents

Chapter 1: General Introduction	1
1.1 Introduction.....	1
1.2 Background to the Study.....	1
1.3 Problem Statement.....	2
1.4 Objectives of the Study.....	3
1.4.1 General objective	3
1.4.2 Specific objectives	3
1.5 Research Questions.....	4
1.6 Significance of the Study	4
1.7 Scope and Delimitation of the Study	5
1.8 Definition of Key Terms	5
1.9 Organisation of the Dissertation	6
Chapter 2: Literature Review.....	7
2.1 Introduction.....	7
2.2 Conceptual Framework.....	7
2.2.1 Phishing and its variants	8
2.2.2 Anatomy of a phishing message	9
2.2.3 E-mail authentication and origin.....	9
2.3 Theoretical Framework.....	9
2.3.1 Protection Motivation Theory.....	9
2.3.2 Dual-process theory and the psychology of persuasion.....	10
2.3.3 Design Science Research	10
2.3.4 Security design principles	10
2.4 Empirical Review.....	10
2.4.1 List-based detection	10

2.4.2 Heuristic and rule-based detection.....	11
2.4.3 Machine-learning detection	11
2.4.4 Deep learning and large language models	11
2.4.5 Awareness and training.....	12
2.4.6 Studies in the African and Cameroonian context	12
2.4.7 Typology of scams observed in Cameroon.....	12
2.4.8 Ensemble methods and explainability	13
2.5 Review of Existing Solutions.....	14
2.6 Research Gap	14
2.7 Summary	15
Chapter 3: Research Methodology.....	16
3.1 Introduction.....	16
3.2 Research Design.....	16
3.3 Development Methodology	18
3.4 Data Gathering	20
3.4.1 Requirements data.....	20
3.4.2 Message data.....	20
3.4.3 Corpus construction procedure	21
3.5 Population and Sample	23
3.6 Quantitative Analysis.....	23
3.6.1 Training procedure of the text model.....	24
3.6.2 Threats to validity	25
3.7 Processes, Methods, Techniques and Tools.....	25
3.7.1 Modelling.....	25
3.7.2 Detection techniques.....	25
3.7.3 Tools and technologies	26
3.7.4 Ethical considerations	27

3.8 Summary	27
Chapter 4: Analysis, Design, Implementation and Findings	28
4.1 Introduction.....	28
4.2 System Analysis.....	28
4.2.1 Analysis of the current situation	28
4.2.2 Actors	28
4.2.3 Functional requirements.....	30
4.2.4 Non-functional requirements	30
4.3 System Design	31
4.3.1 Architecture.....	31
4.3.2 Design of the detection pipeline	32
4.3.3 Class design	36
4.3.4 Database design	37
4.3.5 Dynamic behaviour.....	38
4.3.6 Design of the sender-origin tracer.....	41
4.3.7 Security operations design	43
4.3.8 Security design.....	44
4.3.9 Deployment design	46
4.3.10 User-interface design	47
4.4 Implementation	48
4.4.1 Public interface and visitor flow	48
4.4.2 Result and explanation.....	50
4.4.3 User dashboard and mailbox scanning	53
4.4.4 Awareness module	57
4.4.5 Administration and security operations	59
4.4.6 Key implementation details	68
4.5 Testing.....	70

4.5.1 Test strategy	70
4.5.2 Functional test cases	70
4.5.3 Security test cases	71
4.6 Findings.....	71
4.6.1 Detection performance.....	71
4.6.2 Response time	73
4.6.3 Achievement of the specific objectives	73
4.6.4 Results per test source.....	73
4.6.5 Error analysis	74
4.6.6 Limitations observed.....	74
4.7 Summary	75
References.....	76
Appendix A: User Acceptance Questionnaire	79
Part 1 — Profile	79
Part 2 — Perception (Likert 1–5).....	79
Part 3 — Performance measures (observed).....	79
Part 4 — Open questions	80

List of Tables

Table 1.1: Definition of key terms	6
Table 2.1: Typology of phishing and scam messages observed in Cameroon	13
Table 2.2: Comparison of existing solutions with PhishGuard-AI.....	14
Table 3.1: Application of the DSR activities in this study	18
Table 3.2: Content of the development iterations	20
Table 3.3: Sources of the message corpus	21
Table 3.4: Families of the Cameroonian held-out test set	23
Table 3.5: Evaluation metrics	24
Table 3.6: Tools and technologies used.....	27
Table 4.1: Actors of the system	28
Table 4.2: Functional requirements and implementation status	30
Table 4.3: Non-functional requirements	31
Table 4.4: Signals of the detection pipeline.....	35
Table 4.5: Scoring rules of the URL-intelligence signal	36
Table 4.6: Flags raised by the origin tracer.....	43
Table 4.7: Security principles and their implementation	46
Table 4.8: Summary of the authentication and session mechanisms.....	46
Table 4.9: Main back-end modules.....	68
Table 4.10: Main REST endpoints.....	70
Table 4.11: Functional test cases	71
Table 4.12: Security test cases	71
Table 4.13: Detection performance on held-out Cameroonian messages.....	71
Table 4.14: Measured analysis latency	73
Table 4.15: Achievement of the specific objectives	73
Table 4.16: F1-score of each configuration per test source (flag threshold 40)	73

Table 4.17: Errors of the pipeline without AI on Cameroonian test messages, by family74

List of Figures

Figure 2.1: Conceptual framework of the study	8
Figure 2.2: Positioning of PhishGuard-AI with respect to the research gap	15
Figure 3.1: Design Science Research process applied to PhishGuard-AI (after Peffers et al., 2007)	17
Figure 3.2: The four development iterations.....	19
Figure 3.3: Composition of the 6,370-message corpus.....	21
Figure 4.1: Use-case diagram of PhishGuard-AI.....	29
Figure 4.2: Three-tier architecture of PhishGuard-AI	32
Figure 4.3: Detection pipeline with four fused signals	34
Figure 4.4: Class diagram (main classes)	37
Figure 4.5: Entity-relationship diagram	38
Figure 4.6: Sequence diagram of a visitor scan and result claim.....	39
Figure 4.7: Sequence diagram of a mailbox synchronisation	40
Figure 4.8: Activity diagram of an analysis.....	41
Figure 4.9: Security operations workflow	44
Figure 4.10: Layers of security controls	45
Figure 4.11: Deployment diagram	47
Figure 4.12: Landing page with the verification box.....	49
Figure 4.13: Live analysis progress showing the four signals	49
Figure 4.14: Result gated until the visitor signs in	50
Figure 4.15: Registration dialog	50
Figure 4.16: Result card with level, summary and highlighted cues	51
Figure 4.17: Full result: evidence per signal and recommended actions	52
Figure 4.18: Sender-origin card shown to the user	53
Figure 4.19: User dashboard	54
Figure 4.20: Analysis history	54

Figure 4.21: Connected mailboxes with live scan	55
Figure 4.22: Mobile view of a result (PWA)	56
Figure 4.23: Mobile view of the dashboard	57
Figure 4.24: Awareness quiz.....	58
Figure 4.25: Quiz feedback explaining the cues	58
Figure 4.26: Plain-language privacy page	59
Figure 4.27: Administrator overview.....	60
Figure 4.28: Threat radar: origins of dangerous e-mails.....	60
Figure 4.29: Threat radar with a selected source	61
Figure 4.30: Origin investigation (administrator view)	61
Figure 4.31: Hop-by-hop path of an e-mail	62
Figure 4.32: User management with registration risk.....	63
Figure 4.33: Detail of a pending registration	63
Figure 4.34: Review queue with triage suggestion.....	64
Figure 4.35: Correlated incidents.....	64
Figure 4.36: Incident detail with timeline.....	65
Figure 4.37: Threat intelligence and blocklist	66
Figure 4.38: Allowlist of official domains with health checks	66
Figure 4.39: System health	67
Figure 4.40: Audit log.....	67
Figure 4.41: Platform settings.....	68
Figure 4.42: Precision, recall and F1-score per configuration.....	72
Figure 4.43: False-positive rate per configuration	72

List of Acronyms and Abbreviations

AI	Artificial Intelligence
ANTIC	Agence Nationale des Technologies de l'Information et de la Communication
API	Application Programming Interface
ASN	Autonomous System Number
CIRT	Computer Incident Response Team
CSRF	Cross-Site Request Forgery
DKIM	DomainKeys Identified Mail
DMARC	Domain-based Message Authentication, Reporting and Conformance
DNS	Domain Name System
DNSBL	DNS-based Blackhole List
DSR	Design Science Research
ERD	Entity-Relationship Diagram
FCFA	Franc de la Coopération Financière en Afrique centrale
FPR	False-Positive Rate
FR / NFR	Functional / Non-Functional Requirement
HTTP(S)	HyperText Transfer Protocol (Secure)
IMAP	Internet Message Access Protocol
IOC	Indicator of Compromise
IP	Internet Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MFA	Multi-Factor Authentication
ML	Machine Learning
MoMo	Mobile Money (MTN)
OAuth	Open Authorization
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
OTP	One-Time Password
PWA	Progressive Web App
SMART	Specific, Measurable, Achievable, Relevant, Time-bound
SMS	Short Message Service
SOC	Security Operations Centre
SPF	Sender Policy Framework

SQL	Structured Query Language
TF-IDF	Term Frequency - Inverse Document Frequency
TLD	Top-Level Domain
TLS	Transport Layer Security
TOTP	Time-based One-Time Password
UCI	University of California, Irvine
UI / UX	User Interface / User Experience
URL	Uniform Resource Locator

Chapter 1: General Introduction

1.1 Introduction

Digital technology has transformed daily life in Cameroon. Mobile phones are now the main gateway to banking, trade, administration and communication, and mobile-money services such as MTN Mobile Money (MoMo) and Orange Money have brought millions of previously unbanked people into the digital economy. This progress has a dark side: every new digital service creates a new opportunity for fraudsters. Among all forms of cybercrime, **phishing** — the use of deceptive messages to trick a person into revealing secrets, sending money or opening a malicious link — remains the cheapest to run and the most effective, because it attacks the human rather than the machine.

This dissertation presents **PhishGuard-AI**, a web platform designed and implemented to help the Cameroonian public detect phishing messages and learn to recognise them. The platform combines machine learning, a large language model, link analysis and expert rules adapted to the local context, and it wraps them in an experience that a non-specialist can use from a phone in a few seconds. This first chapter sets out the background of the study, the problem it addresses, the objectives and research questions, and the scope, significance and organisation of the work.

1.2 Background to the Study

Phishing is not new. The term appeared in the mid-1990s when attackers impersonated America Online staff to steal account passwords. Since then, it has grown into an industrialised activity: phishing kits are sold ready-made, messages are sent by the million, and generative AI now allows attackers to write fluent, personalised lures in any language. The Anti-Phishing Working Group records more than a million phishing attacks per quarter worldwide (APWG, 2024), and industry reports consistently rank phishing and pretexting among the top initial access vectors in data breaches (Verizon, 2024).

In Africa, the problem is amplified by three factors. First, the growth of mobile money has created a large population of users who hold value in an account protected only by a PIN. Second, awareness of cybersecurity remains low outside professional circles. Third, the defensive tools available — secure e-mail gateways, browser warnings, antivirus software — were designed for English-language corporate e-mail and do not understand local brands, local

languages or local channels such as SMS and WhatsApp. INTERPOL's African Cyberthreat Assessment identifies online scams and phishing as the most prominent cyber threats on the continent (INTERPOL, 2023).

In Cameroon, the national agency for information and communication technologies (ANTIC) regularly warns the public against fraudulent messages imitating operators, banks and public institutions. Typical Cameroonian lures include: a message claiming that a MoMo account will be suspended unless the PIN is "confirmed" on a link; an ENEO electricity bill that must be paid "today before 5 p.m."; a fake bank security alert from Afriland First Bank or UBA; a lottery prize that requires a "processing fee"; a job offer that pays 50,000 FCFA a day for liking videos; or a relative who has "changed number" and urgently needs money. These messages are written in French, English or a mixture of both, often contain phone numbers rather than links, and exploit urgency, fear and greed.

At the same time, recent advances in natural-language processing make it possible to build detectors that understand the content of a message and not only its technical headers. Classical text classifiers are fast and cheap; large language models (LLMs) such as Claude can reason about context and intent; and URL-analysis techniques can reveal fresh domains, look-alike brand names and redirections. None of these techniques is sufficient alone, but together they can provide a reliable and explainable verdict. PhishGuard-AI was designed on this premise.

1.3 Problem Statement

Cameroonian citizens receive phishing messages daily through SMS, WhatsApp and e-mail, but have no simple, reliable and locally adapted tool to verify whether a message is a scam before acting on it. Existing tools suffer from four gaps:

- **Context gap:** detectors are trained on English corporate e-mail and ignore Cameroonian brands (MTN, Orange, ENEO, CAMWATER, local banks), French-language cues and mobile-money scenarios.
- **Channel gap:** most tools protect a corporate mailbox; they do not accept a pasted SMS, a WhatsApp screenshot or a forwarded message from an ordinary citizen.
- **Explanation gap:** when a tool blocks a message it rarely explains why, so the user learns nothing and remains vulnerable to the next attack.

- **Operations gap:** there is no local platform that aggregates reports from the public, correlates campaigns and gives analysts a view of who is being targeted and from where.

As a consequence, victims lose money and credentials, trust in digital services erodes, and institutions have little visibility on ongoing campaigns. The problem this study addresses is therefore **the absence of an accessible, explainable and context-aware phishing detection and awareness platform for the Cameroonian public.**

1.4 Objectives of the Study

1.4.1 General objective

The general objective of this study is to **design, implement and evaluate an AI-based web platform that detects phishing messages targeting Cameroonian users and raises their awareness of phishing techniques.**

1.4.2 Specific objectives

1. **Build a hybrid detection engine** that fuses a machine-learning text model, a large-language-model analysis, URL intelligence and Cameroon-specific rules, and reaches an F1-score of at least 0.85 with a false-positive rate below 15 % on held-out Cameroonian messages, by the end of the third development iteration.
2. **Develop a multi-channel web platform** (responsive web application and installable PWA) that accepts pasted text, links, screenshots (OCR), forwarded e-mails and read-only Gmail/Outlook mailboxes, and returns an explained verdict in under 10 seconds in the median case.
3. **Integrate an awareness component** — plain-language explanations with highlighted cues, recommended actions and an interactive quiz on local scam scenarios — available to every registered user.
4. **Provide a secured administration and security-operations module** (user approval, review queue, incident correlation, threat map with sender-origin tracing, audit log and sudo mode) that protects accounts and data according to recognised security principles.

1.5 Research Questions

In line with the guidelines, this study is driven by research questions rather than hypotheses. The main research question is:

How can artificial intelligence be combined with local threat knowledge to build an accessible platform that reliably detects phishing messages targeting Cameroonian users and improves their ability to recognise them?

It is broken down into the following specific research questions, each mapped to one specific objective:

1. Which combination of detection signals (text model, LLM, URL intelligence, expert rules) gives the best balance between detection rate and false alarms on Cameroonian phishing messages?
2. How can the platform accept the channels actually used by Cameroonians (SMS, WhatsApp, e-mail, screenshots) while respecting the privacy of their data?
3. How can a detection result be presented so that it both protects and educates a non-specialist user?
4. Which security controls are required so that a platform that stores sensitive messages and mailbox access does not itself become a target?

1.6 Significance of the Study

The study is significant at several levels:

- **For citizens:** PhishGuard-AI offers a free, fast and understandable way to check a suspicious message before paying, clicking or sharing a PIN, and teaches them to recognise the next scam on their own.
- **For institutions and operators:** the platform aggregates public reports into incidents, exposes the brands being impersonated and the infrastructure used, and can feed blocklists — a resource that CIRTs, banks and mobile-money operators can use.
- **For research:** the study documents a reproducible way to evaluate phishing detectors on a local, French-language, mobile-money-oriented corpus, and shows empirically that signal fusion reduces false positives.

- **For the author:** the project integrates the main domains of the cybersecurity curriculum — secure software development, identity and access management, cryptography, network forensics, machine learning and security operations.

1.7 Scope and Delimitation of the Study

Geographic and linguistic scope. The platform targets users in Cameroon and handles French and English messages. Local rules and brand lists cover Cameroonian operators, banks and utilities; the approach can be extended to other CEMAC countries.

Functional scope. Included: analysis of text, URLs, screenshots and e-mails (pasted, uploaded as .eml, forwarded, or fetched from a connected Gmail/Outlook mailbox); visitor analysis with gated result; user dashboard and history; awareness quiz; administrator SOC functions; sender-origin tracing for e-mails. Excluded: automatic interception of SMS on the handset (Android/iOS restrict SMS access), voice-phishing (vishing) detection, blocking of messages at the operator level, and malware analysis of attachments.

Temporal scope. The work was carried out during the 2025/2026 academic year in four iterations. The evaluation presented in Chapter 4 is a technical, offline evaluation; a field study with end users is planned but not yet completed.

1.8 Definition of Key Terms

Term	Definition used in this study
Phishing	A social-engineering attack in which a message impersonates a trusted entity to obtain money, credentials or the execution of a harmful action.
Smishing	Phishing delivered by SMS (and, by extension, messaging apps such as WhatsApp).
Mobile money	An electronic wallet tied to a mobile phone number (e.g. MTN MoMo, Orange Money) that allows transfers and payments.
Verdict	The classification returned by PhishGuard-AI: safe, suspicious or phishing, with a risk level (Low, Medium, High, Critical).
Risk score	A number from 0 to 100 produced by fusing the detection signals; above 40 a message is flagged.
Signal	An independent detector (text model, AI, URL intelligence, rules) that produces its own score and evidence.
False positive	A legitimate message wrongly flagged as dangerous.
Large Language Model	A neural network trained on very large text corpora and able to reason about text, e.g. Claude by Anthropic.
Origin tracing	Reconstructing the path of an e-mail from its Received headers to estimate the location and network of the sender.

Term	Definition used in this study
SOC	Security Operations Centre: the function that monitors, triages and responds to security events.

Table 1.1: Definition of key terms

1.9 Organisation of the Dissertation

The remainder of this dissertation is organised as follows. **Chapter 2** reviews the literature on phishing, detection techniques and awareness, presents the conceptual and theoretical framework and identifies the research gap. **Chapter 3** describes the research methodology: the Design Science Research approach, the data gathering, the corpus and sample, the quantitative evaluation method and the tools and techniques used. **Chapter 4** presents the analysis, design, implementation and testing of PhishGuard-AI and reports the findings. **Chapter 5** (to be presented separately) will discuss the findings, conclude and make recommendations.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the knowledge on which PhishGuard-AI is built. It first defines the conceptual framework of the study, then presents the theories that explain why people fall for phishing and how a detection system should be designed. It then reviews the empirical literature on detection techniques — from blocklists to machine learning and large language models — and on awareness training, before comparing existing solutions and identifying the gap this work fills.

2.2 Conceptual Framework

The conceptual framework (Figure 2.1) links the problem, the intervention and the expected outcomes. The **independent variables** are the characteristics of a message: its text (urgency, requests for PIN or payment, impersonated brand), its links (domain age, look-alike spelling, blocklist status) and, for e-mails, its technical origin (sending IP, authentication results). The **intervention** is the PhishGuard-AI platform, which fuses four detection signals and presents the result with an explanation. The **dependent variables** are the detection quality (precision, recall, F1, false-positive rate), the response time, and the user's awareness, measured by the ability to recognise scams in the quiz. **Moderating variables** include the channel (SMS, WhatsApp, e-mail), the language and the user's digital literacy.

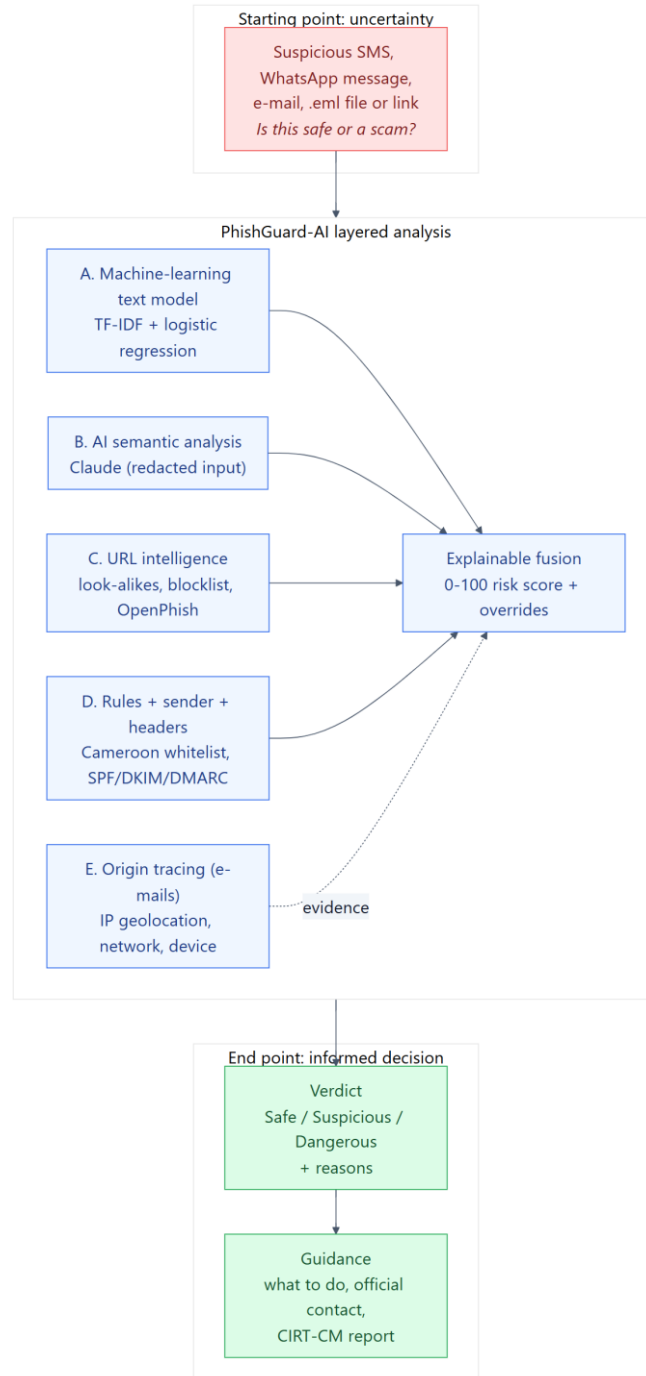


Figure 2.1: Conceptual framework of the study

2.2.1 Phishing and its variants

Phishing is defined by the Anti-Phishing Working Group as "a crime employing both social engineering and technical subterfuge to steal consumers' personal identity data and financial account credentials" (APWG, 2024). Variants are named after their channel or target: **spear phishing** targets a specific person with personalised content; **whaling** targets executives;

smishing uses SMS; **vishing** uses voice calls; **quishing** hides the link in a QR code; and **business e-mail compromise** impersonates a colleague or supplier to redirect payments. In Cameroon the dominant forms are smishing and WhatsApp scams linked to mobile money, followed by e-mail phishing against bank and webmail accounts.

2.2.2 Anatomy of a phishing message

Whatever the channel, a phishing message combines three elements (Jakobsson & Myers, 2007): a **pretext** (a trusted sender, e.g. "MTN MoMo"), a **trigger** (an emotion such as fear of losing access, urgency or greed) and a **call to action** (click a link, send a PIN, pay a fee, call a number). Detection can target each element: the pretext through brand and sender analysis, the trigger through linguistic cues, and the call to action through URL and payment-request analysis. PhishGuard-AI's rule set and explanations are organised along these three elements.

2.2.3 E-mail authentication and origin

E-mail was designed without sender authentication. Three standards were added later: **SPF** (RFC 7208) lets a domain list the servers allowed to send its mail; **DKIM** (RFC 6376) adds a cryptographic signature to the message; and **DMARC** (RFC 7489) tells receivers how to treat mail failing both checks and aligns them with the visible From domain. Each relay also adds a *Received* header, so the chain of headers records the path of the message. Reading this chain from the bottom up reveals the first public IP address — often the attacker's server or a compromised host — which can be geolocated and checked against DNS blacklists. Large webmail providers such as Gmail hide the user's IP, which limits this technique.

2.3 Theoretical Framework

2.3.1 Protection Motivation Theory

Protection Motivation Theory (Rogers, 1975) explains that people adopt protective behaviour when they perceive a threat as severe and likely (threat appraisal) and believe that a recommended response is effective and that they are able to perform it (coping appraisal). For PhishGuard-AI this implies that a warning alone is not enough: the result card must show *why* the message is dangerous (severity) and give simple, concrete actions ("do not send your PIN, call 8787 from your MTN line") that the user feels able to follow.

2.3.2 Dual-process theory and the psychology of persuasion

Dual-process theory distinguishes fast, automatic thinking from slow, deliberate thinking (Kahneman, 2011). Phishing succeeds by pushing victims into the fast mode through urgency and emotion. Cialdini's (2006) principles of persuasion — authority, scarcity, social proof, reciprocity, liking and commitment — describe the levers used. Vishwanath et al. (2011) showed empirically that urgency cues and habitual e-mail use strongly predict victimisation. The platform therefore highlights these cues in the message itself, to move the user back to deliberate thinking.

2.3.3 Design Science Research

Design Science Research (DSR) is a paradigm in which knowledge is produced by building and evaluating an artefact that solves a real problem (Hevner et al., 2004). Peffers et al. (2007) propose a six-activity process: identify the problem, define objectives, design and develop, demonstrate, evaluate and communicate. DSR is the methodological foundation of this study (Chapter 3).

2.3.4 Security design principles

Saltzer and Schroeder (1975) formulated principles that still guide secure systems: economy of mechanism, fail-safe defaults, complete mediation, open design, separation of privilege, least privilege, least common mechanism and psychological acceptability. Cavoukian (2009) added *privacy by design*: privacy as the default setting, embedded in the design and respecting the user. These principles are applied explicitly in the design of PhishGuard-AI (Section 4.3.8), e.g. read-only mailbox scopes (least privilege), results hidden from anonymous visitors (fail-safe default) and storage of only the subject of safe e-mails (data minimisation).

2.4 Empirical Review

2.4.1 List-based detection

The oldest defence is the blocklist: browsers and gateways check each URL against lists such as Google Safe Browsing, PhishTank or OpenPhish. Lists are precise but reactive: Sheng et al. (2009) found that most lists caught fewer than 20 % of phishing URLs at hour zero, while campaigns typically last only a few hours. Lists also do not help against messages that contain no link — very common in mobile-money scams that ask the victim to call or send money to a number.

2.4.2 Heuristic and rule-based detection

Heuristic systems score messages using handcrafted features: presence of an IP address in a URL, number of dots or hyphens, suspicious keywords, mismatch between displayed and real link. CANTINA (Zhang et al., 2007) used TF-IDF keywords of a web page to verify its identity with a search engine. Rules are transparent and cheap, and they encode expert knowledge that data may not contain (e.g. "MTN never asks for your PIN"); but they are brittle and, used alone, detect only the patterns their authors thought of. Our own evaluation confirms this: rules alone reach a precision of 1.0 but a recall of only 0.064 on Cameroonian messages (Chapter 4).

2.4.3 Machine-learning detection

Machine learning learns patterns from labelled examples. Fette et al. (2007) achieved over 96 % accuracy on e-mail with ten features and a random forest. For SMS, Almeida et al. (2011) published the **SMS Spam Collection** (5,574 messages) and showed that linear models on bag-of-words features perform very well. TF-IDF weighting (Manning et al., 2008) combined with linear classifiers such as logistic regression or support vector machines remains a strong baseline, fast and interpretable. Character n-grams make the model robust to misspellings and deliberate obfuscation ("M0M0", "c0nfirmez"), common in local scams. For URLs, Sahingoz et al. (2019) and others report above 95 % accuracy with lexical features and gradient-boosted trees.

The main weakness of ML is its dependence on the training distribution: a model trained on English spam performs poorly on French mobile-money scams, and a model trained on a narrow corpus overreacts to words such as "compte" or "code" that also appear in legitimate transaction notifications. This explains the high false-positive rate (31.9 %) of our text model alone, and motivates signal fusion.

2.4.4 Deep learning and large language models

Deep models (CNN, LSTM, then transformers such as BERT) improved detection by learning contextual representations, at the cost of large data needs and computing resources. Large language models go further: they can be asked, without task-specific training, to judge whether a message is phishing and to explain why. Koide et al. (2024) showed that GPT-4 class models detect phishing e-mails with high accuracy and useful explanations; Heiding et al. (2024) showed that LLMs are also used by attackers to write convincing phishing and that they can detect it well. Their limits are latency (seconds per request), cost, dependency on an external provider, and possible hallucination — hence the choice in PhishGuard-AI to use the LLM as

one weighted signal among four, never as the only judge, and to keep the platform functional when the LLM is unavailable.

2.4.5 Awareness and training

Technical controls cannot block every message, so users remain the last line of defence. Kumaraguru et al. (2010) showed with PhishGuru that embedded training — teaching at the moment the user interacts with a phishing message — improves retention compared with notices. Sheng et al. (2010) found that education significantly reduced susceptibility across demographics, and that younger users were the most vulnerable. Games such as Anti-Phishing Phil improved discrimination between legitimate and fraudulent URLs. These results justify PhishGuard-AI's design choice to **explain every verdict** by highlighting the cues in the message and to offer a quiz on local scenarios.

2.4.6 Studies in the African and Cameroonian context

Research on phishing in Africa remains limited and mostly descriptive. INTERPOL (2023) ranks online scams and phishing as the top threat in Africa and notes the role of mobile money. Studies on mobile-money fraud in East and West Africa describe social-engineering schemes — fake reversal requests, fake agents, "wrong transfer" calls — similar to those observed in Cameroon. Very few works propose a detection tool trained or evaluated on French-language, Central African messages, and none, to our knowledge, combines detection, awareness and security operations for the Cameroonian public.

2.4.7 Typology of scams observed in Cameroon

From the document analysis and the messages collected for this study, the scams circulating in Cameroon can be grouped into families that share a pretext, a trigger and a call to action (Table 2.1). This typology was used to design the corpus (Chapter 3), the expert rules and the quiz scenarios. Two observations matter for detection. First, several families contain **no link at all**: the victim is asked to call a number, to send money or to read a code to a fake agent. URL-based defences are blind to these. Second, legitimate messages from the same brands use the same vocabulary — "compte", "code", "solde", "transaction" — which makes naive keyword or bag-of-words detection prone to false alarms.

Family	Typical pretext and trigger	Call to action	Contains a link?
Account suspension	MTN MoMo / Orange Money: "your account will be blocked within 24 h"	Confirm PIN on a site	Usually (look-alike domain)

Family	Typical pretext and trigger	Call to action	Contains a link?
PIN / code request	"An agent will call you, give him the code you receive"	Read an OTP to the fraudster	No
Fake relative	"Mum, it is me, I changed number, send 25 000 FCFA urgently"	Send money	No
Fake bill	ENEO / CAMWATER: unpaid bill, cut-off today at 5 p.m.	Pay on a site or to a number	Often
Bank alert	Afriland, BICEC, UBA: suspicious activity, card blocked	Log in on a fake portal	Yes
Prize / lottery	"Your number won 500 000 FCFA in the Orange tombola"	Pay a processing fee	Sometimes
Job / investment	"Earn 50 000 FCFA a day liking videos"	Pay a registration fee, join WhatsApp	Sometimes
Delivery / customs	"Your parcel is held at customs"	Pay fees on a site	Yes
Government refund	"Tax refund from the DGI / subsidy"	Enter card details	Yes

Table 2.1: Typology of phishing and scam messages observed in Cameroon

2.4.8 Ensemble methods and explainability

Combining several classifiers is a classical way to improve robustness: when detectors make different kinds of errors, a weighted vote or average reduces the overall error (Dietterich, 2000). In phishing detection, ensembles that mix content, URL and header features consistently outperform single-view models, because an attacker must defeat all views simultaneously. Two families of combination exist: **learned** fusion (stacking, where a meta-classifier learns the weights) and **fixed** fusion (weights set by experts). Learned fusion needs a large labelled set covering all signal combinations — which does not exist for Cameroonian messages — so this study uses fixed weights with deterministic overrides, and evaluates them empirically.

Explainability is equally important. Doshi-Velez and Kim (2017) argue that interpretability is required whenever a system's decisions affect people who must understand or contest them. For a citizen deciding whether to pay a bill, a bare score is useless; a list of reasons ("the link imitates MTN but is registered under .tk", "the message asks for your PIN") is both a justification and a lesson. Each PhishGuard-AI signal therefore returns human-readable evidence along with its score, and the fusion keeps track of which signal produced which piece of evidence.

2.5 Review of Existing Solutions

Solution	Local context (FR, MoMo)	SMS / screenshot input	Explains verdict	Awareness	SOC / campaign view
Google Safe Browsing	No	No (URL only)	No	No	No
Microsoft Defender for Office 365	No	No (corporate mail)	Partial	Attack simulation (paid)	Yes (enterprise)
PhishTank / OpenPhish	No	No (URL only)	No	No	Feed only
VirusTotal	No	No (URL/file)	Partial (engine names)	No	No
Truecaller / operator SMS filters	Partial	Yes (on device)	No	No	No
KnowBe4 (training)	No	No	—	Yes (paid)	Partial
PhishGuard-AI	Yes	Yes (text, link, OCR, e-mail, mailbox)	Yes (highlighted cues + actions)	Yes (quiz, tips)	Yes (incidents, radar, blocklist)

Table 2.2: Comparison of existing solutions with PhishGuard-AI

The comparison in Table 2.2 shows that each existing product answers one part of the problem. Browser and list-based services protect a user who clicks a link, but only once the domain has been reported, and they do not handle a message without a link. Enterprise e-mail security suites are powerful, but they protect a corporate mailbox that the typical Cameroonian mobile-money user does not have, and they are priced for organisations. On-device SMS filters classify senders rather than content and give no explanation. Training platforms teach, but do not help at the moment a suspicious message arrives. None of them is designed around Cameroonian brands, French-language cues or the mobile-money scam families of Table 2.1.

2.6 Research Gap

The review shows that the existing solutions are either (a) generic and technically strong but unaware of the Cameroonian context and inaccessible to ordinary citizens, or (b) awareness programmes without detection. Academic works focus on improving one detection technique on English datasets. The gap addressed by this study is the combination of **(1)** a hybrid, explainable detector evaluated on local messages, **(2)** a multi-channel citizen interface, **(3)**

embedded awareness, and **(4)** a security-operations layer that turns individual reports into collective intelligence (Figure 2.2).

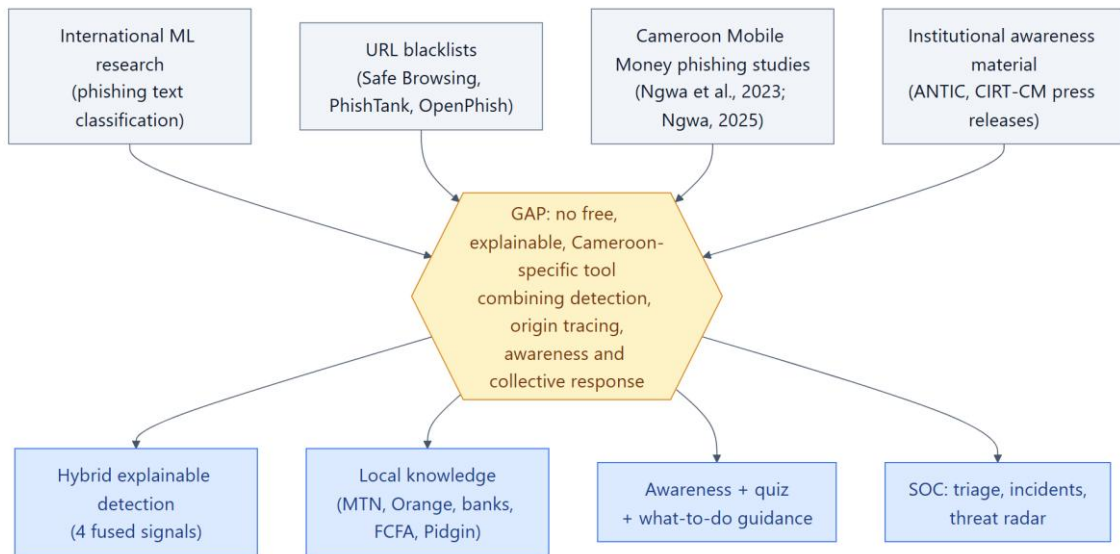


Figure 2.2: Positioning of PhishGuard-AI with respect to the research gap

2.7 Summary

Phishing exploits predictable human reactions, and each detection technique has a clear weakness when used alone: lists are late, rules are narrow, ML depends on its data and LLMs are slow and external. Awareness works best when it is embedded in the moment of decision. These findings shape the design of PhishGuard-AI: a fusion of complementary signals, an explanation for every verdict, and a platform built for the channels and language of Cameroonian users. The next chapter describes how the artefact was built and evaluated.

Chapter 3: Research Methodology

3.1 Introduction

This chapter describes how the study was conducted: the research design, the development process, how data were gathered, the population and sample used to build and evaluate the detector, the quantitative analysis method, and the processes, techniques and tools used to implement the platform. The aim is to make the work reproducible and its results verifiable.

3.2 Research Design

The study adopts **Design Science Research** (DSR), a problem-solving paradigm in which the research contribution is an artefact — here, the PhishGuard-AI platform — together with the knowledge gained by building and evaluating it (Hevner et al., 2004). DSR fits this project because the research questions ask *how* a solution can be built and *how well* it performs, rather than *why* a social phenomenon occurs. The study follows the six activities of Peffers et al. (2007), shown in Figure 3.1.

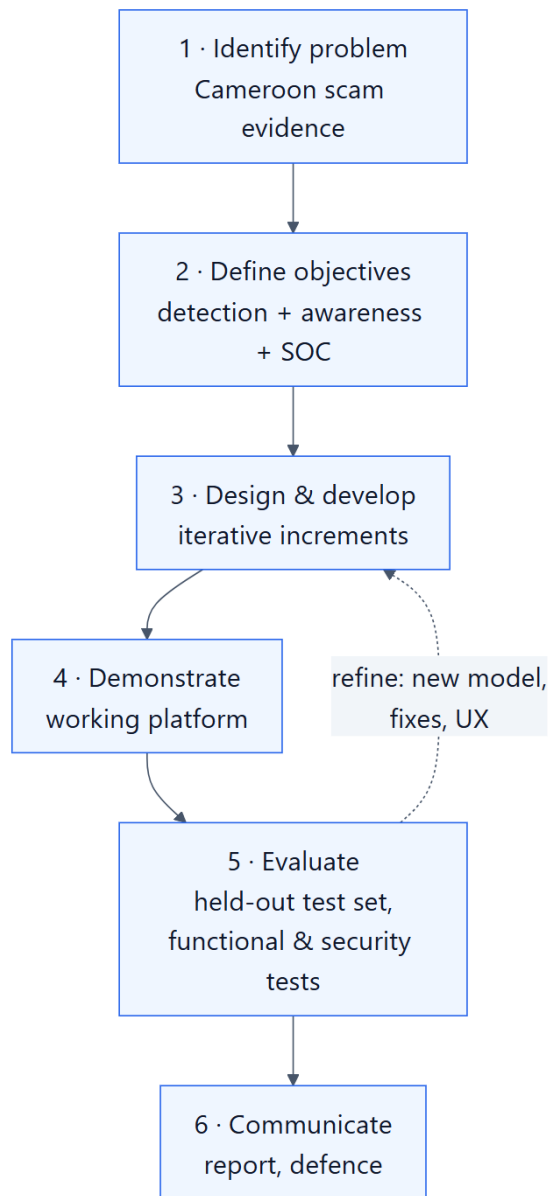


Figure 3.1: Design Science Research process applied to PhishGuard-AI (after Peffers et al., 2007)

DSR activity	What was done in this study
1. Problem identification	Literature review, review of ANTIC/INTERPOL alerts, collection of real scam examples circulating in Cameroon (Chapters 1–2).
2. Objectives of a solution	One general and four SMART specific objectives; functional and non-functional requirements (Section 4.2).
3. Design and development	Four iterative development cycles producing the platform (Section 3.3, Section 4.3–4.4).
4. Demonstration	Use of the platform on real scenarios: pasted SMS, screenshots, forwarded e-mails, connected Gmail mailbox (Section 4.4).

DSR activity	What was done in this study
5. Evaluation	Functional and security tests, and a quantitative offline evaluation on a held-out corpus with ablation of each signal (Section 4.5–4.6).
6. Communication	This dissertation, the technical documentation and the defence presentation.

Table 3.1: Application of the DSR activities in this study

3.3 Development Methodology

Within the DSR "design and development" activity, the software was built with an **iterative and incremental** approach inspired by Scrum: each iteration had a goal, produced a working version, and was evaluated before the next one. This allowed early feedback and the correction of weaknesses discovered by testing — for example, the high false-positive rate of the first text model led, in iteration 4, to a rebuilt corpus and a new model. Figure 3.2 summarises the four iterations.

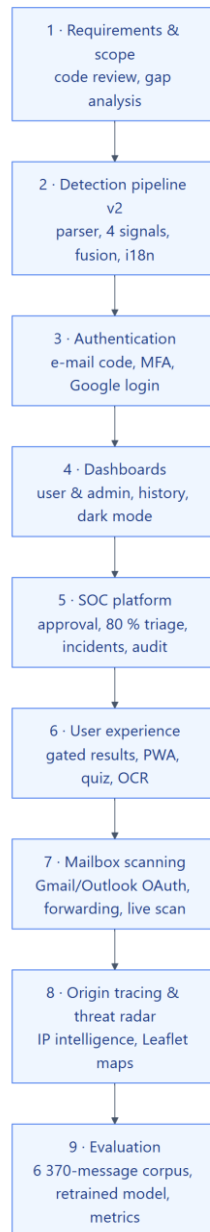


Figure 3.2: The four development iterations

Iteration	Goal and main deliverables	Evaluation at the end
1 — Foundation	Flask API, PostgreSQL schema, authentication (bcrypt, sessions, e-mail/SMS OTP, Google OAuth), first text classifier and rule engine, Vue 3 front end.	Unit tests of URL features; manual tests of the analysis flow.
2 — Detection pipeline	Four-signal pipeline (ML, Claude AI, URL intelligence, rules), fusion with weight redistribution, overrides, e-mail parser, OCR of screenshots.	Challenge set of hand-written messages; latency measurements.
3 — Platform and SOC	Visitor gate and claim tokens, user dashboard, quiz, PWA, Gmail/Outlook read-	Functional and security test cases (Section 4.5).

Iteration	Goal and main deliverables	Evaluation at the end
	only scanning, forward-to address, admin SOC: approvals, review queue, triage, incidents, blocklist, audit log, sudo mode, system health.	
4 — Intelligence and evaluation	Sender-origin tracing and threat radar, live mailbox scan, corpus rebuilt (6,370 messages), new TF-IDF + logistic-regression model, reproducible evaluation script with ablation.	Quantitative evaluation on held-out Cameroonian test set (Section 4.6).

Table 3.2: Content of the development iterations

3.4 Data Gathering

Two kinds of data were gathered: **requirements data**, to understand the problem and users; and **message data**, to build and evaluate the detector.

3.4.1 Requirements data

- **Document analysis:** public warnings by ANTIC and operators, INTERPOL and APWG reports, and the terms of service of mobile-money operators (e.g. that agents never ask for a PIN).
- **Observation:** collection of scam messages received by the author, family and classmates on SMS and WhatsApp (anonymised: phone numbers and names removed).
- **Informal interviews:** discussions with students and relatives on how they react to suspicious messages and what they would expect from a verification tool. The main needs expressed were: "tell me quickly if it is a scam", "explain in simple words", "works on my phone", and "do not ask me to install anything complex".

3.4.2 Message data

No public, labelled corpus of Cameroonian phishing messages exists. The message corpus was therefore assembled from four sources (Table 3.3), using a reproducible script (`ml_data/build_dataset.py`) that writes a single `corpus.csv` with the text, the label (phishing or legitimate), the source and a *template group* identifier.

Source	Messages	Description
Cameroonian templates (training)	586	Scam and legitimate messages written from real patterns (MoMo, Orange Money, ENEO, banks, jobs, lottery, "family emergency"; genuine transaction notifications, OTP codes, bills), expanded with variations of amounts, names, numbers and domains.

Source	Messages	Description
Cameroonian templates (test)	178	Messages generated from template groups that are never used in training (grouped hold-out).
Challenge set	24	Hand-written difficult cases: legitimate messages with alarming words, subtle scams without links, mixed French/English.
UCI SMS Spam Collection	5,574	Public English SMS dataset (Almeida et al., 2011): 747 spam, 4,827 legitimate.
Administrator labels	8	Messages reviewed and labelled by an administrator in the platform's review queue (automatic triage decisions excluded).
Total	6,370	

Table 3.3: Sources of the message corpus

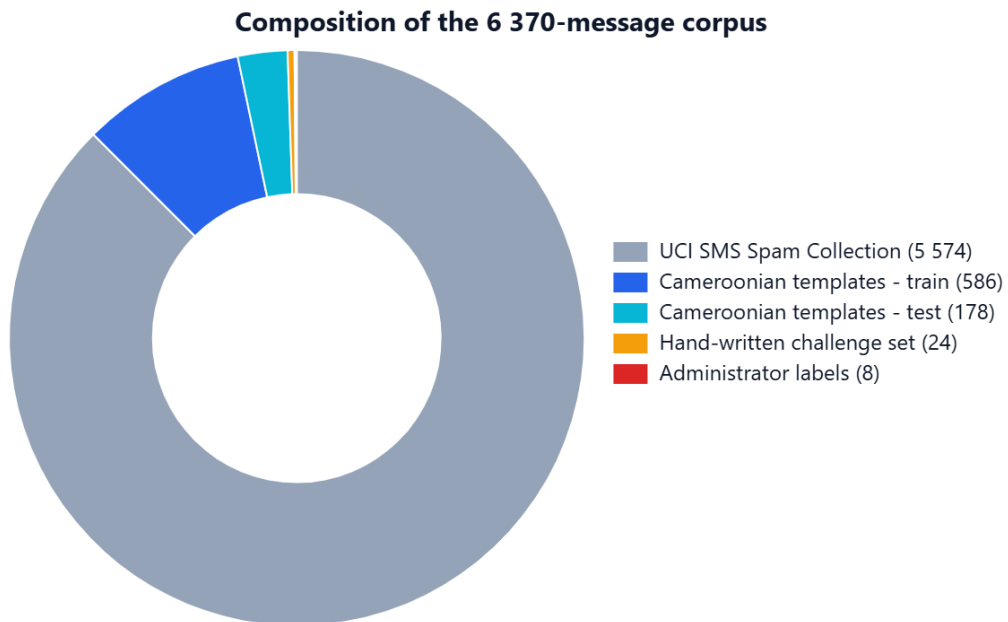


Figure 3.3: Composition of the 6,370-message corpus

3.4.3 Corpus construction procedure

The Cameroonian part of the corpus was built by *template expansion*, a technique used when real labelled data are scarce. The procedure is as follows:

1. **Write templates.** For each scam family of Table 2.1 and for each type of legitimate message likely to be confused with it, several templates were written in French, English and Cameroonian Pidgin, reproducing the structure, tone and typical errors of real messages. Each template contains *slots*, e.g. {op} for an operator, {bank}, {amt} for an amount, {name}, {phone} or {domain}.

2. **Fill the slots.** Each template is expanded into 14 variants by drawing random slot values from realistic lists (six operators, six banks, amounts from 5,000 to 1,000,000 FCFA, common Cameroonian first names, plausible look-alike domains). The random generator is seeded (seed 42), so the corpus is identical at every run.
3. **Include hard cases on purpose.** *Hard negatives* are genuine messages that mention codes, accounts, amounts or official links (e.g. a real Orange Money receipt or an OTP code). *Hard positives* are scams without any link (fake relative, fake agent). These cases prevent the model from learning shortcuts such as "a message with a code is a scam".
4. **Split by template.** 25 % of the templates are randomly assigned to the test set and all their variants go with them; the remaining templates form the training set.
5. **Add independent data.** The UCI SMS corpus is split 80/20 with stratification; the challenge set and the administrator labels are appended with their own source tag.
6. **Write one file.** Every row of `corpus.csv` carries its text, label, language, channel, family, template, source and split, so every metric can be reported per source.

Family	Label	Messages	Example (abridged)
Bank login	Scam	14	Afriland: suspicious activity, log in at afriland-secure-login.com
Delivery / customs	Scam	14	Your parcel is blocked, pay the customs fee
Fake relative	Scam	14	Mum, it is me, I changed number, send 25 000 FCFA
Government refund	Scam	14	DGI refund: enter your card details
PIN request	Scam	12	An agent will call you, give him the code
Prize	Scam	14	You won 500 000 FCFA, pay 5 000 FCFA fees
Utility bill	Scam	14	ENEO: unpaid bill, cut-off today
Appointment	Legitimate	14	Your appointment is confirmed for Thursday
Bank information	Legitimate	6	Your monthly statement is available in the app
MoMo receipt	Legitimate	14	Transfer received from ... New balance ...
Official link	Legitimate	20	Discover our new offers on the official site
Personal	Legitimate	14	I will be late, keep food for me
Work	Legitimate	14	The meeting is moved to Thursday 3 p.m.
Challenge set	Both	24	12 scams and 12 genuine messages written by hand
Administrator labels	Scam	2	Reviewed reports

Table 3.4: Families of the Cameroonian held-out test set

3.5 Population and Sample

Population. The target population of the study is the set of unsolicited messages (SMS, WhatsApp, e-mail) received by Cameroonian users, and, for the awareness component, the Cameroonian public who own a smartphone. This population is unbounded and cannot be enumerated.

Sample for the detector. The corpus above is a *purposive sample*: messages were selected to cover the main scam families observed in Cameroon and the legitimate messages most easily confused with them. The key methodological choice is the **grouped hold-out split**: all variants generated from the same template belong to the same group, and groups are assigned either to training or to testing. This prevents the model from being tested on near-copies of training messages, a common source of inflated results. The Cameroonian test set contains **204** messages (178 template variants plus the 24 challenge messages and 2 administrator labels kept out of training), of which 110 are phishing and 94 legitimate.

Sub-sample for the AI signal. Because each call to the Claude API costs money and takes several seconds, the full pipeline *with* AI was evaluated on a stratified random sub-sample of **40** test messages (24 phishing, 16 legitimate). This is acknowledged as a limitation: confidence intervals on this sub-sample are wider.

Sample for acceptance testing. A user-acceptance study with 30 participants (students and staff of The ICT University and members of the public in Yaoundé) is planned using the questionnaire in Appendix A. At the time of writing it has not yet been conducted; no results are reported for it.

3.6 Quantitative Analysis

The detector is evaluated as a binary classifier: a message is *flagged* if its risk score is above 40 (level Medium or higher). Each test message falls into one of four outcomes: true positive (TP, phishing flagged), false negative (FN, phishing missed), true negative (TN, legitimate not flagged) and false positive (FP, legitimate flagged). The following metrics are computed:

Metric	Formula	Meaning for the user
Precision	$TP / (TP + FP)$	When PhishGuard says "danger", how often is it right?
Recall (detection rate)	$TP / (TP + FN)$	What share of real scams is caught?

Metric	Formula	Meaning for the user
F1-score	$2 \cdot P \cdot R / (P + R)$	Balance of precision and recall (main metric, objective 1).
False-positive rate	$FP / (FP + TN)$	What share of genuine messages is wrongly flagged? (objective 1: below 15 %)
Latency p50 / p95	Median / 95th percentile of analysis time	How long the user waits (objective 2: median below 10 s).

Table 3.5: Evaluation metrics

To answer research question 1, an **ablation study** is performed: the same test set is scored by each signal alone (text model only, rules only, AI only) and by the full pipeline with and without the AI signal. Comparing these configurations isolates the contribution of each signal. The evaluation is fully automated by `evaluation/evaluate.py`, which writes machine-readable (`results.json`) and human-readable (`results.md`) reports, so that every figure in Chapter 4 can be regenerated.

3.6.1 Training procedure of the text model

The text model is trained by `train_ml.py` on the training split only, in five steps:

1. **Normalisation** of each message: lower-case; strip accents; replace URLs by URL, phone numbers by PHONE, amounts by AMOUNT and codes by CODE; map digits used as letters back to letters ("m0m0" → "momo"). This lets the model learn *that* a message contains an amount, not *which* amount.
2. **Word features:** TF-IDF on unigrams and bigrams appearing in at least two messages, with sub-linear term frequency.
3. **Character features:** TF-IDF on character 3- to 5-grams inside word boundaries, which captures spelling variants and obfuscation.
4. **Classifier:** logistic regression with regularisation $C = 1.5$ and balanced class weights, fitted on the union of both feature sets.
5. **Export:** the whole pipeline (normaliser, vectorisers, classifier) is saved in a single `classifier.joblib` file with its metadata (training date, corpus size, validation scores). The previous model file was kept as a backup (`classifier_legacy_10msgs.joblib`) so that a regression can be rolled back.

Logistic regression was preferred over more complex models for three reasons: it trains in seconds on a laptop, which allows frequent retraining from administrator reviews; its output is

a calibrated probability, which fits naturally into the weighted fusion; and its coefficients can be inspected to see which words or character patterns push a message towards "scam".

3.6.2 Threats to validity

- **Construct validity.** The Cameroonian test messages come from templates written by the author. Although test templates are never seen in training, they share the author's style; the challenge set and the UCI corpus are included to measure performance on independent text.
- **Internal validity.** The same threshold (40) is used for all configurations, and the thresholds were fixed before the final evaluation, so no configuration is tuned on the test set.
- **External validity.** Results on a synthetic corpus may not transfer to real traffic. The review queue of the deployed platform continuously collects real, human-labelled messages that will be used to re-evaluate the system.
- **Statistical conclusion validity.** The AI configuration was evaluated on 40 messages; one error changes its F1-score by about two points. These figures are indicative, not definitive.

3.7 Processes, Methods, Techniques and Tools

3.7.1 Modelling

The system was modelled with **UML** (use-case, class, sequence and activity diagrams) and an **entity-relationship diagram** for the database. The architecture follows a layered client–server model: a single-page application communicating with a REST API over HTTPS, the API delegating detection to a pipeline and persistence to an ORM.

3.7.2 Detection techniques

- **Text normalisation:** lower-casing, accent folding, replacement of URLs, phone numbers, amounts and codes by placeholder tokens, and de-obfuscation of digits used as letters.
- **TF-IDF vectorisation** on word unigrams/bigrams and character 3–5-grams, combined in one feature space; **logistic regression** ($C = 1.5$, balanced classes) as classifier, chosen for speed, calibrated probabilities and interpretability (Pedregosa et al., 2011).

- **Large-language-model analysis** with Claude through the Anthropic API, prompted to return a structured judgement (classification, category, confidence, reasons, recommendation) on a redacted copy of the message.
- **URL intelligence:** URL extraction and unshortening, lexical features, registrable-domain extraction (tldextract), look-alike detection against Cameroonian brands, TLD reputation, blocklist and OpenPhish feed lookup, and a gradient-boosted URL model (XGBoost).
- **Expert rules** encoding Cameroon-specific knowledge (PIN requests, payment of fees to receive a prize, fake bill cut-offs, number-change scams) in French and English.
- **Origin tracing:** parsing of Received headers, identification of the first public hop, IP geolocation, reverse DNS, DNS blacklist lookup and SPF/DKIM/DMARC result extraction.
- **OCR** of screenshots in the browser with Tesseract.js, so that images never need to leave the device.

3.7.3 Tools and technologies

Layer	Technology	Justification
Back end	Python 3.13, Flask 2.3, SQLAlchemy 1.4, pg8000	Mature, lightweight web framework; Python gives direct access to the ML ecosystem.
Database	PostgreSQL 18	Reliable relational database with JSON columns for evidence and timelines.
Machine learning	scikit-learn 1.9, XGBoost 2.0, joblib, NumPy, SciPy	Standard, well-documented libraries; models stored as files and loaded at start-up.
Artificial intelligence	Anthropic API (Claude), pydantic 2	State-of-the-art LLM with structured outputs; validated with pydantic schemas.
Security	bcrypt, Flask-Login, Flask-WTF, pyotp, Fernet (cryptography)	Password hashing, session management, CSRF protection, TOTP, token encryption.
Front end	Vue 3, TypeScript, Vite, Pinia, Tailwind CSS 4	Reactive, typed single-page application; fast builds; utility-first responsive design.
Maps and OCR	Leaflet (markercluster, heat), Tesseract.js	Open-source mapping without API key; in-browser OCR.
Integrations	Gmail API, Microsoft Graph, IMAP, SMTP, Twilio, ip-api.com, OpenPhish	Mailbox scanning, forwarding, notifications, geolocation and threat feed.
Engineering	Git/GitHub, VS Code, pytest, Postman, Playwright	Version control, testing of API and user interface, screenshots.
Modelling	Mermaid (UML), draw.io	Diagrams as code, versioned with the source.

Table 3.6: Tools and technologies used

3.7.4 Ethical considerations

No personal data of third parties are published in this dissertation: example messages are anonymised, screenshots were produced with demonstration accounts, and any real e-mail address visible on screen was masked. The platform itself follows data-minimisation rules (Section 4.3.8). Mailbox access requires the explicit OAuth consent of the owner and is read-only. Messages submitted by users are used for retraining only after administrator review, and users can delete their history at any time.

3.8 Summary

The study combines a design-science approach, an iterative development process and a quantitative, reproducible evaluation on a grouped hold-out split. The next chapter applies this methodology: it analyses the requirements, presents the design and the implementation of PhishGuard-AI, and reports the results of the tests and of the evaluation.

Chapter 4: Analysis, Design, Implementation and Findings

4.1 Introduction

This chapter presents the artefact produced by the study. It starts with the analysis of the system (actors and requirements), then describes the design (architecture, detection pipeline, data model, dynamic behaviour and security design), the implementation (with screenshots of the working platform), the testing, and finally the quantitative findings of the evaluation. In accordance with the guidelines, interpretation and conclusions are left to Chapter 5.

4.2 System Analysis

4.2.1 Analysis of the current situation

Today, a Cameroonian who receives a suspicious message has three options: ignore it, ask a relative, or act on it. None of them is reliable. Operators publish warnings on social networks, but they are generic and arrive after the campaign. When a person is defrauded, the report — if any — goes to the operator or the police and is not shared; no one has a consolidated view of campaigns. The proposed system addresses both sides: an instant verification service for citizens, and a consolidated intelligence view for analysts.

4.2.2 Actors

Actor	Description and main goals
Visitor	Anonymous person on the public site. Can read awareness content, play the quiz and submit a message; the result is kept hidden until sign-in (claim token).
Registered user	Citizen with an approved account. Analyses messages, links, screenshots and e-mails; connects Gmail/Outlook in read-only mode; reads history, statistics and a personal threat map; reports false results; manages security (MFA, sessions).
Administrator / analyst	Operates the SOC: approves risky registrations, reviews reported analyses, confirms incidents, manages the blocklist and allowlist, investigates sender origins, configures the platform, reads the audit log.
External systems	Claude API, Gmail API, Microsoft Graph, IMAP/SMTP servers, Twilio SMS, ip-api.com geolocation, DNS blacklists, OpenPhish feed.
Scheduler (system actor)	Background thread that synchronises mailboxes, reads forwarded e-mails, correlates incidents, refreshes the threat feed and checks allowlisted domains.

Table 4.1: Actors of the system

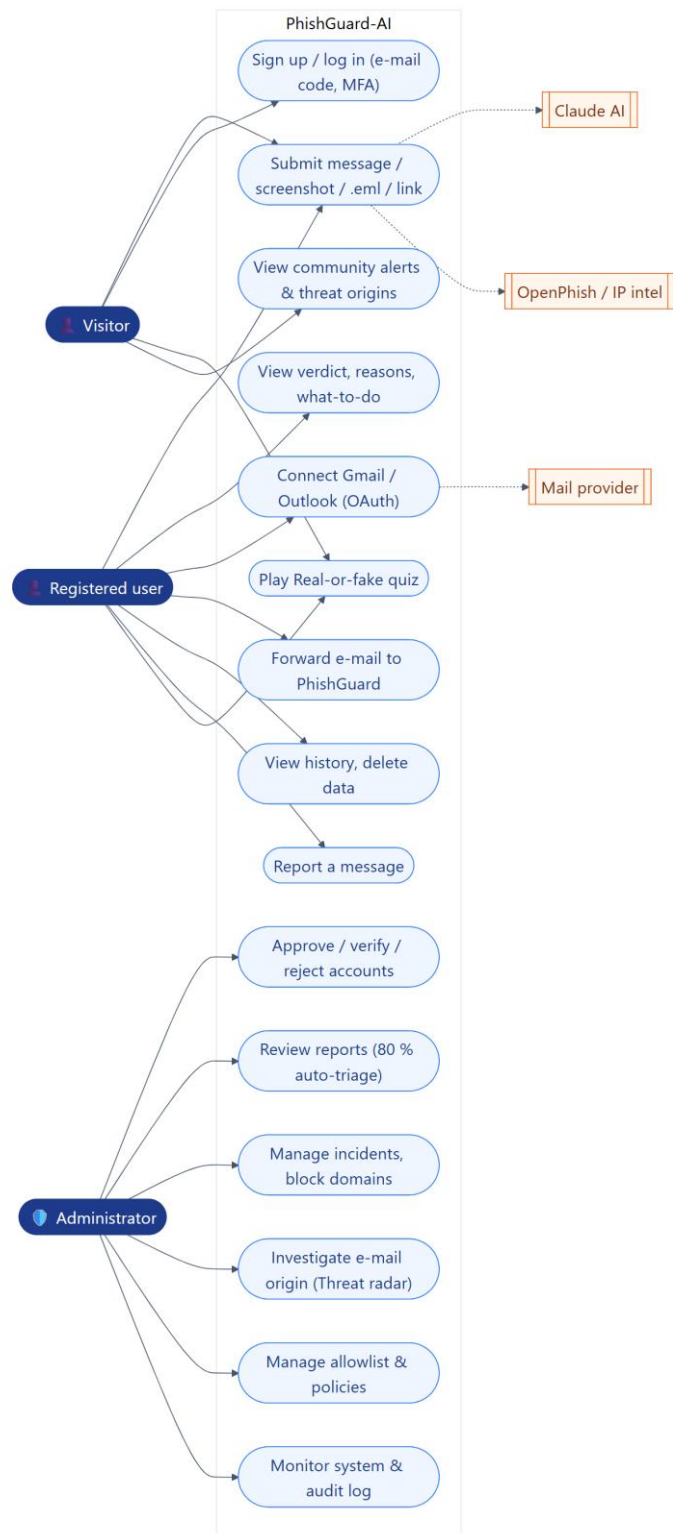


Figure 4.1: Use-case diagram of PhishGuard-AI

4.2.3 Functional requirements

ID	Requirement	Status
FR1	Analyse pasted text (SMS, WhatsApp, e-mail) in French or English and return a score, level, verdict and explanation.	Implemented
FR2	Analyse links: expansion, domain intelligence, look-alike detection, blocklist and threat-feed lookup.	Implemented
FR3	Extract text from screenshots (OCR) on the device and analyse it.	Implemented
FR4	Analyse e-mails from an uploaded .eml file, a forwarded e-mail, or a connected Gmail/Outlook mailbox (read-only OAuth).	Implemented
FR5	Trace the origin of an e-mail sender (IP, city, network type, blacklists) with a simplified view for users and a full view for administrators.	Implemented
FR6	Let visitors submit a message but reveal the result only after sign-in or registration.	Implemented
FR7	Register and authenticate users (password, Google, e-mail/SMS OTP, TOTP MFA) with risk-based approval of new accounts.	Implemented
FR8	Provide a user dashboard: history, statistics, threat map, mailbox status with live scan, deletion of data.	Implemented
FR9	Provide awareness content: highlighted cues, recommended actions, read-aloud, quiz with scoring.	Implemented
FR10	Let users report a wrong verdict; queue reports for administrator review with an engine suggestion (triage).	Implemented
FR11	Correlate dangerous messages into incidents and feed the blocklist.	Implemented
FR12	Offer administrators a threat radar, system health, audit log, settings and user management.	Implemented
FR13	Retrain the text model from reviewed data and evaluate it reproducibly.	Implemented (offline scripts)
FR14	Publish anonymised public alerts on the landing page.	Implemented
FR15	Intercept SMS directly on the handset.	Out of scope

Table 4.2: Functional requirements and implementation status

4.2.4 Non-functional requirements

ID	Quality	Requirement	Status
NFR1	Performance	Median analysis under 10 s with AI, under 1 s without.	Met (7.2 s / 10 ms)
NFR2	Availability	The platform keeps working if the AI, URL model or a feed is unavailable (graceful degradation).	Met
NFR3	Security	Passwords hashed with bcrypt; CSRF protection; session invalidation; sudo mode for sensitive admin actions; audit of every security event.	Met
NFR4	Privacy	Read-only mailbox scopes, encrypted tokens, safe e-mails stored as subject only, users can delete history, results never shown to anonymous callers.	Met

ID	Quality	Requirement	Status
NFR5	Usability	Responsive from 360 px, installable PWA, share target, bilingual FR/EN, plain-language explanations.	Met
NFR6	Explainability	Every verdict lists the evidence and the signal that produced it.	Met
NFR7	Maintainability	Modular services, one job each; reproducible training and evaluation scripts; automated tests.	Met
NFR8	Scalability	Stateless API behind a WSGI server (gunicorn) and PostgreSQL.	Designed, not load-tested

Table 4.3: Non-functional requirements

4.3 System Design

4.3.1 Architecture

PhishGuard-AI follows a three-tier architecture (Figure 4.2). The **presentation tier** is a Vue 3 single-page application, also installable as a PWA. The **application tier** is a Flask REST API organised in blueprints (analysis, auth, OAuth, OTP, user, mailbox, admin, SOC, public), a detection pipeline and a set of services, each with a single responsibility. The **data tier** is a PostgreSQL database plus model files. External services are called only from the back end. Every JSON response is wrapped by a single hook in a uniform envelope (data/meta or error), which makes the front end simpler and errors consistent.

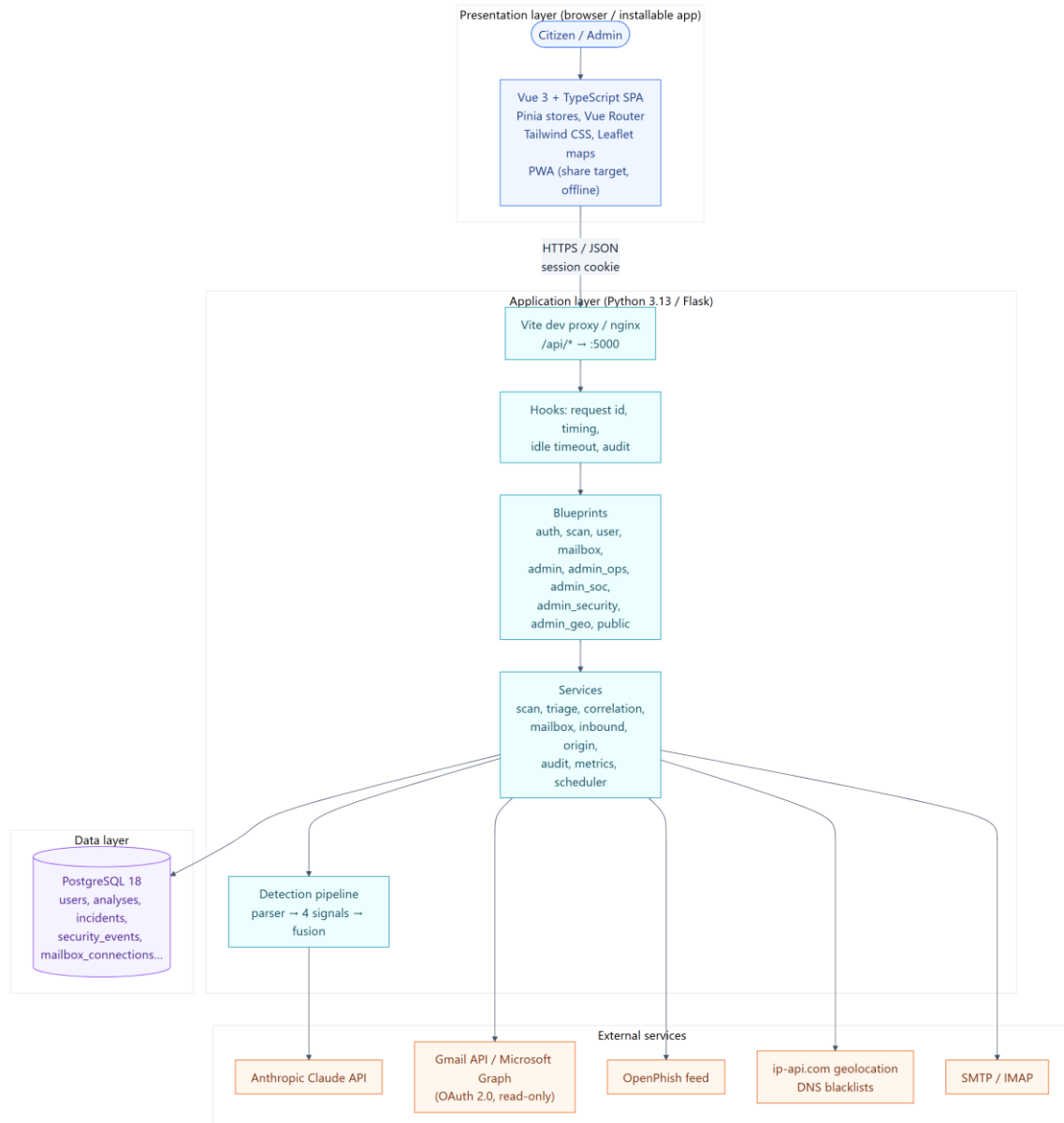


Figure 4.2: Three-tier architecture of PhishGuard-AI

4.3.2 Design of the detection pipeline

The pipeline (Figure 4.3) is the heart of the platform. An input (text, URL, e-mail or OCR text) is first **parsed**: for e-mails, headers, body, links and authentication results are extracted; for all inputs, URLs, phone numbers and amounts are identified. Four signals then run in parallel:

- **Text model (weight 30 %)** — TF-IDF + logistic regression returns a probability of phishing.
- **Claude AI (25 %)** — returns a structured judgement: classification, scam category, confidence and reasons.

- **URL intelligence (25 %)** — scores each link; the worst link determines the signal.
- **Expert rules (20 %)** — Cameroon-specific patterns in French and English.

The scores are fused by a weighted average. If a signal is unavailable (no link in the message, AI not configured or timed out), its weight is redistributed proportionally to the others, so the score is always on the same 0–100 scale. Two **deterministic overrides** follow: a link on the blocklist or a proven brand spoof forces the score to at least 85; a message from an official, authenticated sender (SPF/DKIM/DMARC pass) with only clean links is capped at 30. Finally, the score is mapped to a level (above 80 Critical, above 60 High, above 40 Medium, otherwise Low) and the evidence is merged into a human explanation. For e-mails, the origin tracer runs alongside and adds its own evidence (e.g. "sent from a datacentre abroad while claiming to be a Cameroonian bank").

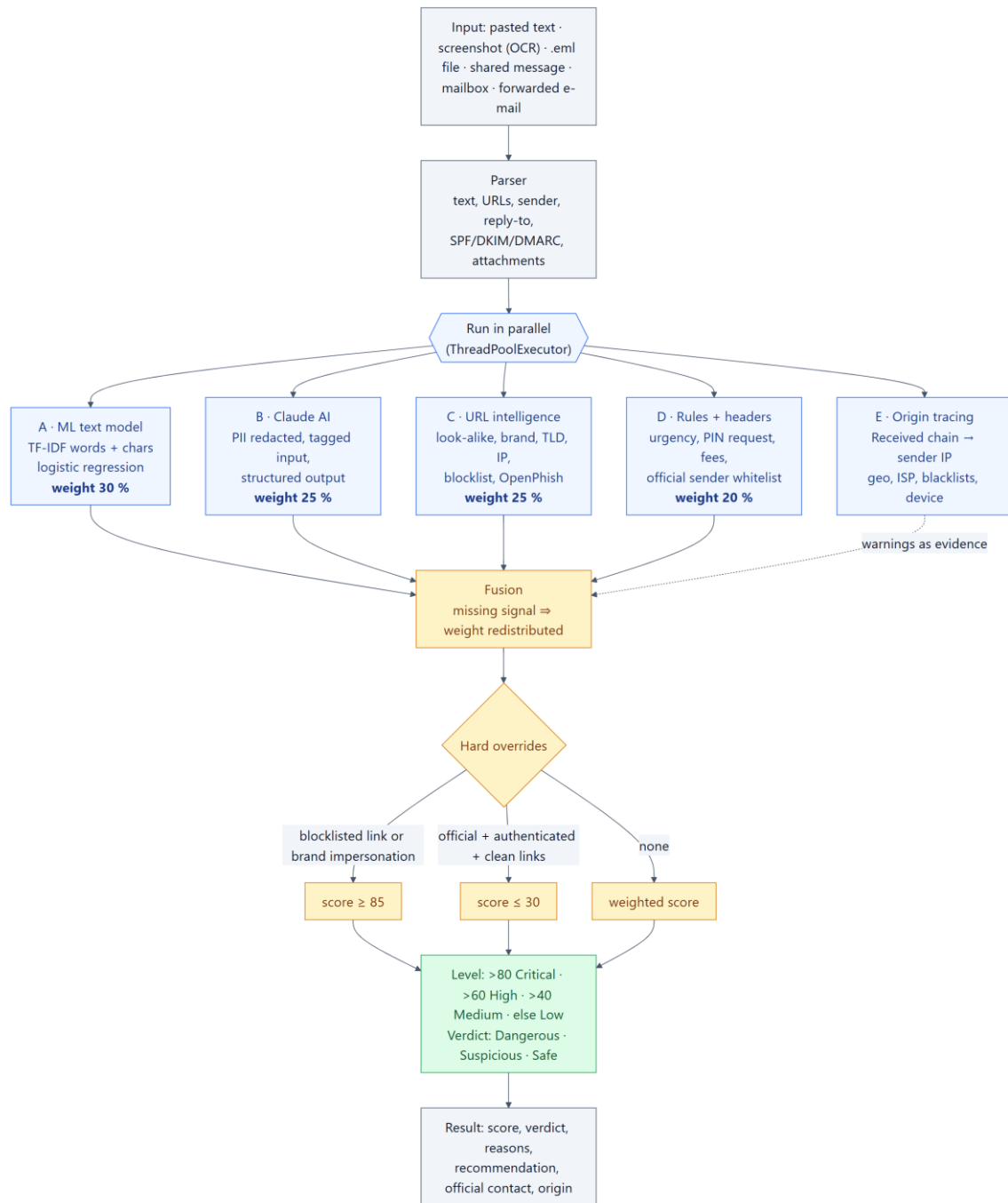


Figure 4.3: Detection pipeline with four fused signals

Signal	Weight	Technique	Strength / weakness
Text model	30 %	TF-IDF (word 1–2-grams + character 3–5-grams), logistic regression	Fast (ms), offline, high recall / over-reacts to alarming words
Claude AI	25 %	LLM with structured output on a redacted copy	Understands context and intent / slow, external, costly
URL intelligence	25 %	Lexical features, look-alike brands, TLD, blocklist, OpenPhish, XGBoost	Catches fresh malicious domains / useless without links

Signal	Weight	Technique	Strength / weakness
Rules	20 %	Local patterns (PIN, fees, cut-off, number change)	Very precise, explainable / low recall

Table 4.4: Signals of the detection pipeline

Formally, let S be the set of available signals, $s_i \in [0, 100]$ the score of signal i and w_i its base weight. The fused score is:

$$\text{score} = \sum_{i \in S} (w_i / \sum_{j \in S} w_j) \cdot s_i$$

followed by the overrides: $\text{score} \leftarrow \max(\text{score}, 85)$ if a link is blocklisted or the sender spoofs a brand; otherwise $\text{score} \leftarrow \min(\text{score}, 30)$ if the sender is on the allowlist, authentication passed and the worst link scores below 30. The algorithm is summarised below.

```

Algorithm 1 - Fusion of detection signals
Input : signals {ml, ai, url, rules} (score or null), facts
Output: score, level, weights, overrides
1 W <- {ml: .30, ai: .25, url: .25, rules: .20}
2 A <- { k in signals | signals[k] is not null }
3 for each k in A: w[k] <- W[k] / sum(W[j] for j in A)
4 score <- sum(w[k] * signals[k] for k in A)
5 if facts.blocklisted or facts.brand_spoof: score <- max(score, 85)
6 else if facts.sender_whitelisted and facts.auth_pass
7     and facts.worst_url_score < 30: score <- min(score, 30)
8 level <- Critical if score > 80, High if > 60, Medium if > 40, else Low
9 return score, level, w, overrides

```

Worked example. Consider the message "MTN MoMo: your account will be suspended within 24h. Confirm your PIN now at <http://mtn-momo-verif.tk/confirm>". With illustrative signal values — text model 97, Claude 95, URL intelligence 100 (brand impersonation +50, suspicious TLD .tk +30, plain HTTP +10, lure word "confirm" +10) and rules 60 (PIN request, suspension pressure, deadline) — the fused score is $0.30 \times 97 + 0.25 \times 95 + 0.25 \times 100 + 0.20 \times 60 = 89.9$, level **Critical**. If the AI is unavailable, the remaining weights become 0.40, 0.33 and 0.27 and the score is $0.40 \times 97 + 0.333 \times 100 + 0.267 \times 60 \approx 88.1$: the verdict is unchanged, which illustrates the graceful degradation required by NFR2. Conversely, a genuine Orange Money receipt ("Transfer received from ... new balance ...") gets a moderate text-model score because of words like "solde" and "transaction", but no link, no rule and a low AI score, so the fused score stays in the Low band.

Condition on the link	Points	Rationale
Domain on the allowlist of official Cameroonian domains	score = 0	Known official site
Look-alike of an official domain (e.g. rntn.cm)	+60	Typosquatting

Condition on the link	Points	Rationale
Brand name inside an unrelated domain (mtn-momo-verif.tk)	+50	Brand impersonation
Raw IP address instead of a domain	+35	Avoids domain registration
Suspicious TLD (.tk, .ga, .ml, .cf, .gq, .xyz, .top, .icu, .buzz, .click)	+30	Free or cheap TLDs abused by campaigns
Punycode (xn--) host	+30	Homograph attack
URL shortener (bit.ly, tinyurl.com, cutt.ly ...)	+25	Hides the destination
User-info in the URL (http://mtn.cm@evil.com)	+25	Misleading host
Three or more sub-domain levels	+10	mtn.cm.secure.example.com pattern
Plain HTTP	+10	No TLS
Lure words in the path (login, verify, confirm ...)	+10	Credential-harvesting page
Domain on the blocklist (analyst or incident)	score $\geq$ 95	Known malicious

Table 4.5: Scoring rules of the URL-intelligence signal

The rule-based URL score is complemented by the gradient-boosted URL model when it is available; the worst link of the message determines the URL signal. The rules signal works in the same additive way on the message and its headers: +10 per secret requested (PIN, password, OTP), +20 for an advance-fee formula, +40 when the sender domain imitates an institution, +15 when the Reply-To domain differs from the sender, +20 when SPF, DKIM or DMARC fails, and +30 for a dangerous attachment type (.exe, .apk, .js, .html ...).

4.3.3 Class design

Figure 4.4 shows the main classes. On the domain side, User owns Analysis records, Mailbox connections, SecurityEvent entries and quiz progress. On the processing side, the Pipeline coordinates a Parser, four signal classes that share the same interface (analyse(input) → SignalResult), a Fusion component and the OriginTracer. The services layer (ScanService, MailboxService, TriageService, CorrelationService, AuditService) isolates business rules from the HTTP layer.

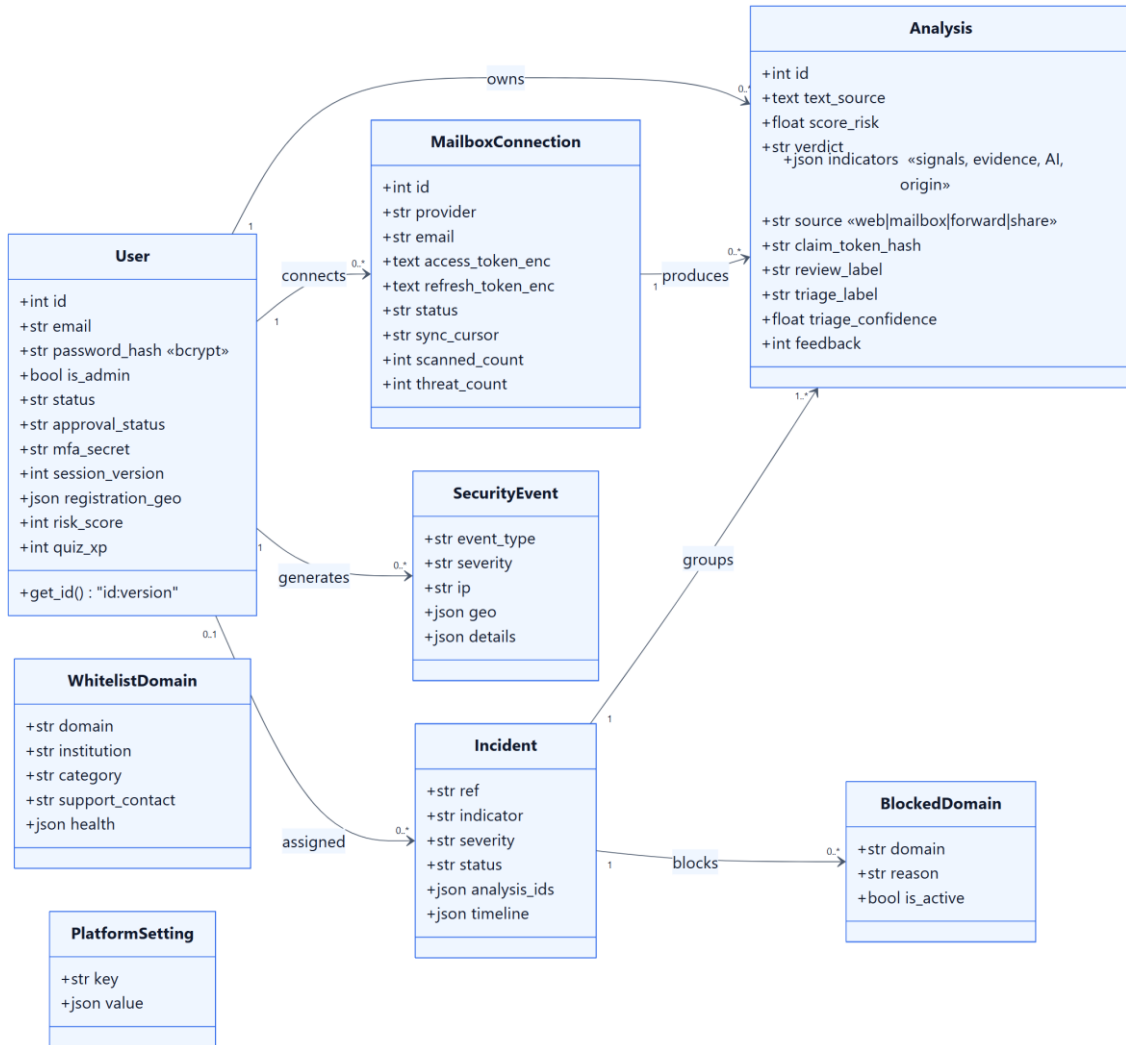


Figure 4.4: Class diagram (main classes)

4.3.4 Database design

The database (Figure 4.5) is normalised around the users and analyses tables. analyses stores the score, verdict, level, source channel and a JSON column with the evidence and origin. Supporting tables include mailboxes (encrypted OAuth tokens and sync cursor), mailbox_messages (deduplication on mailbox and external id), scan_claims (hashed visitor tokens), incidents (with JSON timeline), blocked_domains, allowed_domains, security_events (login history and audit log), platform_settings and quiz tables. Schema scripts are applied in numeric order (01_schema.sql to 09_user_experience.sql).

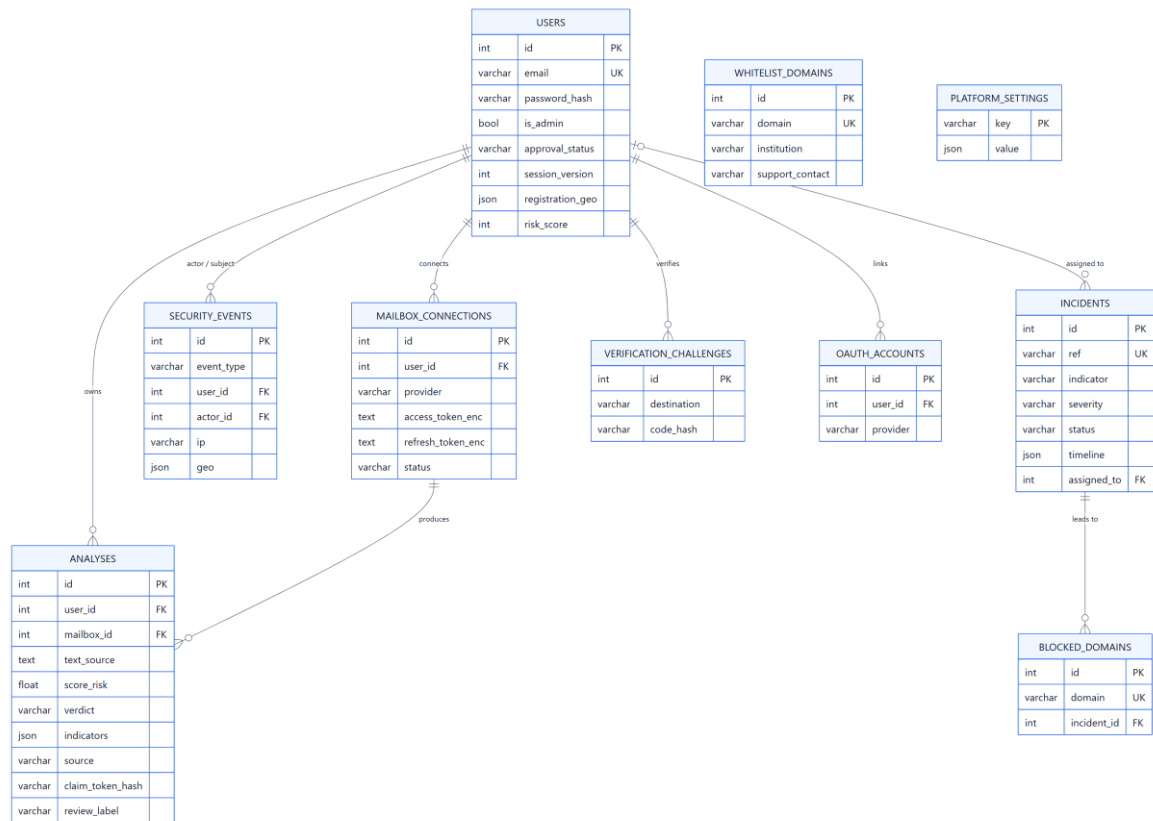


Figure 4.5: Entity-relationship diagram

4.3.5 Dynamic behaviour

Figure 4.6 (landscape page) shows the sequence of a visitor scan: the message is analysed immediately, but the API returns only a single-use claim token; after sign-in the front end calls the claim endpoint and the result is revealed. Figure 4.7 shows a mailbox synchronisation: the scheduler or the user starts a sync, the service refreshes the OAuth token, fetches new messages since the last cursor, runs the pipeline on each one, stores the result and reports progress to the live-scan widget. Figure 4.8 describes, as an activity diagram, the decision flow of an analysis from input to verdict.

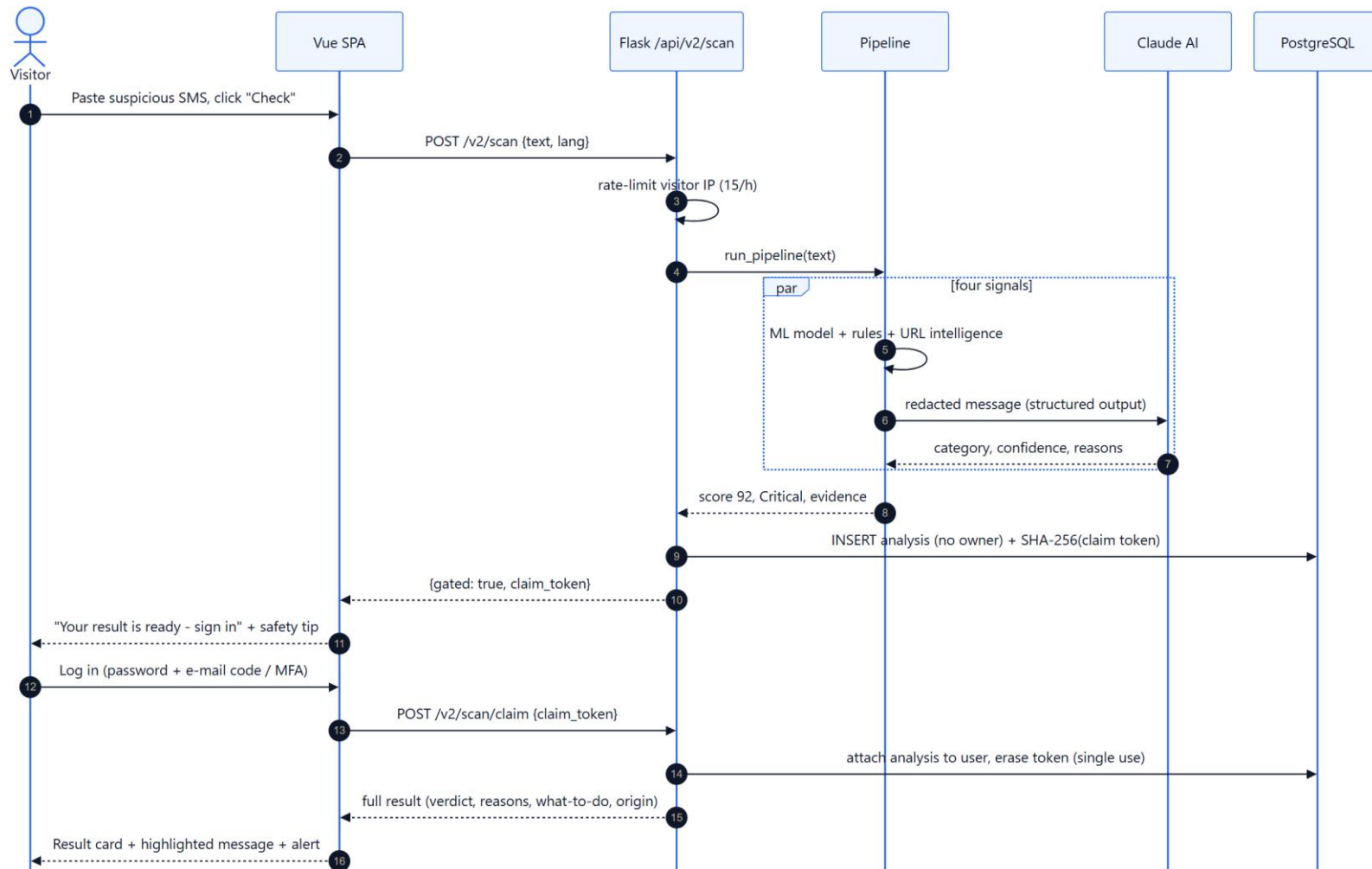


Figure 4.6: Sequence diagram of a visitor scan and result claim

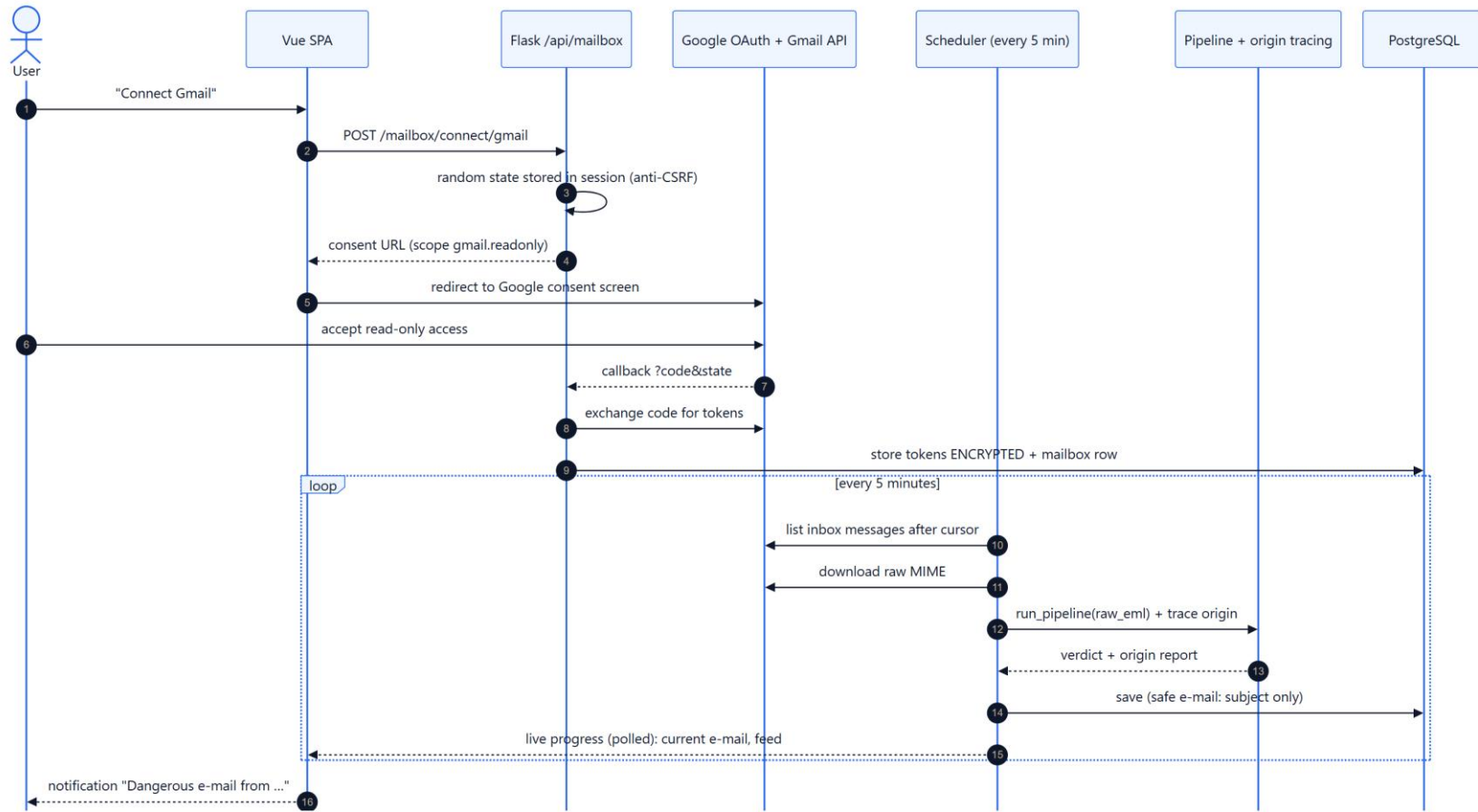


Figure 4.7: Sequence diagram of a mailbox synchronisation

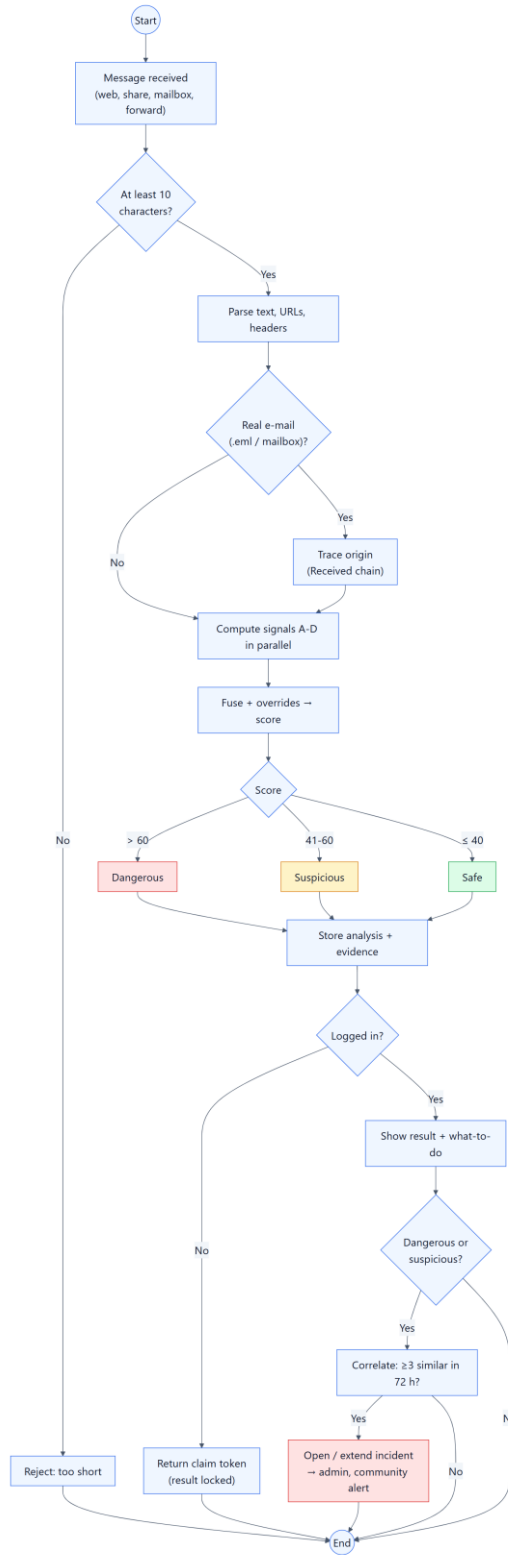


Figure 4.8: Activity diagram of an analysis

4.3.6 Design of the sender-origin tracer

Every mail server that relays a message adds a *Received* header at the top, of the form "from <host> [<ip>] by <server> ... ; <date>". The first header written — the lowest one — is

therefore the closest to the sender. The tracer (Algorithm 2) reads the chain from the bottom, measures the delay between hops, recognises large providers, and identifies the sender IP either from an explicit header added by some webmails (X-Originating-IP) or as the first public IP address. When that first hop belongs to a large provider such as Google, the real sender is hidden behind the provider: the tracer reports "hidden by provider" and **refuses to geolocate the provider's server**, because showing Mountain View for every Gmail sender would be misleading.

```

Algorithm 2 - Tracing the origin of an e-mail
Input : raw e-mail, claimed brand (optional)
Output: origin report (admin view) -> public_view() for users
1 hops <- parse(Received headers) in reverse order (first hop first)
2 compute delay between hops; tag hops run by known providers
3 ip <- X-Originating-IP / X-Sender-IP header if public
4 if none: ip <- first public "from" IP in hops
5 if no ip or the origin hop is a provider server:
6     precision <- hidden ; do not geolocate ; flag hidden_by_provider
7 else in parallel: geo <- geolocate(ip) ; bl <- DNSBL(ip) ; relays <- locate(other
hops)
8 precision <- medium if geo is hosting/proxy, high otherwise
9 accuracy_km <- 50 (mobile) | 15 (fixed) | 5 (datacentre)
10 flags: datacenter, proxy, blacklisted, scripted mailer,
        foreign_for_local_brand (country != CM and a Cameroonian brand is claimed),
        timezone_mismatch (Date offset vs IP zone >= 3 h), long_delay (> 1 h)
11 return report

```

Two views are derived from the same report. The **public view** given to users contains only what helps them decide: city and country, type of network, blacklist status, precision and the plain-language flags. The **administrator view** adds the IP addresses, the full hop list, reverse DNS, ASN, the mailer or device and the authentication results, and each consultation is written to the audit log (event `admin_origin_viewed`). This separation applies the principle of least privilege to the users' own data and to the attackers' infrastructure alike.

Flag	Meaning shown to the user
datacenter	Sent from a hosting server, not from a person's phone or home connection — typical of bulk phishing.
proxy	Sent through a VPN or proxy that hides the real location.
blacklisted	The sending server is listed by at least one anti-spam blacklist (SpamCop, Barracuda, UCEPROTECT).
scripted	Composed by a mass-mailing script (e.g. PHPMailer) rather than a normal e-mail application.
foreign_for_local_brand	Claims to be a Cameroonian institution but was sent from abroad.
timezone_mismatch	The time zone of the sender's clock differs by 3 h or more from the location of the IP.
long_delay	The message stayed more than one hour on an intermediate server.

Flag	Meaning shown to the user
hidden_by_provider	The sender used a large webmail service that hides their location.

Table 4.6: Flags raised by the origin tracer

4.3.7 Security operations design

The SOC workflow (Figure 4.9) turns individual analyses into collective intelligence. Users report wrong verdicts; the triage service suggests a decision with a confidence score; depending on the `triage_mode` setting (manual, assisted, automatic above a threshold of 80), an administrator confirms it. The correlation service groups dangerous messages that share a domain or impersonated brand within a time window into an **incident** with a timeline; confirmed malicious domains feed the **blocklist**, which the URL-intelligence signal uses immediately. Administrator decisions — but not automatic ones — are exported for retraining.

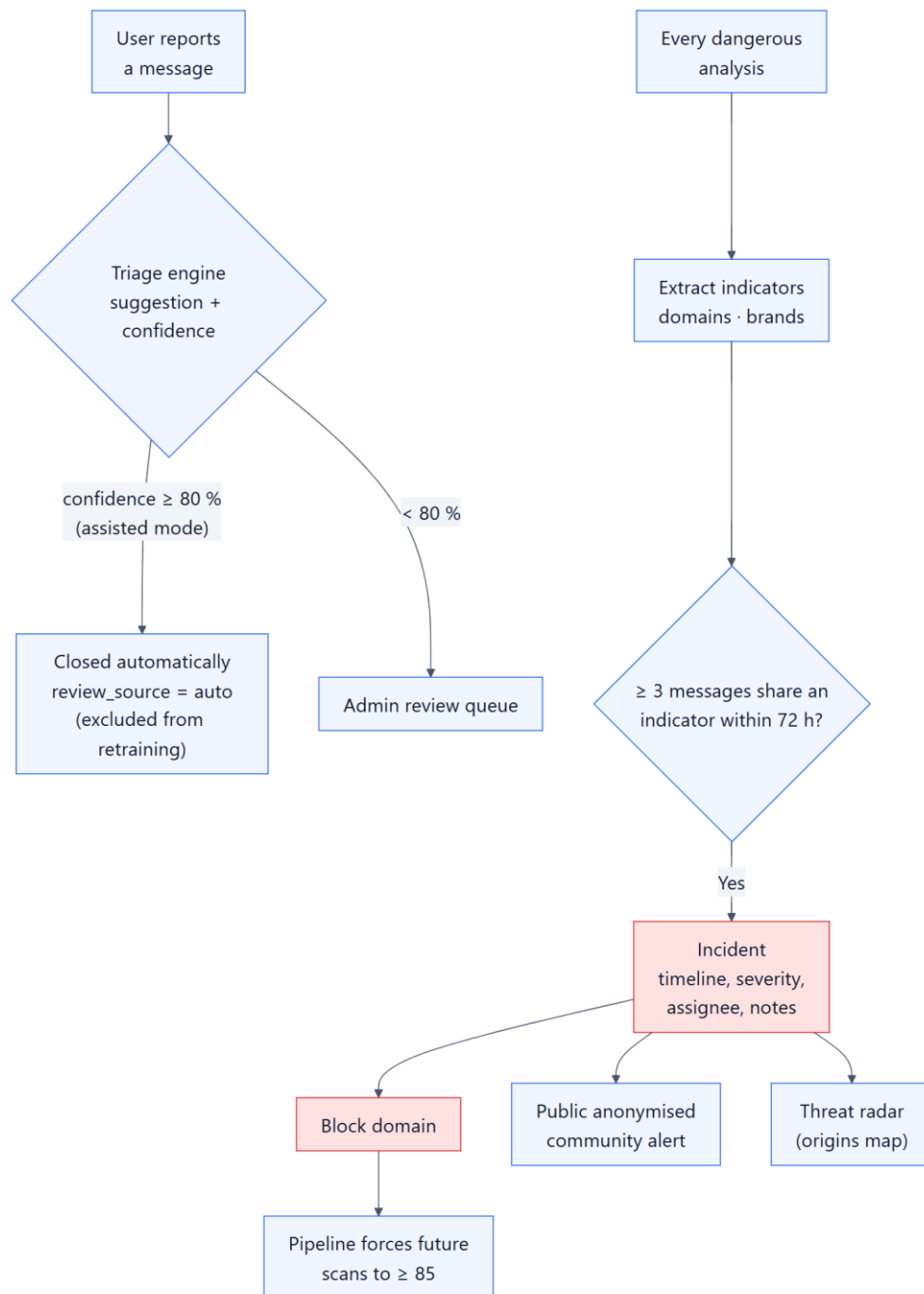


Figure 4.9: Security operations workflow

4.3.8 Security design

Because the platform stores sensitive messages and has access to mailboxes, it is itself a target. Its security design applies the principles of Saltzer and Schroeder (1975) and privacy by design (Cavoukian, 2009); Figure 4.10 shows the layers of defence and Table 4.7 maps each principle to a concrete control.

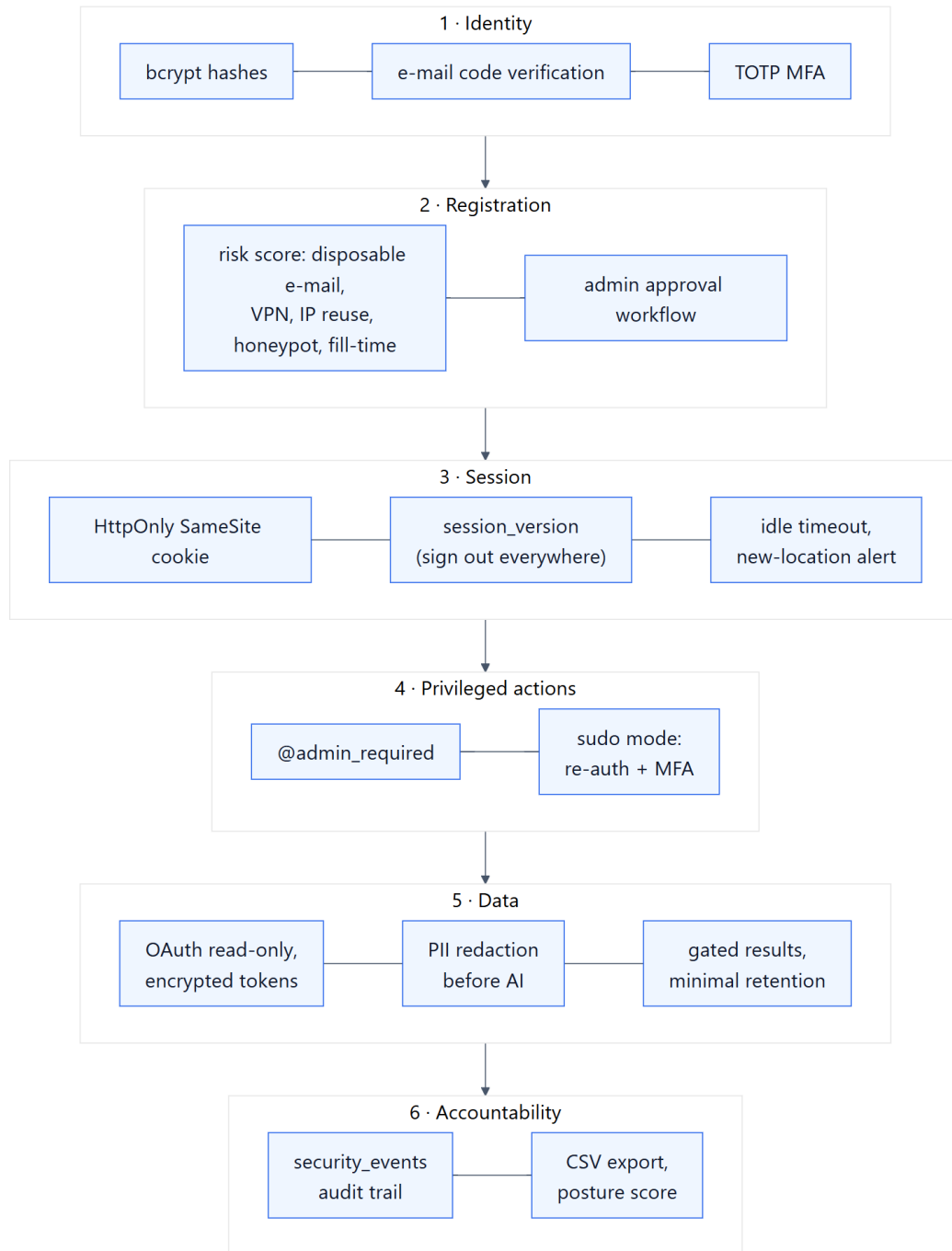


Figure 4.10: Layers of security controls

Principle	Control in PhishGuard-AI
Least privilege	Gmail scope gmail.readonly and Outlook Mail.Read only; admin role separated from user role; database account without superuser rights.
Fail-safe defaults	Anonymous callers never receive a verdict; new risky accounts wait for approval; unknown errors return a generic message.
Complete mediation	Every /api request goes through Flask-Login; load_user re-checks suspension, approval and session version on each request.

Principle	Control in PhishGuard-AI
Separation of privilege	Sensitive admin actions require sudo mode (password re-entry, time-limited) on top of the admin session.
Defence in depth	bcrypt hashing, CSRF tokens, secure cookies, rate-limited OTP, TOTP MFA, idle session timeout.
Accountability	Every login, security change and admin action is written to <code>security_events</code> (audit log); viewing a sender origin is itself audited.
Data minimisation	Safe mailbox e-mails are stored as subject only; claim tokens stored as SHA-256 hashes; users can delete their analyses.
Protection of secrets	OAuth tokens encrypted (Fernet); secrets only in environment variables, never in the repository.
Revocation	Bumping <code>session_version</code> signs a user out of all devices; mailbox tokens can be revoked by the user or the provider.

Table 4.7: Security principles and their implementation

Mechanism	Implementation
Password storage	bcrypt with per-password salt and adaptive cost (Provos & Mazières, 1999).
Sessions	Server-signed Flask-Login cookie; identifier = user id + session version; HttpOnly, SameSite.
Second factor	TOTP (RFC 6238) with pyotp; e-mail and SMS one-time codes for verification.
Federated login	Google OAuth 2.0 (RFC 6749) with state parameter.
Registration risk	Score from disposable e-mail, IP reuse, VPN/datacentre, honeypot field and fill time; decides auto-approval, review or rejection.

Table 4.8: Summary of the authentication and session mechanisms

4.3.9 Deployment design

In development, the Vite server (port 5173) proxies `/api` to Flask (port 5000). In production, the front end is built into static files served by a reverse proxy (e.g. Nginx) over HTTPS, which forwards `/api` to gunicorn workers running the Flask application; PostgreSQL runs on the same host or a managed service (Figure 4.11). The scheduler runs as a daemon thread inside the application.

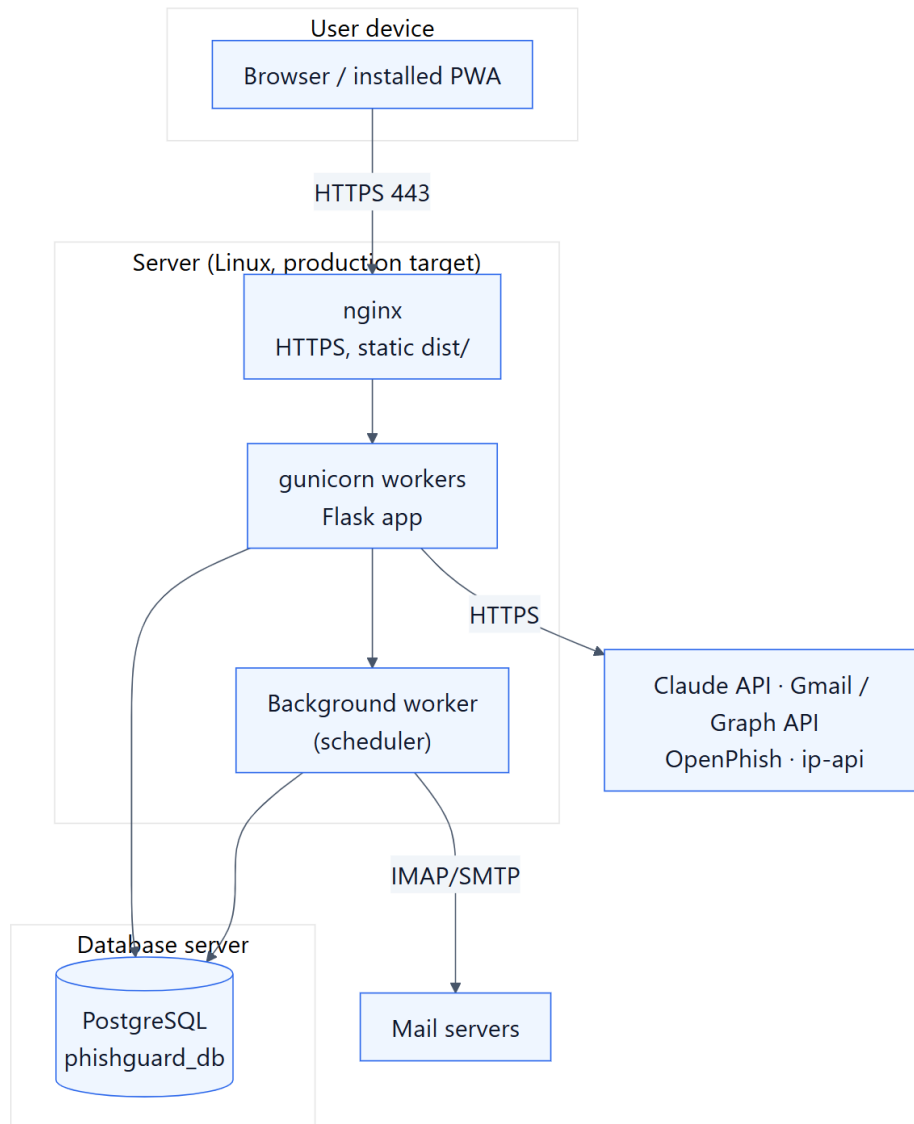


Figure 4.11: Deployment diagram

4.3.10 User-interface design

The interface was designed for a user who is on a phone, possibly anxious about a message, and not a security specialist. Five design rules were applied:

- **One primary action.** Every public page leads to the verification box; on phones, a central "Check" button sits in the bottom tab bar.
- **Show the work.** During analysis, each of the four signals is shown as a step that completes in turn. This reduces the perceived waiting time of the AI signal and explains, before the result, that several independent checks are performed.

- **Colour plus icon plus words.** Levels use a consistent colour (green, amber, orange, red), an icon and a label, so the result is understood by colour-blind users and in bright sunlight.
- **Explain in the message.** Suspicious fragments are highlighted directly in the text the user received, because people remember cues better when seen in context (Kumaraguru et al., 2010).
- **Bilingual and inclusive.** All strings exist in French and English; the explanation can be read aloud by the browser's speech synthesis for users with low literacy or poor eyesight.

The front end uses a small component library: shared result, gate and progress components for the check flow (components/check/), a shared icon component, and an administrator UI kit (page header, panel, statistic tile, pill, sparkline, ring gauge) that gives all SOC screens the same visual language. Dark and light themes are supported.

4.4 Implementation

This section presents the platform as implemented, through screenshots taken on the running system with demonstration accounts.

4.4.1 Public interface and visitor flow

The landing page (Figure 4.12) puts the verification box first, with public anonymised alerts and awareness entry points. A visitor can paste a message and start the analysis immediately; a live progress view shows each signal working (Figure 4.13). When the analysis is finished, the result is hidden behind a gate (Figure 4.14): the visitor must sign in or create an account to see it. Technically, the API stores the result and returns a random token whose SHA-256 hash is saved; the token can be claimed once, within 24 hours, by an authenticated user. This design converts curiosity into registration without ever exposing verdicts to anonymous scripts that could use the platform to test their own phishing messages.

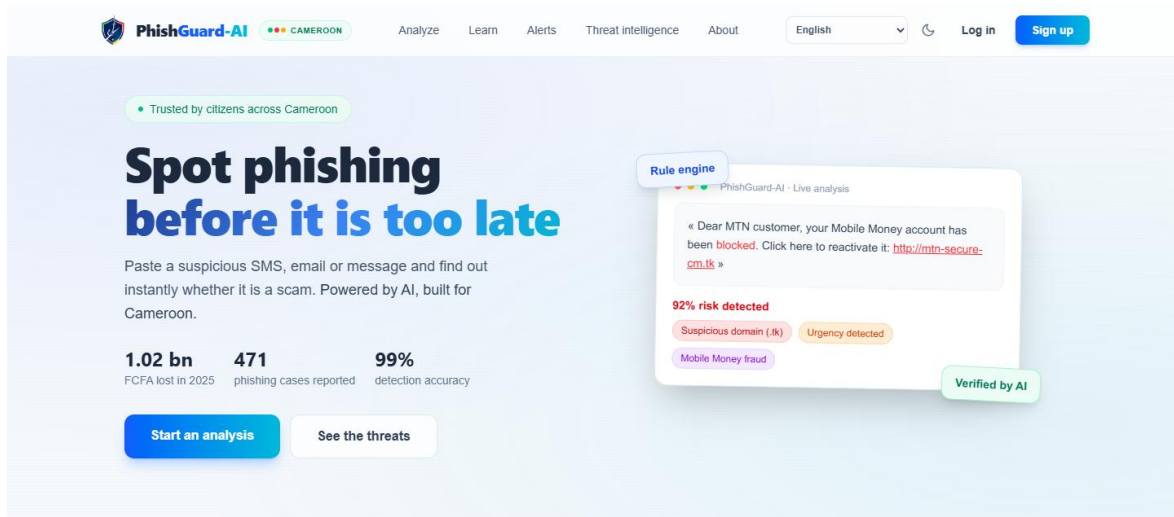


Figure 4.12: Landing page with the verification box

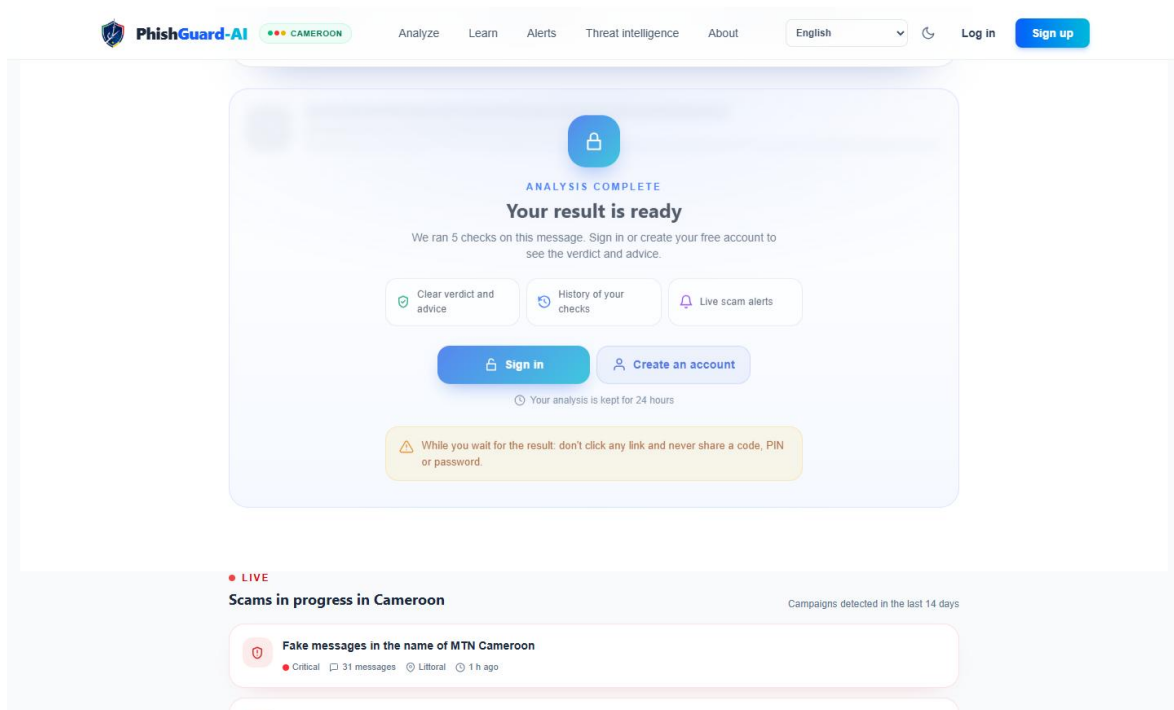


Figure 4.13: Live analysis progress showing the four signals

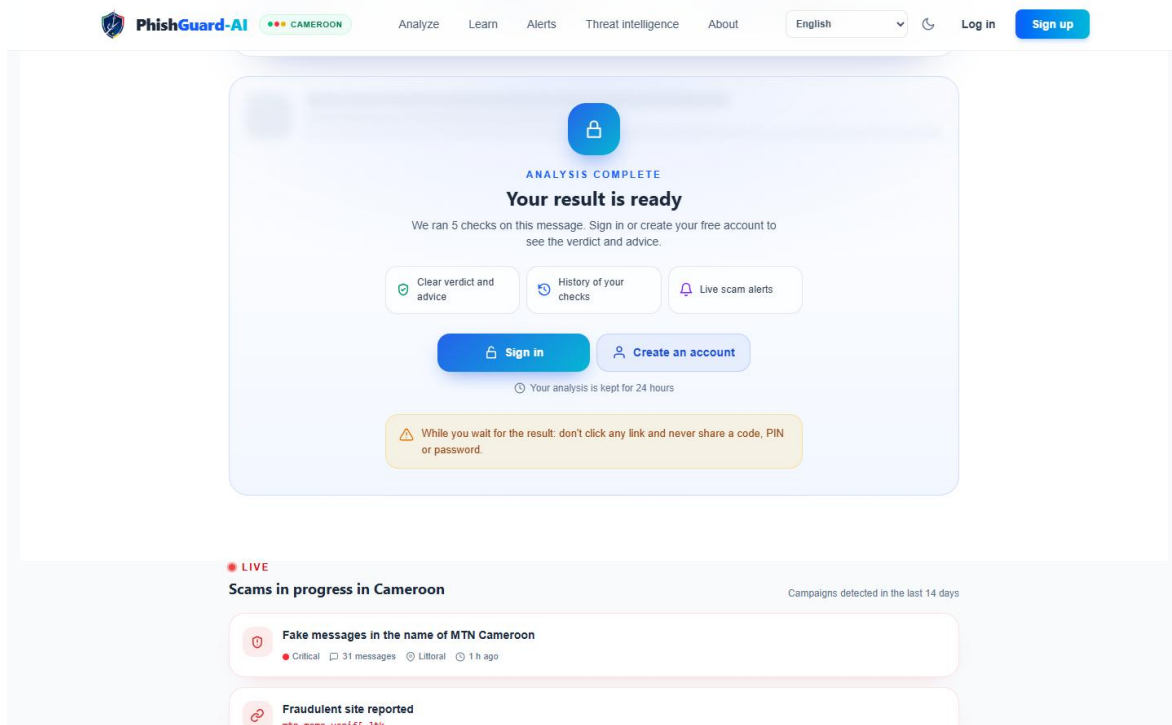


Figure 4.14: Result gated until the visitor signs in

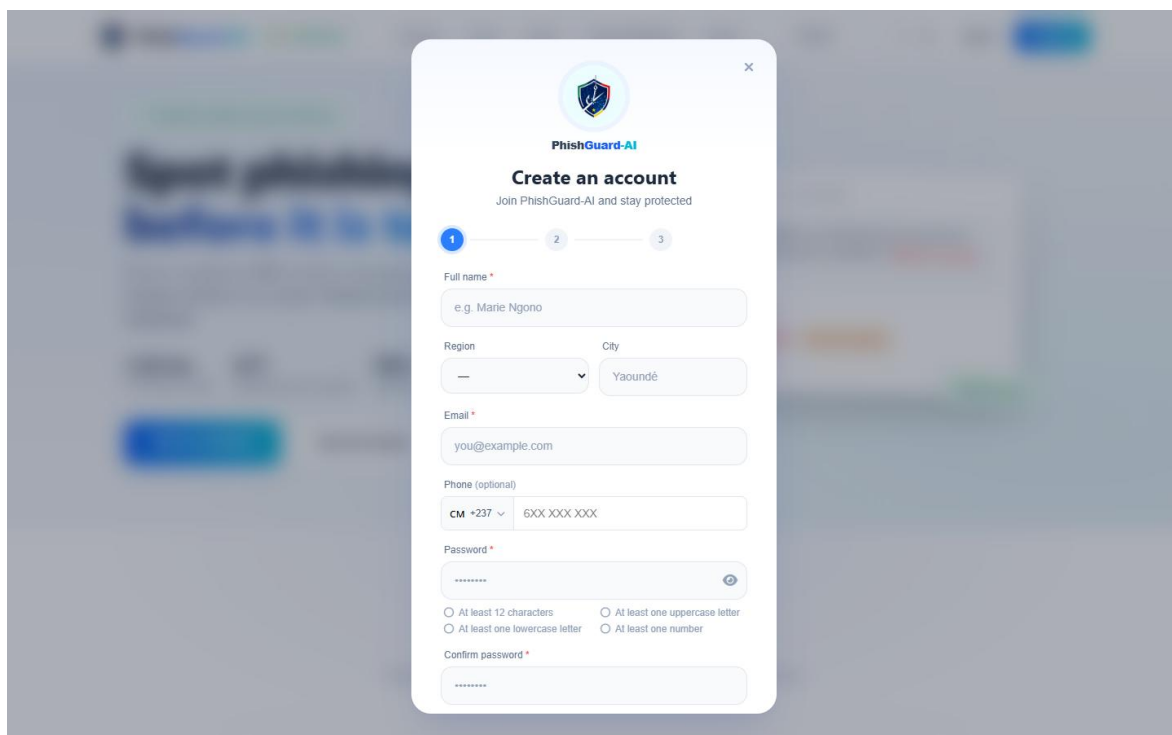


Figure 4.15: Registration dialog

4.4.2 Result and explanation

The result card (Figures 4.16 and 4.17) is the main awareness tool. It shows the level with colour and icon, a one-sentence summary, the message itself with the suspicious fragments

highlighted, the list of evidence grouped by signal, and "what to do now" actions adapted to the scam type (for MoMo: never share your PIN, dial the official number). The user can listen to the explanation (read-aloud) and give feedback if the verdict seems wrong, which sends the case to the review queue.

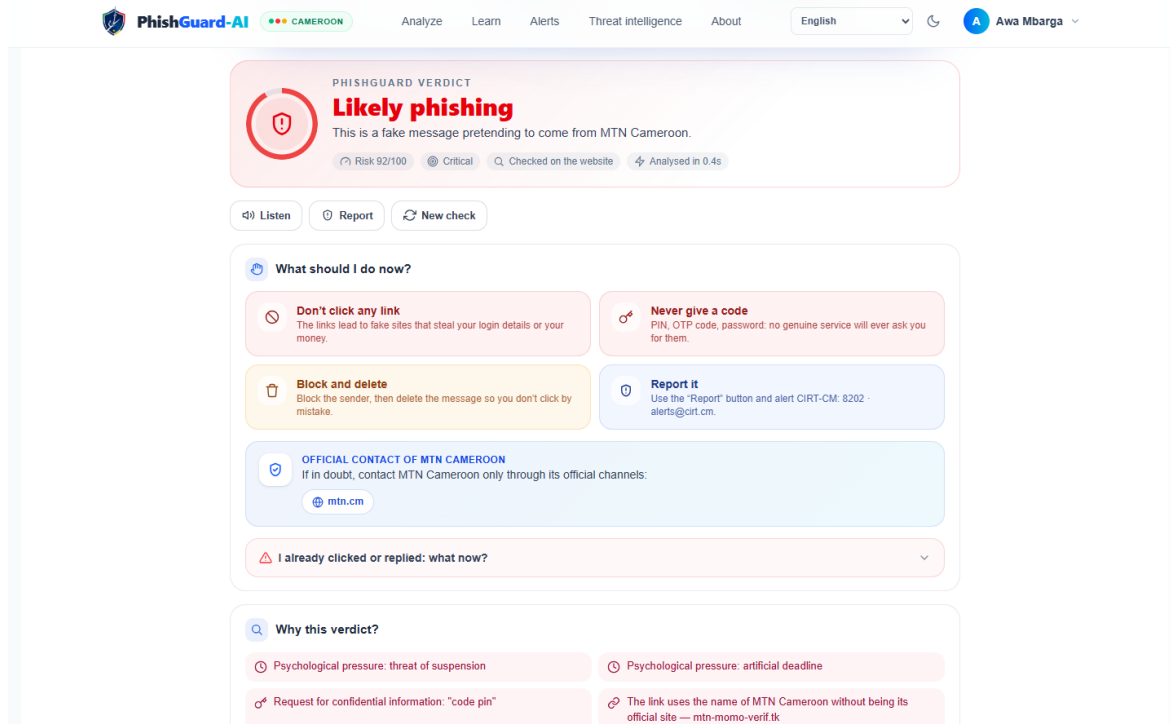


Figure 4.16: Result card with level, summary and highlighted cues

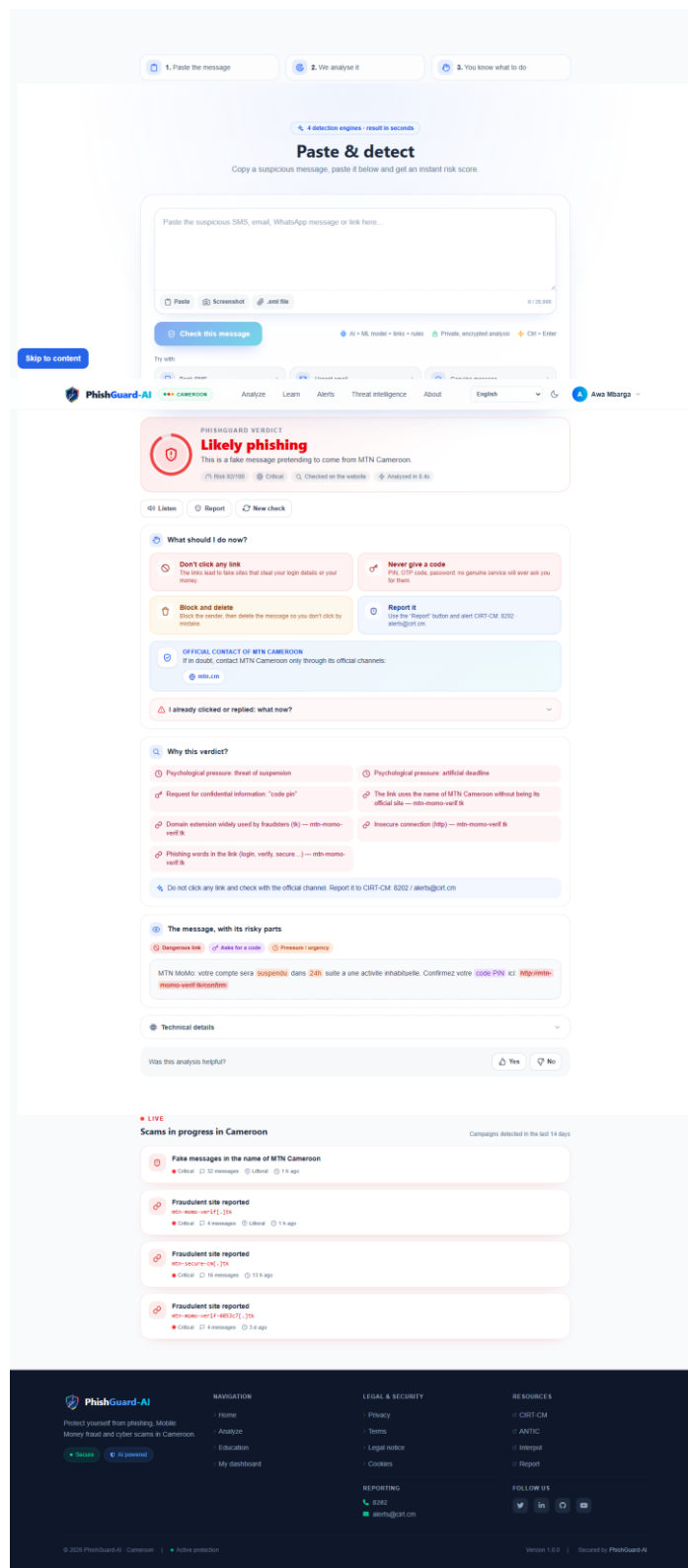


Figure 4.17: Full result: evidence per signal and recommended actions

For e-mails, the origin card (Figure 4.18) tells the user in simple words where the message really came from — city and country of the sending server, type of network (home, mobile, datacentre, VPN), whether the server is blacklisted, and whether this is consistent with the

claimed sender. When the sender used a webmail provider such as Gmail, the card explains that the provider hides the location instead of showing a wrong one.

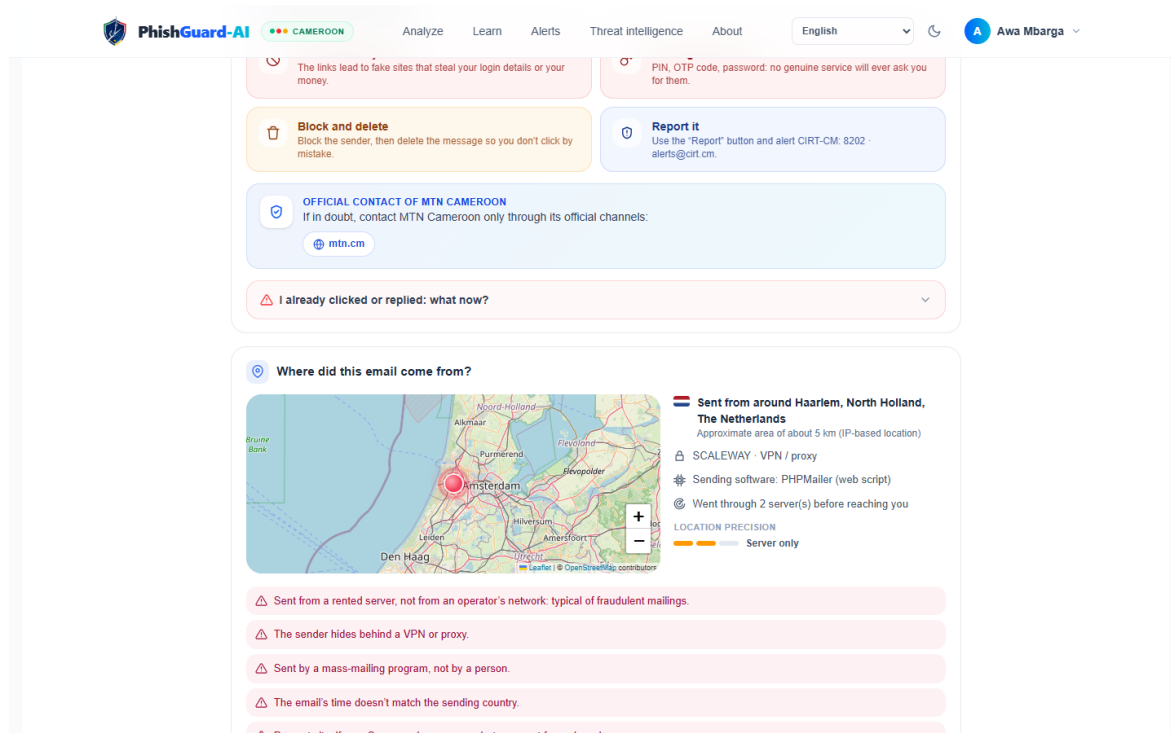


Figure 4.18: Sender-origin card shown to the user

4.4.3 User dashboard and mailbox scanning

The dashboard (Figure 4.19) summarises the user's activity: number of messages checked and threats blocked, a personal threat map, recent analyses and quiz progress. The history (Figure 4.20) lists every analysis with filters and allows deletion. The mailboxes page (Figure 4.21) lets the user connect Gmail or Outlook: the OAuth consent screen asks only for read access; new e-mails are then scanned automatically, and a live-scan animation shows each e-mail being analysed. The user also receives a personal forwarding address as an alternative for other providers.

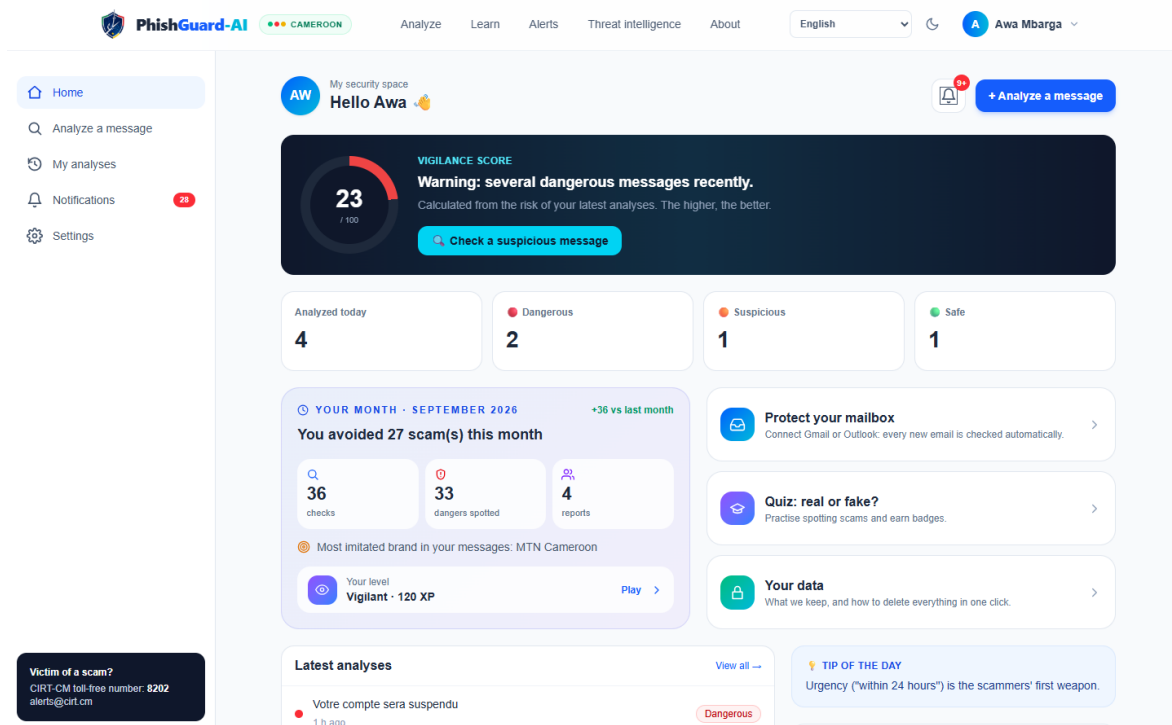


Figure 4.19: User dashboard

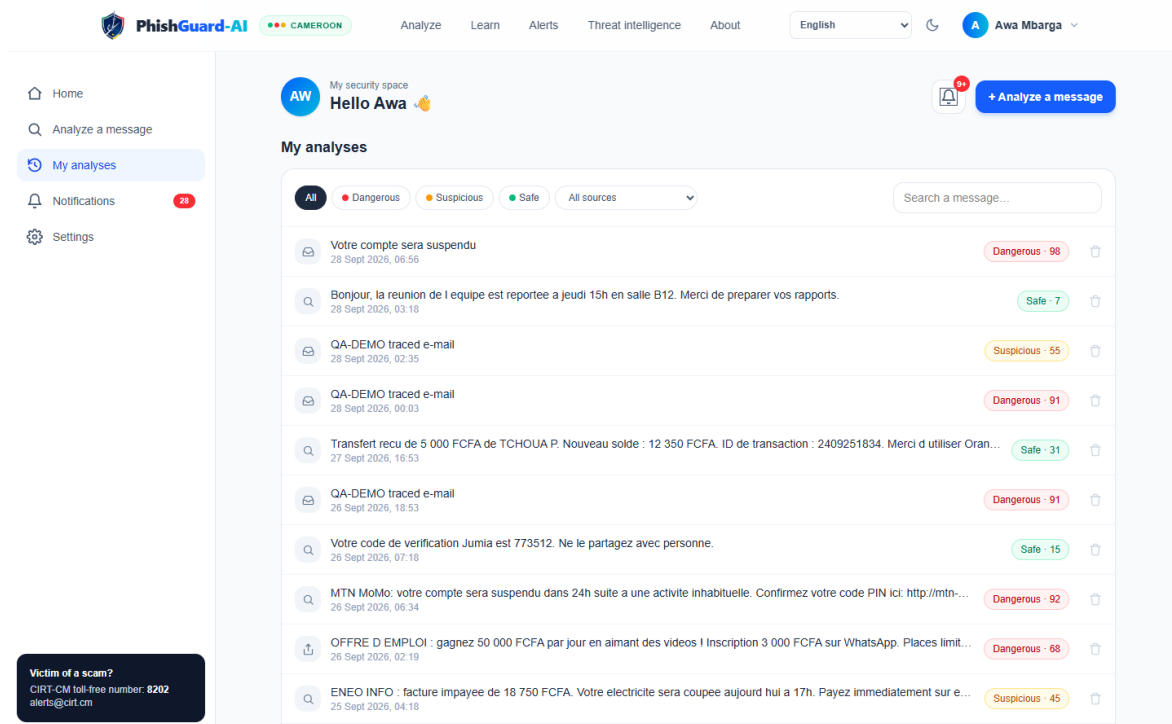


Figure 4.20: Analysis history

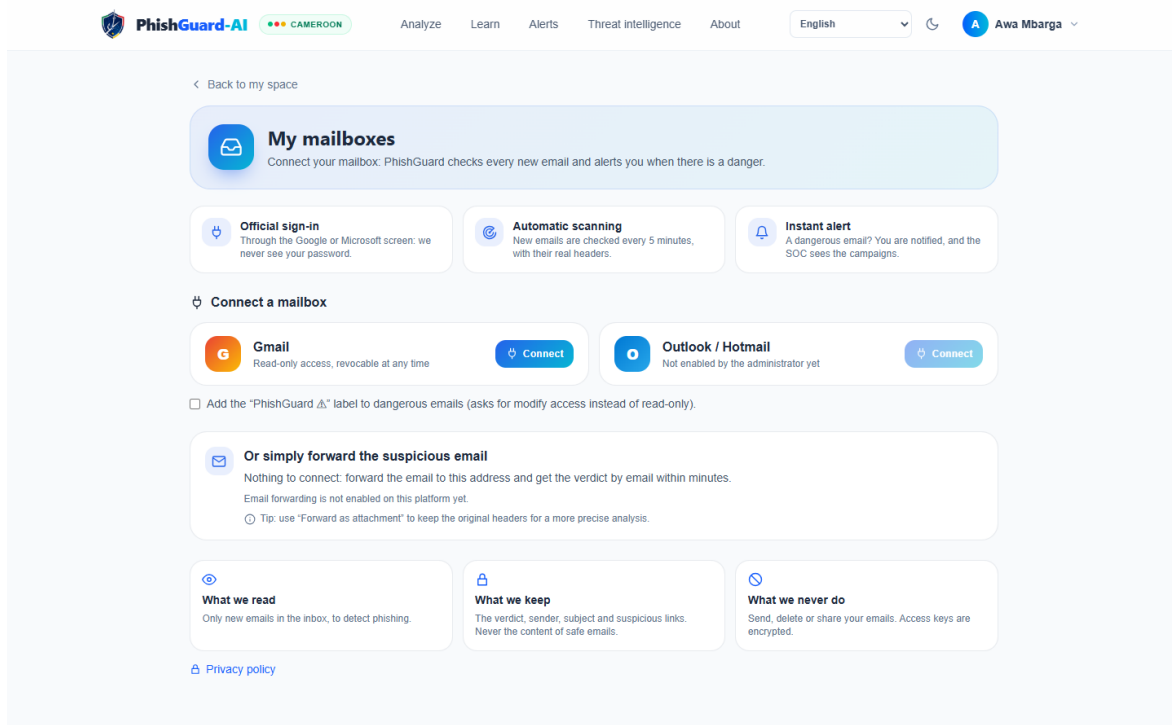


Figure 4.21: Connected mailboxes with live scan

Because most users are on phones, every screen is responsive and the application is installable as a PWA. On Android, PhishGuard-AI appears in the system *Share* menu: a user can share an SMS or a screenshot directly to the application (Figure 4.22).

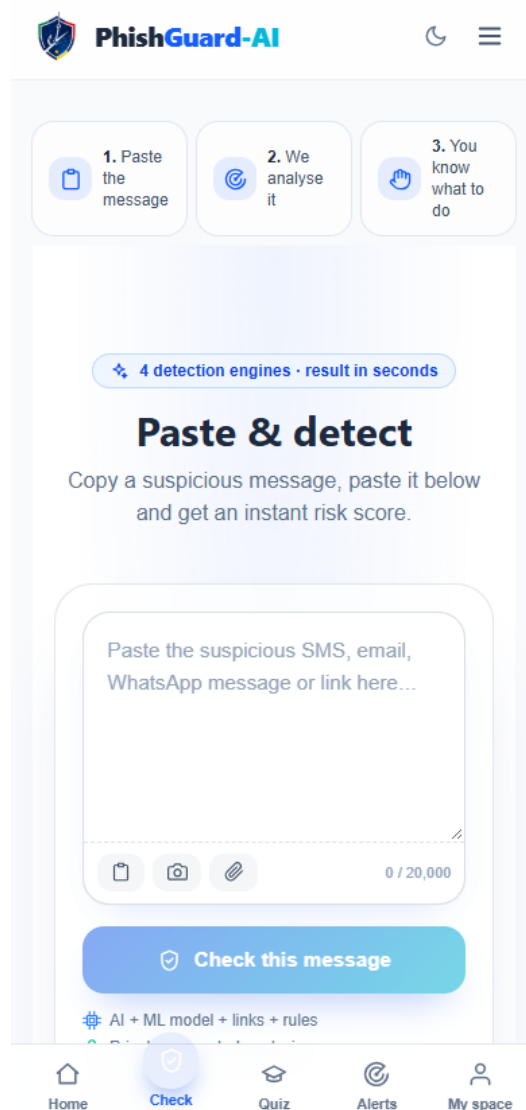


Figure 4.22: Mobile view of a result (PWA)

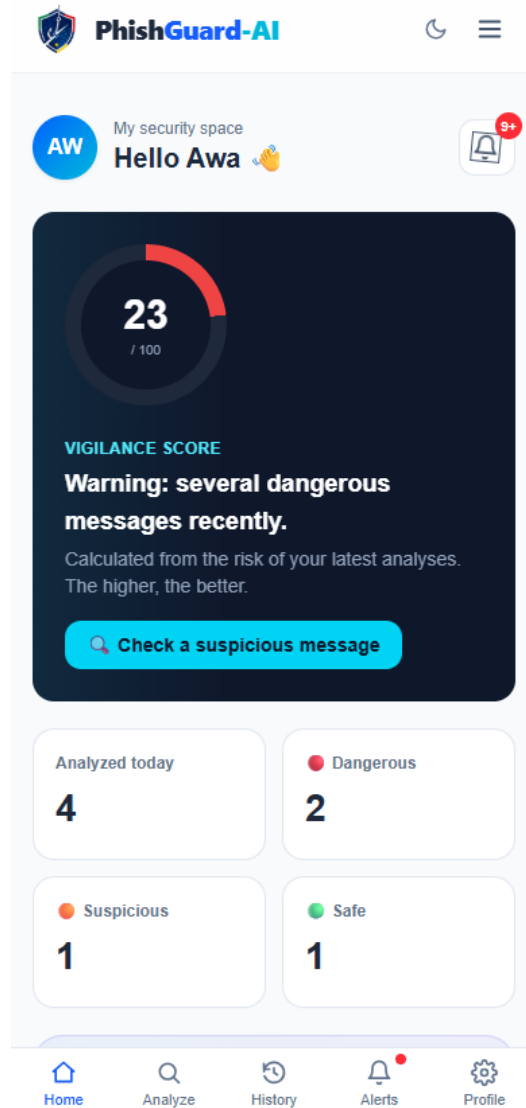


Figure 4.23: Mobile view of the dashboard

4.4.4 Awareness module

The Learn page offers a quiz built on real Cameroonian scenarios (Figure 4.24). For each message the user decides "real or fake"; the answer is followed by an explanation of the cues (Figure 4.25). Points and accuracy are kept in the user profile. A privacy page explains in plain language which data are stored and why (Figure 4.26).

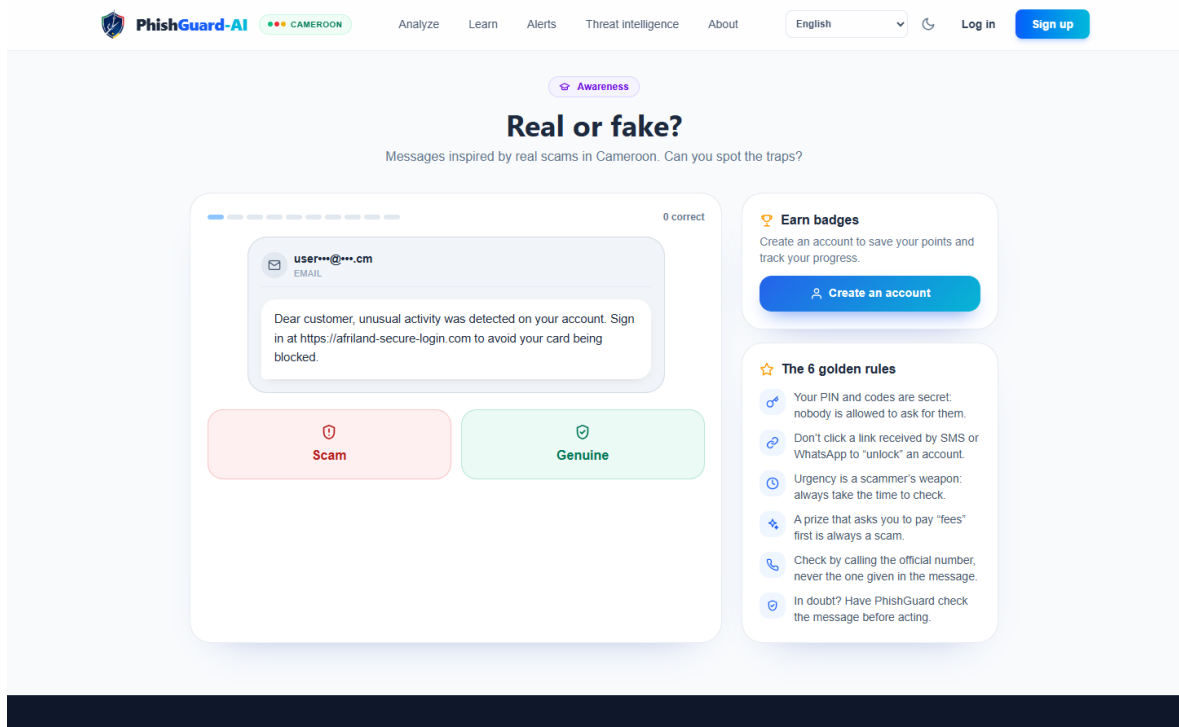


Figure 4.24: Awareness quiz

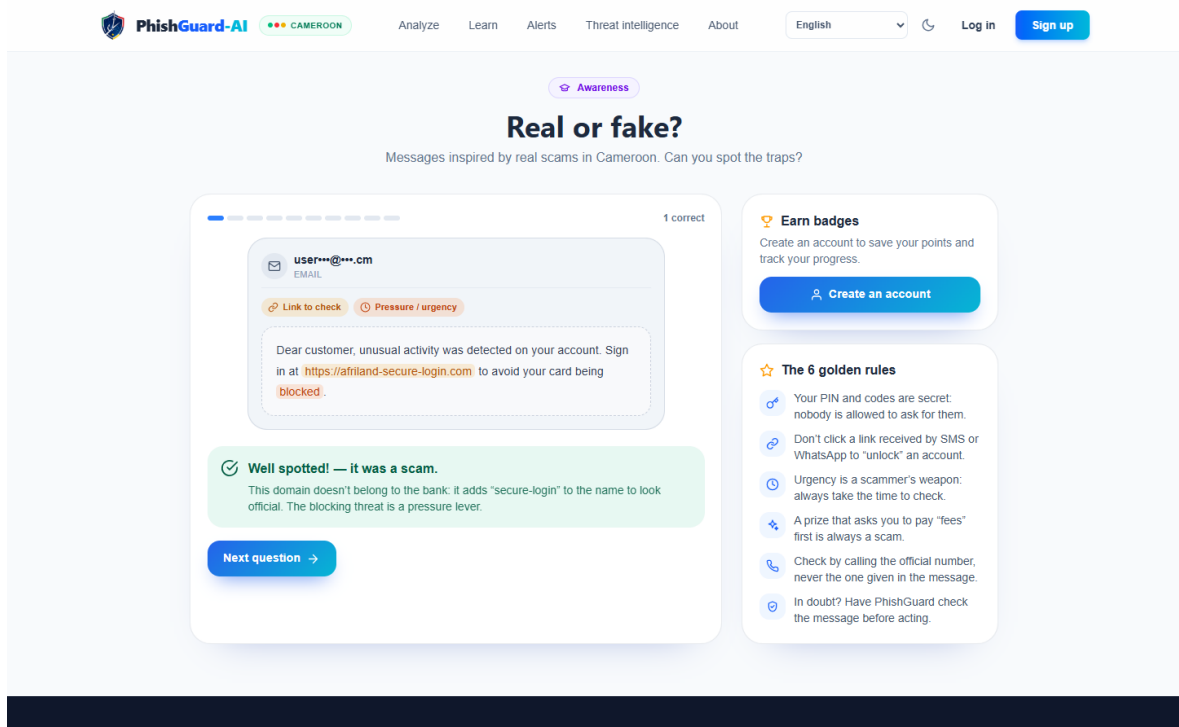


Figure 4.25: Quiz feedback explaining the cues

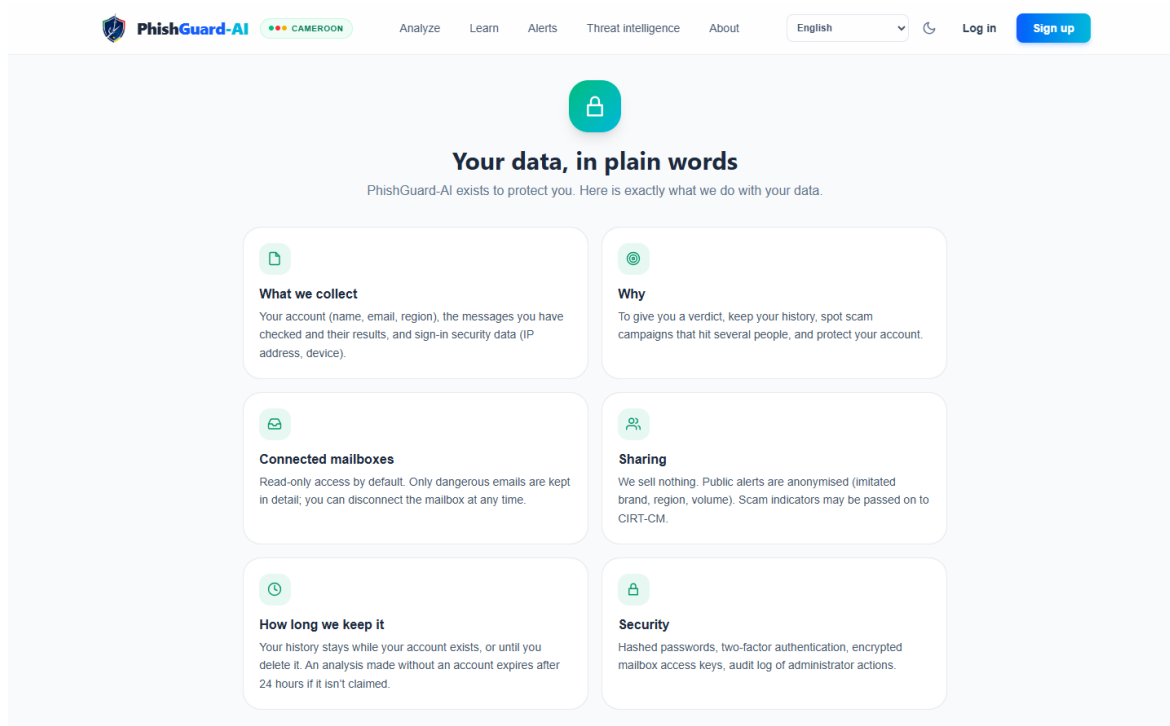


Figure 4.26: Plain-language privacy page

4.4.5 Administration and security operations

The administrator overview (Figure 4.27) gathers the key indicators: analyses and threats over time, pending approvals and reviews, open incidents and system status. The **threat radar** (Figures 4.28 and 4.29) displays on an interactive map the traced origins of dangerous e-mails, with animated attack arcs toward Cameroon, a live feed, a leaderboard of source countries and networks, replay and full-screen modes. Clicking a message opens the **origin investigation** (Figures 4.30 and 4.31): the full hop-by-hop path from the Received headers, the network owner and ASN, reverse DNS, blacklists, the device or mailer that composed the message, and the authentication results. Each opening is written to the audit log.

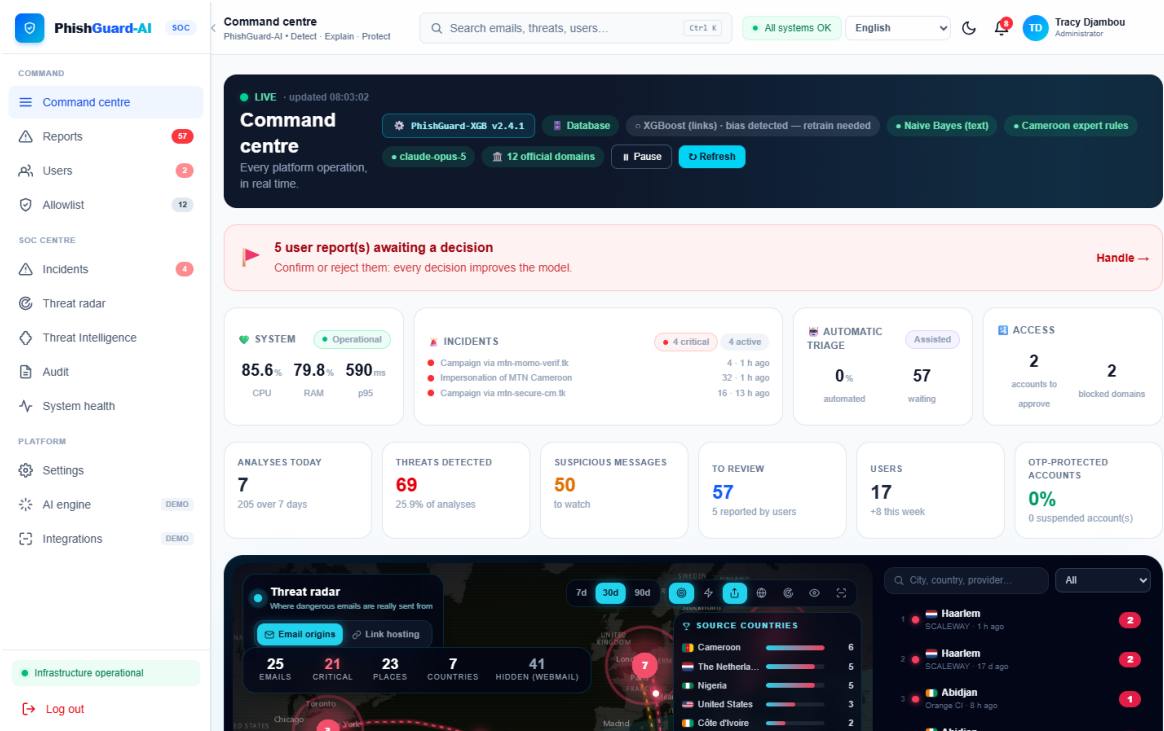


Figure 4.27: Administrator overview

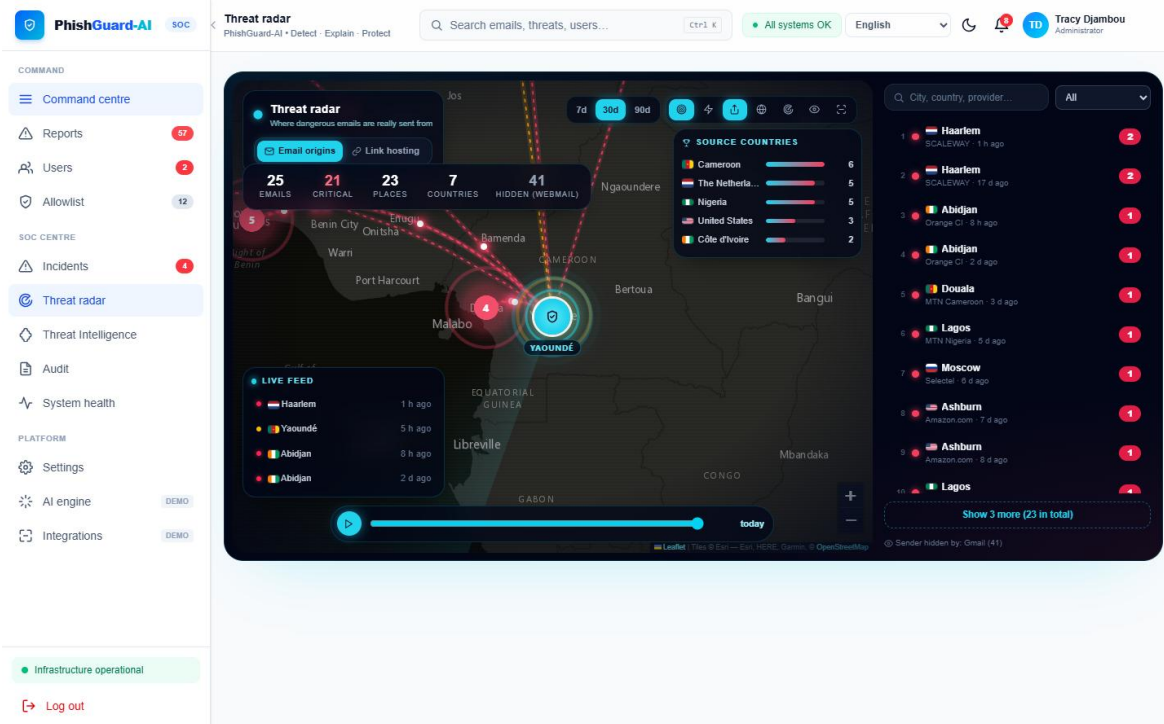


Figure 4.28: Threat radar: origins of dangerous e-mails

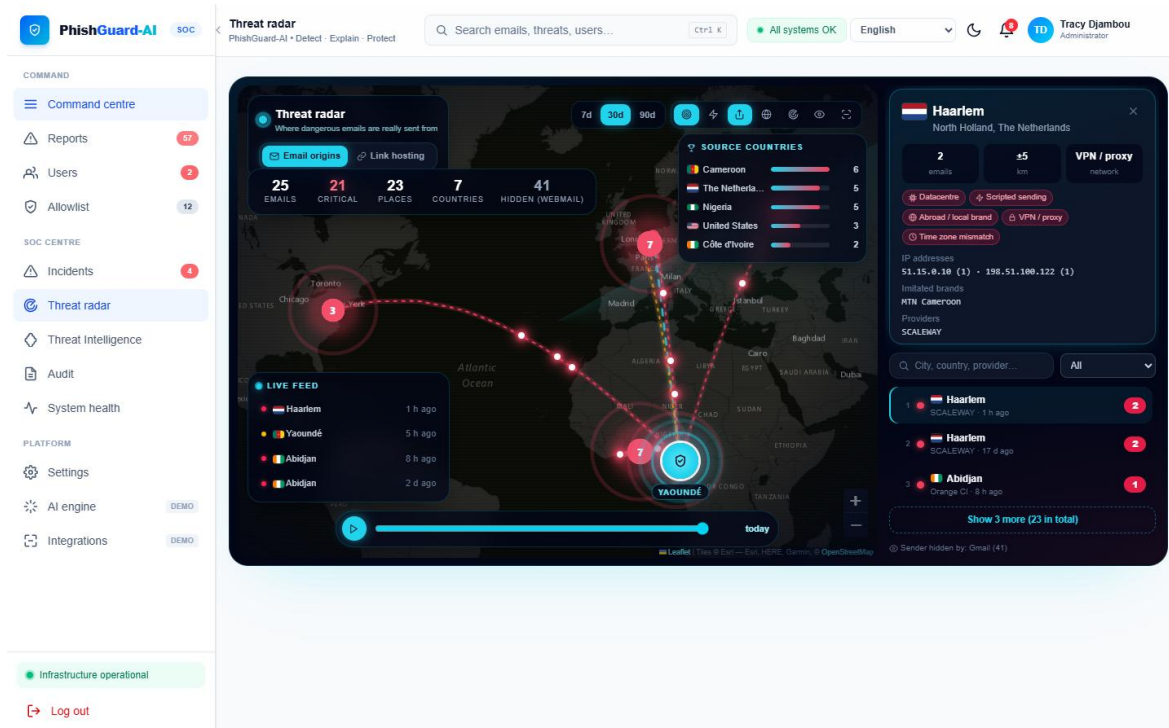


Figure 4.29: Threat radar with a selected source

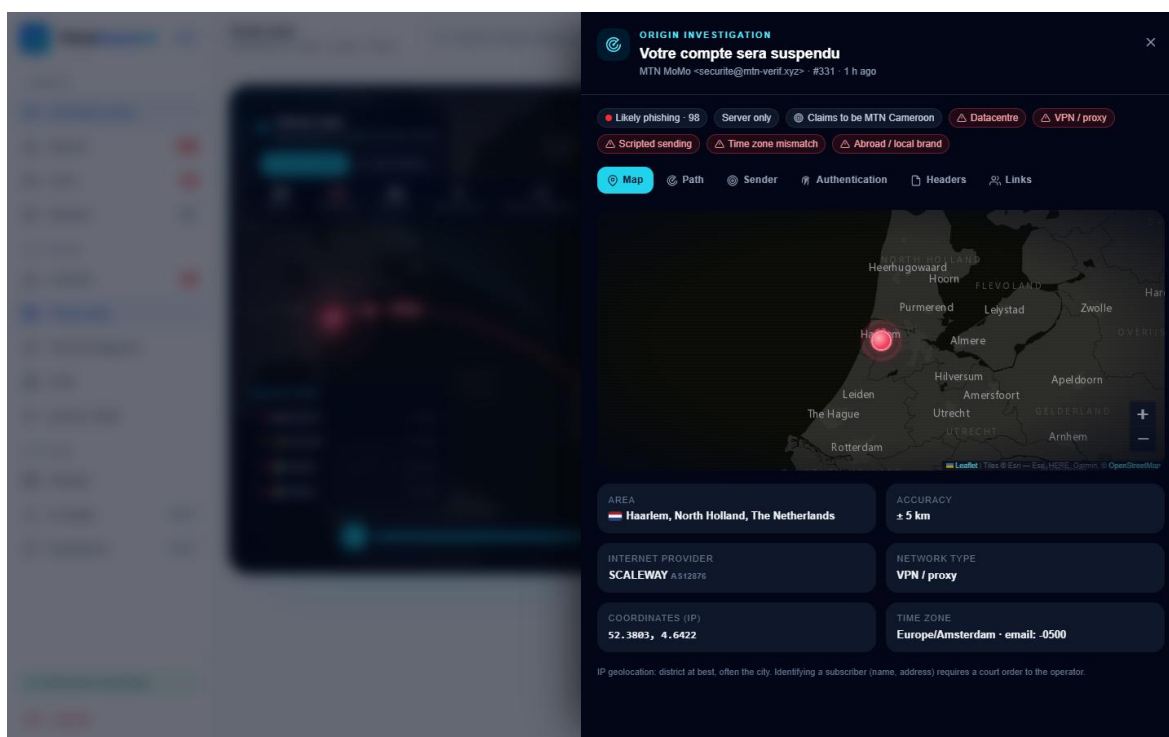


Figure 4.30: Origin investigation (administrator view)

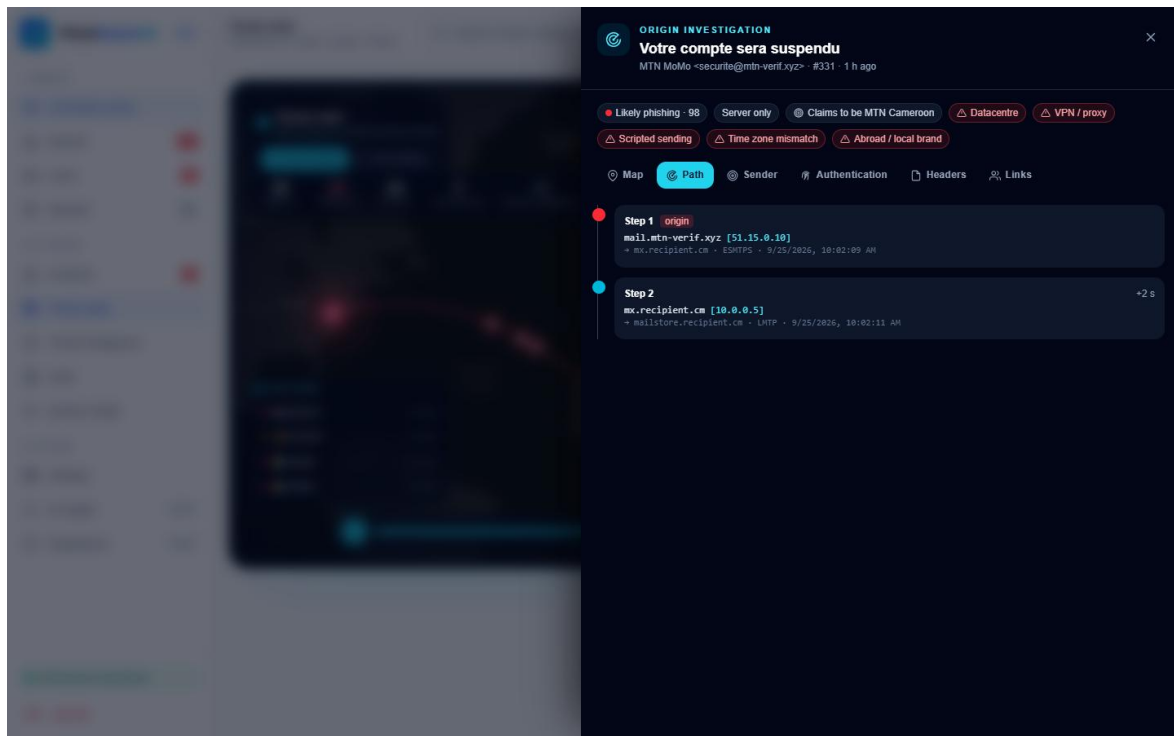


Figure 4.31: Hop-by-hop path of an e-mail

User management (Figures 4.32 and 4.33) shows each new account's registration risk — IP, network, device, flags such as disposable e-mail or very fast form filling — and lets the administrator approve, reject or suspend. The review queue (Figure 4.34) lists user-reported analyses with the engine suggestion and confidence. The incidents page (Figures 4.35 and 4.36) shows correlated campaigns with their timeline, affected users and indicators.

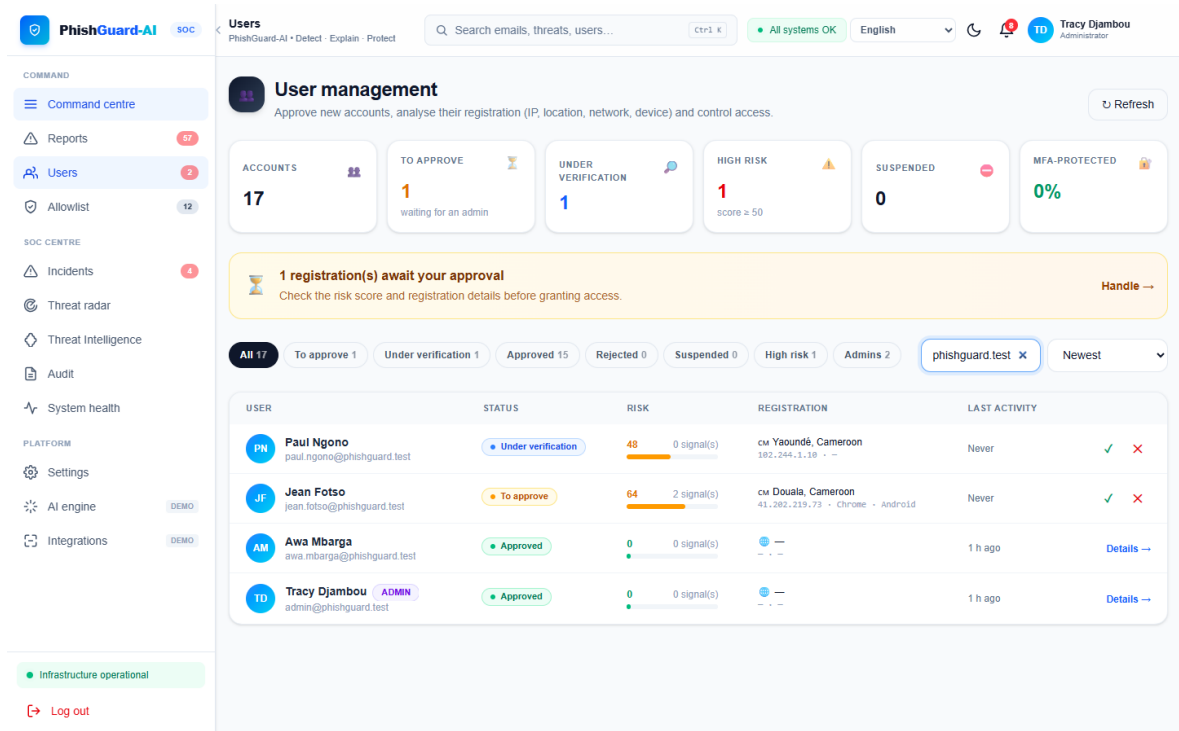


Figure 4.32: User management with registration risk

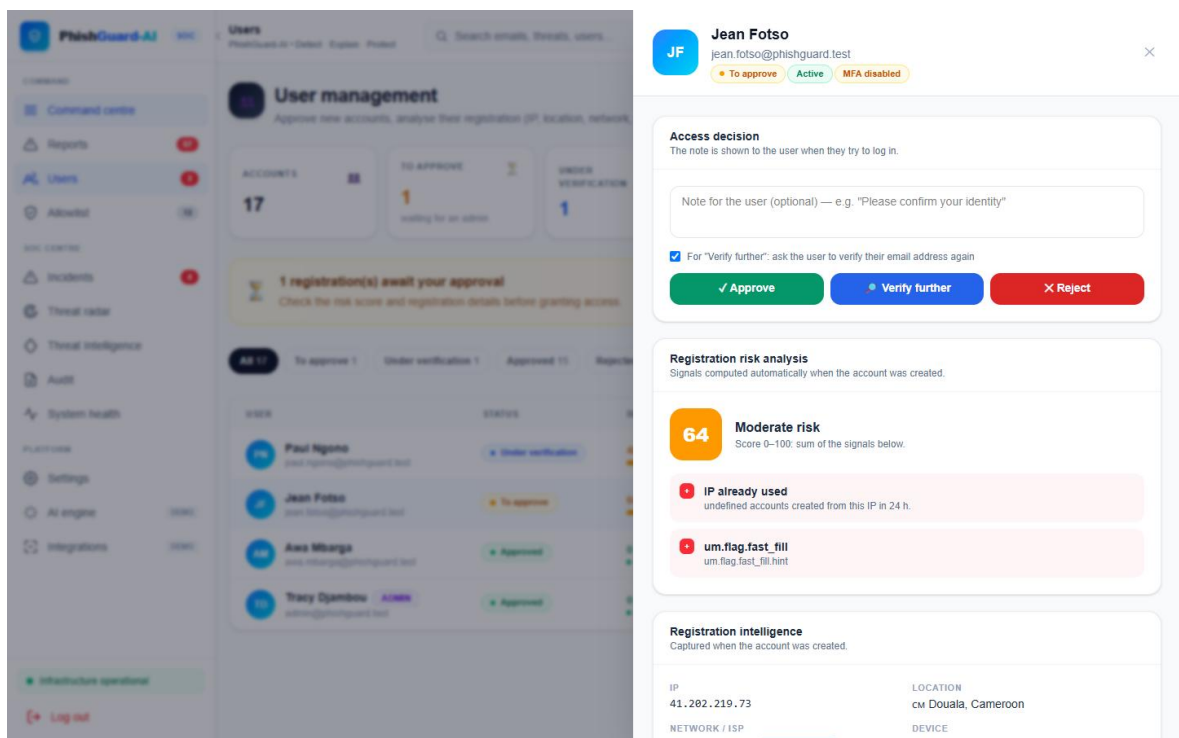


Figure 4.33: Detail of a pending registration

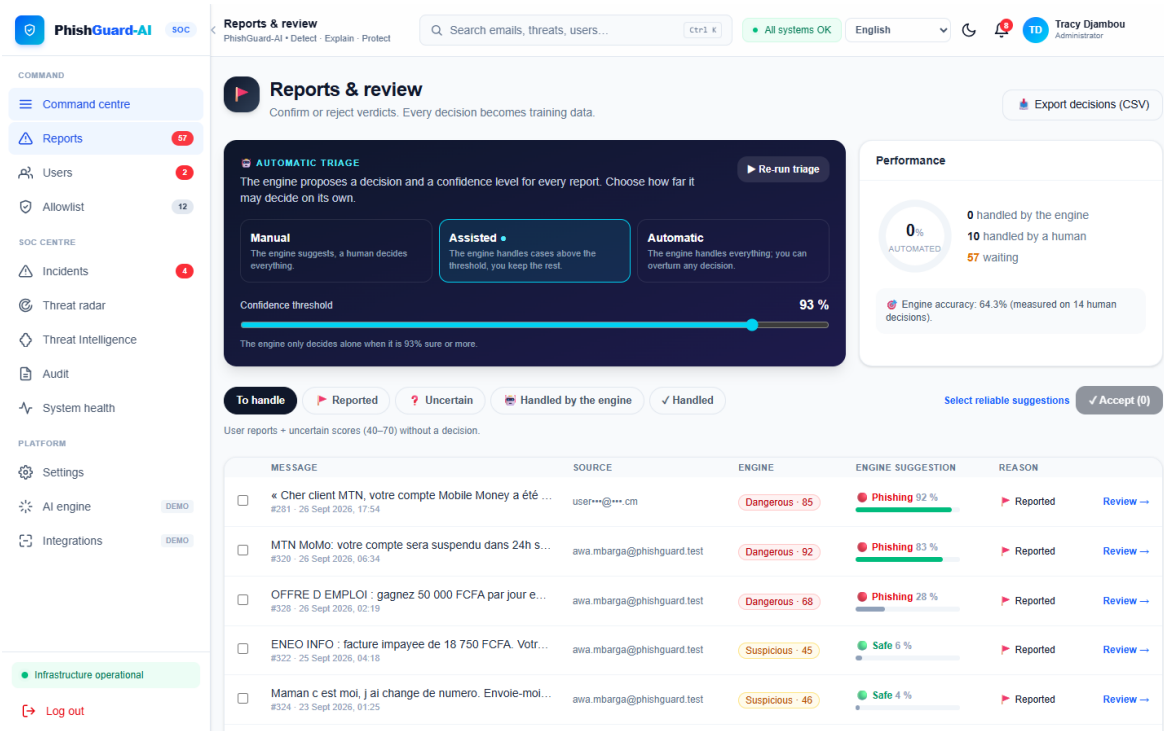


Figure 4.34: Review queue with triage suggestion

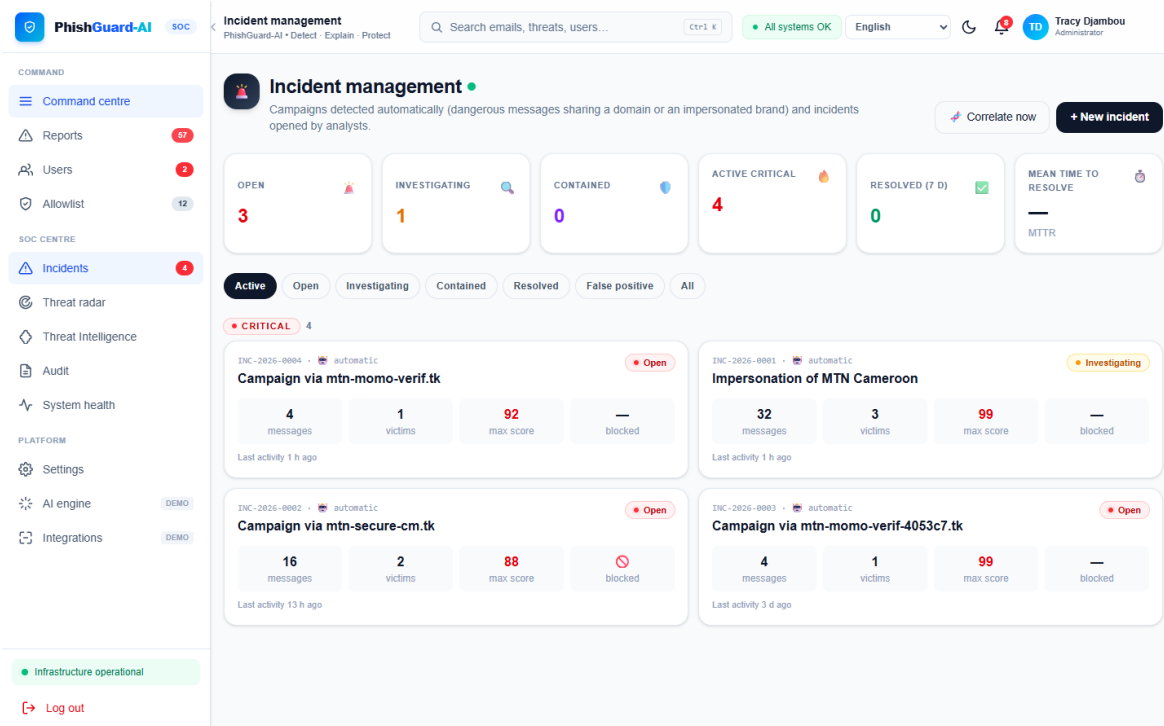


Figure 4.35: Correlated incidents

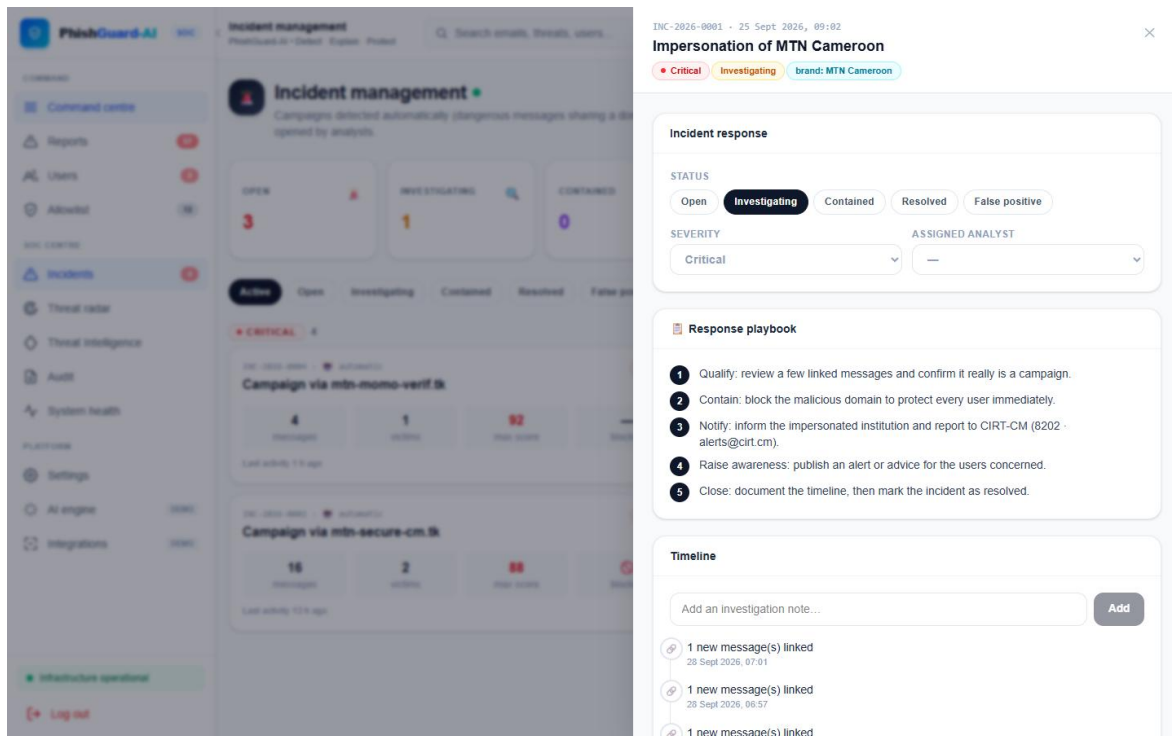


Figure 4.36: Incident detail with timeline

Finally, the threat-intelligence page manages the blocklist and the OpenPhish feed (Figure 4.37); the allowlist of official Cameroonian domains is monitored for DNS and TLS health (Figure 4.38); the system-health page shows the API latency, error rate, database, background jobs and host resources (Figure 4.39); the audit log records every sensitive action (Figure 4.40); and the settings page controls registration mode, triage mode and threshold, session timeout and other policies (Figure 4.41).

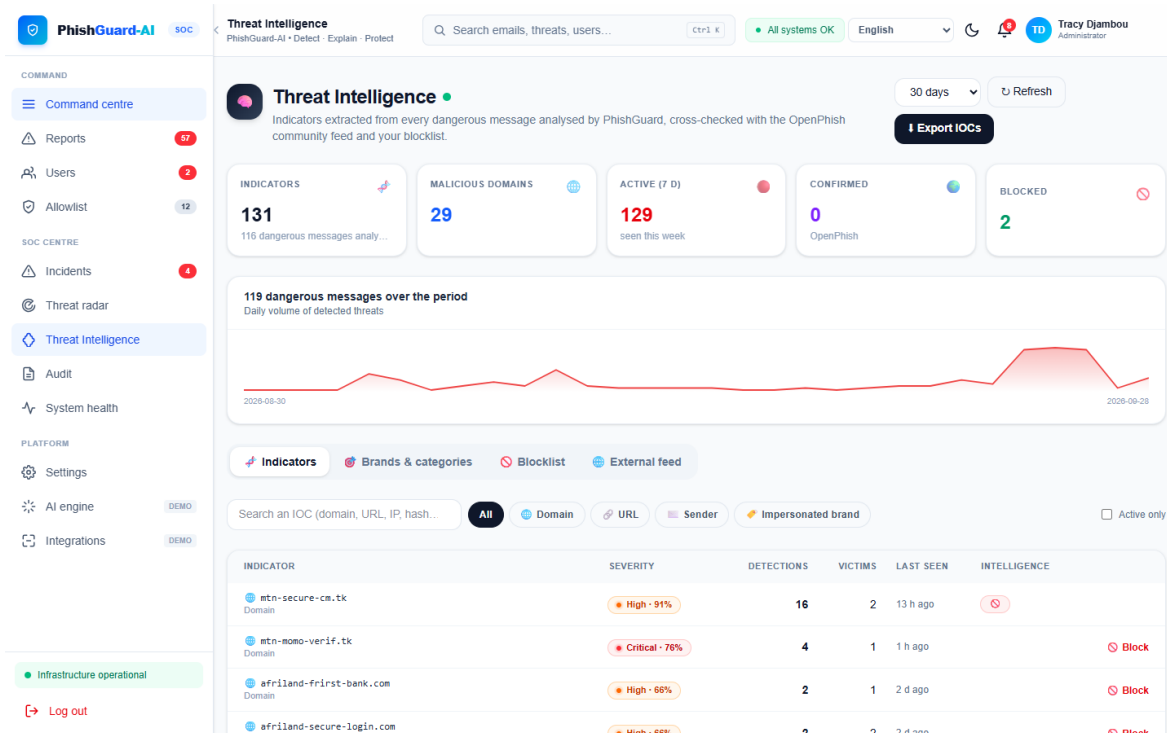


Figure 4.37: Threat intelligence and blocklist

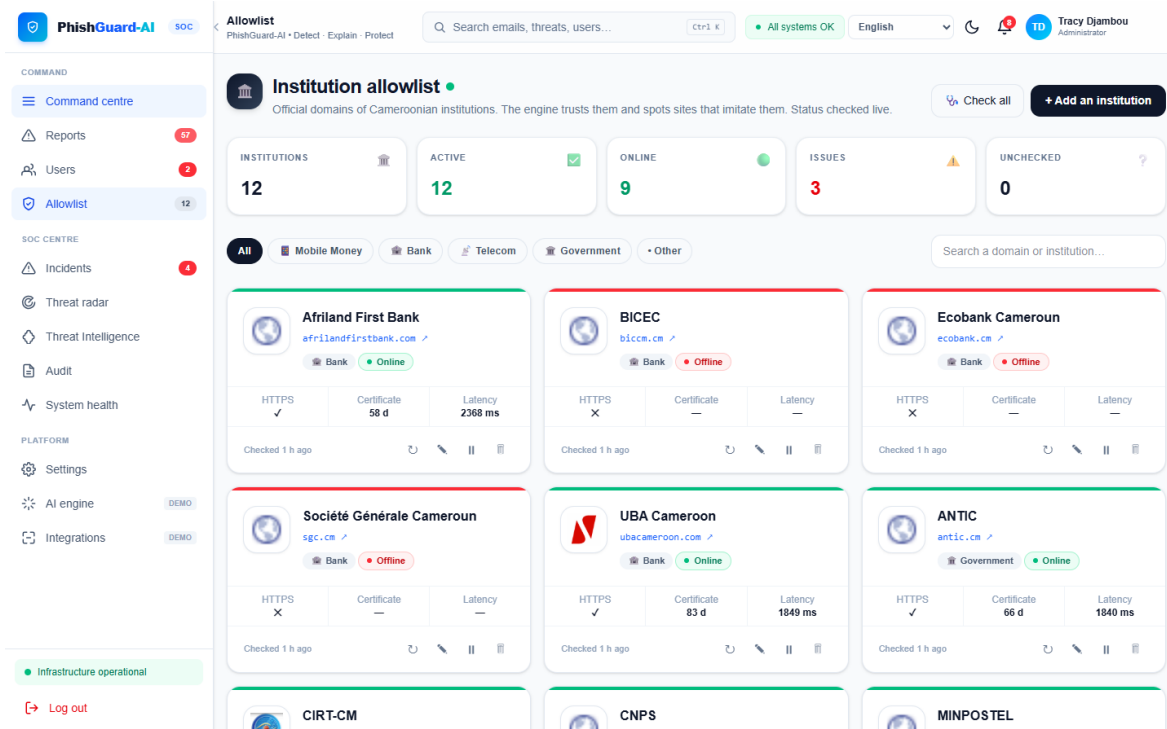


Figure 4.38: Allowlist of official domains with health checks

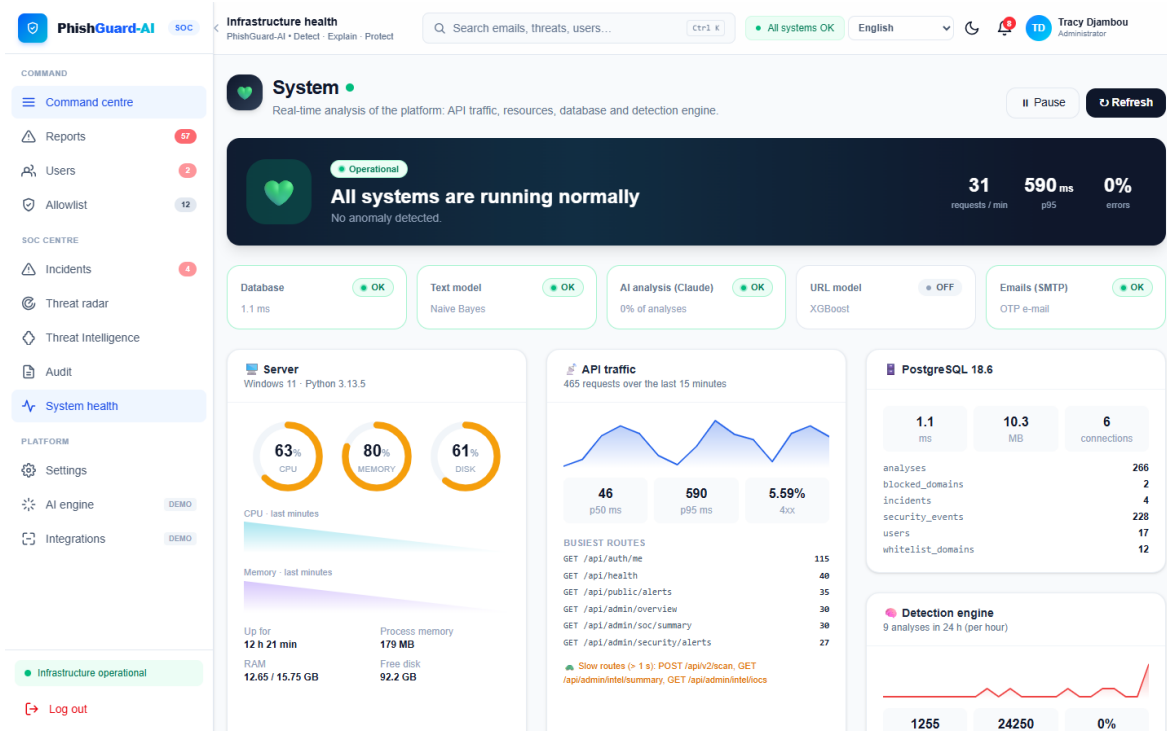


Figure 4.39: System health

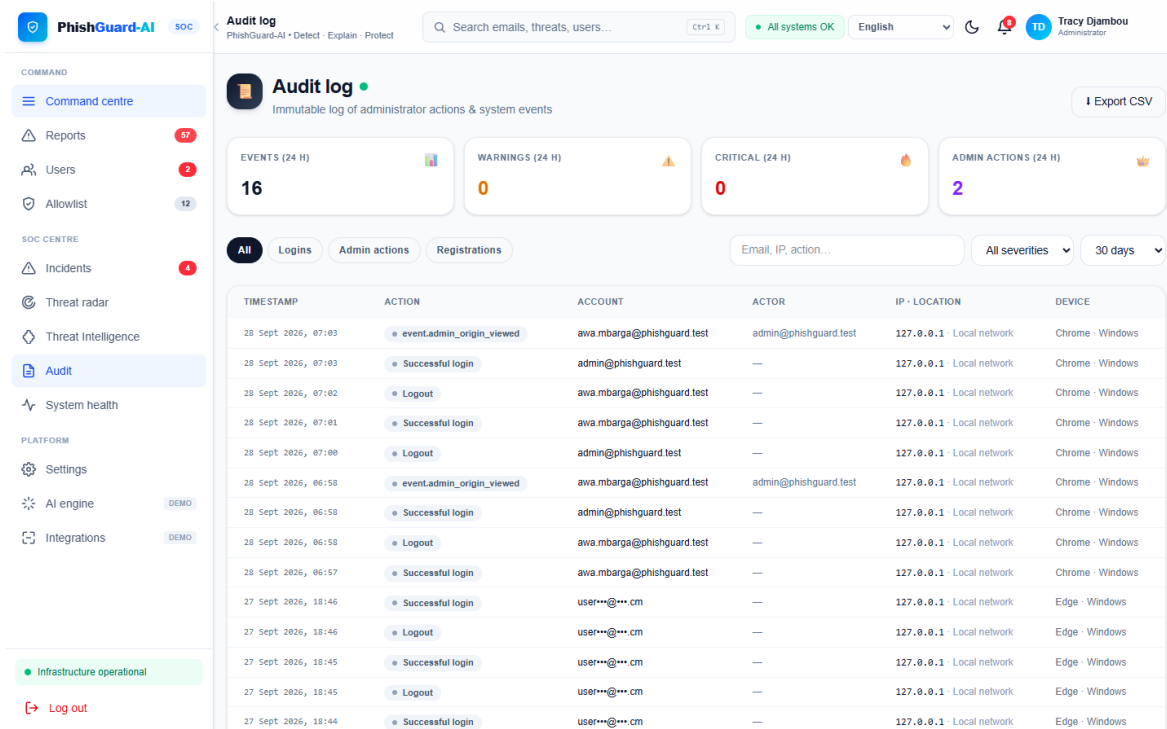


Figure 4.40: Audit log

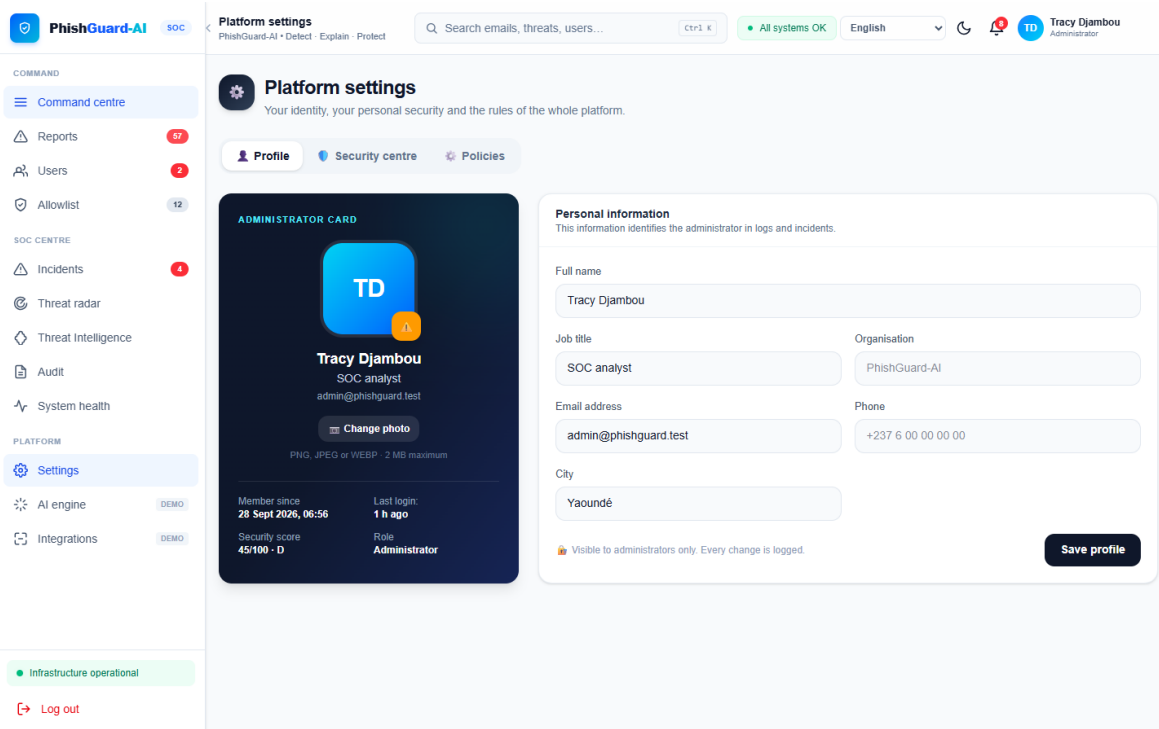


Figure 4.41: Platform settings

4.4.6 Key implementation details

Module	Responsibility
app/__init__.py	Application factory, blueprints, response envelope, request id and timing, session loader.
app/pipeline/	Parser, four signals, fusion, overrides, origin integration.
app/ml_engine/	Text model loader and normaliser, URL features and model, heuristic scorer.
services/scan_service.py	Stores results from every channel, visitor claim tokens, SOC automation.
services/mailbox_service.py	Gmail/Outlook OAuth, token refresh and encryption, incremental sync, live progress.
services/email_origin.py	Received-chain parsing, first public hop, geolocation, DNSBL, flags, user and admin views.
services/triage_service.py, correlation_service.py	Review suggestions; incident correlation and blocklist feed.
services/audit_service.py, registration_risk.py	Security event log; sign-up risk scoring.
services/scheduler.py	Background jobs (mailboxes, forwards, correlation, feeds, health).
train_ml.py, evaluation/evaluate.py	Reproducible training and evaluation.

Table 4.9: Main back-end modules

The AI signal. Before anything leaves the server, the message is **redacted**: e-mail addresses, Cameroonian phone numbers and long account numbers are replaced by [EMAIL], [TEL] and [NUMERO]. The Claude analyser then sends it to the Anthropic API inside <message> tags, with a system prompt that (a) describes the Cameroonian context (MTN Mobile Money, Orange Money, local banks, CNPS, taxes, job scams), (b) states that the message is data to analyse and never an instruction to follow — a defence against prompt injection by the attacker who wrote the message — and (c) forbids citing any phone number or address other than a verified list (CIRT-CM), so that the AI can never invent a "helpline" that could itself be a scam. The answer is parsed into a pydantic schema with a classification (phishing, suspicious, legitimate), a category (mobile-money fraud, job scam, fake site ...), a confidence from 0 to 100, reasons and a recommendation in the user's language. It is mapped to a risk score: $50 + \text{confidence}/2$ for phishing, $30 + \text{confidence}/4$ for suspicious and $50 - \text{confidence}/2$ for legitimate. On API error, refusal or invalid output, the signal returns nothing and its weight is redistributed.

Mailbox scanning. When a user connects Gmail, the back end starts an OAuth 2.0 authorisation-code flow with the `gmail.readonly` scope and a random `state` value checked on return. The access and refresh tokens are encrypted with Fernet before storage. A synchronisation lists messages received after the last cursor, downloads each one in raw form, runs the pipeline, and stores the result — full content for dangerous e-mails, subject only for safe ones. A failing e-mail is logged and skipped rather than aborting the whole synchronisation; a token that can no longer be decrypted or refreshed marks the mailbox as "revoked" and asks the user to reconnect. Synchronisation runs in a background thread and publishes its progress, which the live-scan widget polls.

Method	Endpoint	Purpose
POST	/api/v2/scan	Analyse a message; visitors receive only a claim token
POST	/api/v2/scan/claim	Reveal a gated result after sign-in
POST	/api/auth/register, /login, /logout	Account life cycle
GET	/api/user/summary, /analyses, /origins	Dashboard data, history, personal threat map
DELETE	/api/user/analyses[/<id>]	Delete one or all analyses
POST	/api/user/quiz/answer	Record a quiz answer
GET/POST	/api/mailbox/connect/<provider>, /callback/<provider>	OAuth connection of Gmail / Outlook
POST / GET	/api/mailbox/<id>/sync, /<id>/progress	Start a synchronisation (202) and follow it
GET	/api/public/alerts	Anonymised public alerts

Method	Endpoint	Purpose
GET/POST	/api/admin/users, /reports, /incidents	Approvals, review queue, incidents
GET	/api/admin/origin-map, /analyses/<id>/origin	Threat radar and origin investigation (audited)
GET/POST	/api/admin/security/*	Posture, sudo, sessions, audit events

Table 4.10: Main REST endpoints

4.5 Testing

4.5.1 Test strategy

Testing combined **automated unit and integration tests** with pytest (URL features, claim tokens, scan service, mailbox parsing, origin tracing, access control), **manual functional tests** of each use case through the user interface, **security tests** of the main attack surfaces, and the **quantitative evaluation** of the detector (Section 4.6). The automated suite of the user-experience and SOC services contains 38 tests, all passing on the final version.

4.5.2 Functional test cases

ID	Scenario	Expected result	Result
TC1	Registered user pastes "MTN MoMo: your account will be suspended... confirm your PIN at http://mtn-momo-verif.tk"	Level Critical, PIN request and look-alike domain highlighted	Pass
TC2	User pastes a genuine Orange Money transfer notification	Level Low, no alarm	Pass
TC3	Visitor submits a message	Progress shown, result hidden, claim token issued	Pass
TC4	Visitor signs in after TC3	Result revealed once; second claim refused	Pass
TC5	User uploads a screenshot of an SMS	Text extracted by OCR and analysed	Pass
TC6	User uploads a .eml with Received headers	Verdict plus origin card (city, network, flags)	Pass
TC7	User connects Gmail	Read-only consent; new e-mails scanned; live progress	Pass
TC8	User forwards an e-mail to the PhishGuard address	Verdict received by e-mail	Pass
TC9	User reports a wrong verdict	Item appears in review queue with suggestion	Pass
TC10	Five dangerous messages share a domain	Incident created, domain proposed for blocklist	Pass
TC11	Claude API key removed	Analysis still returned, AI weight redistributed	Pass

ID	Scenario	Expected result	Result
TC12	User deletes an analysis / whole history	Records removed from database	Pass

Table 4.11: Functional test cases

4.5.3 Security test cases

ID	Attack / check	Expected behaviour	Result
ST1	Call /api/v2/scan anonymously and try to read the verdict	Only {gated, claim_token} returned	Pass
ST2	Replay a claim token or use it after 24 h	Refused	Pass
ST3	Normal user calls an /api/admin endpoint	HTTP 403	Pass
ST4	Admin deletes a user without recent password confirmation	SUDO_REQUIRED, confirmation dialog	Pass
ST5	Use a session cookie after "sign out everywhere"	HTTP 401 (session version changed)	Pass
ST6	Suspended user with valid cookie	Rejected by load_user	Pass
ST7	POST without CSRF token on a form route	Rejected	Pass
ST8	Brute force on login / OTP	Rate limited and logged in audit	Pass
ST9	SQL injection in search fields (' OR 1=1 --)	No effect (ORM parameterised queries)	Pass
ST10	Stored XSS in a submitted message	Rendered as text (Vue escaping)	Pass
ST11	Read OAuth tokens directly in the database	Only ciphertext stored	Pass
ST12	Sign up from a disposable e-mail through a datacentre IP	High risk score, account held for review	Pass

Table 4.12: Security test cases

4.6 Findings

4.6.1 Detection performance

Table 4.13 and Figure 4.42 report the performance of each configuration on the held-out Cameroonian test set (a message is counted as flagged when its score is above 40).

Configuration	n	Precision	Recall	F1	FPR	TP / FP / TN / FN
Text model only	204	0.784	0.991	0.876	31.9 %	109 / 30 / 64 / 1
Rules only	204	1.000	0.064	0.120	0.0 %	7 / 0 / 94 / 103
Full pipeline without AI	204	0.899	0.809	0.852	10.6 %	89 / 10 / 84 / 21
Claude AI only	40	0.923	1.000	0.960	12.5 %	24 / 2 / 14 / 0
Full pipeline with AI	40	0.920	0.958	0.939	12.5 %	23 / 2 / 14 / 1

Table 4.13: Detection performance on held-out Cameroonian messages

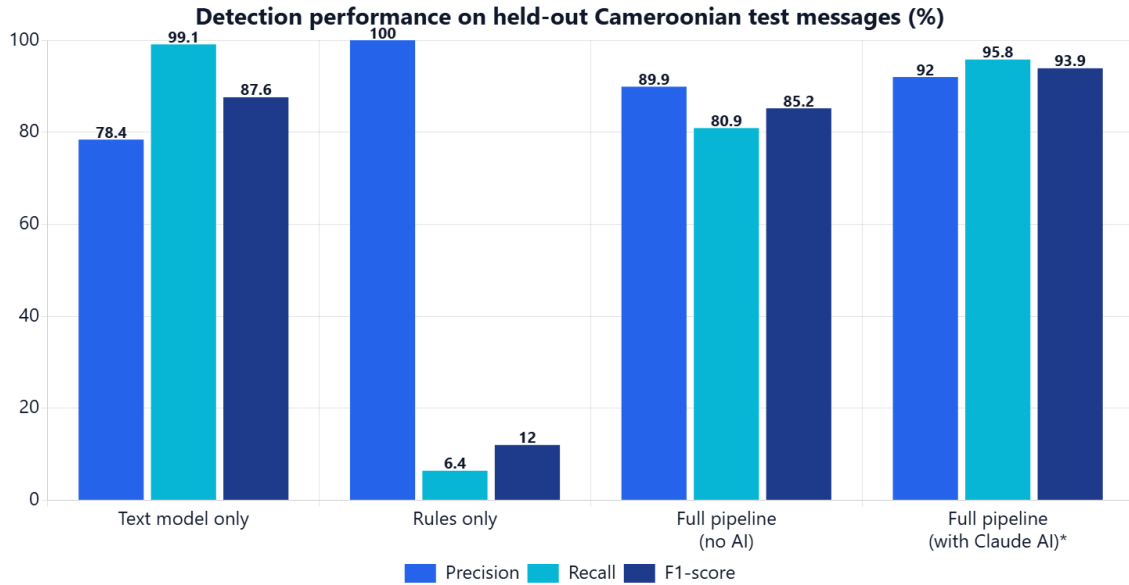


Figure 4.42: Precision, recall and F1-score per configuration

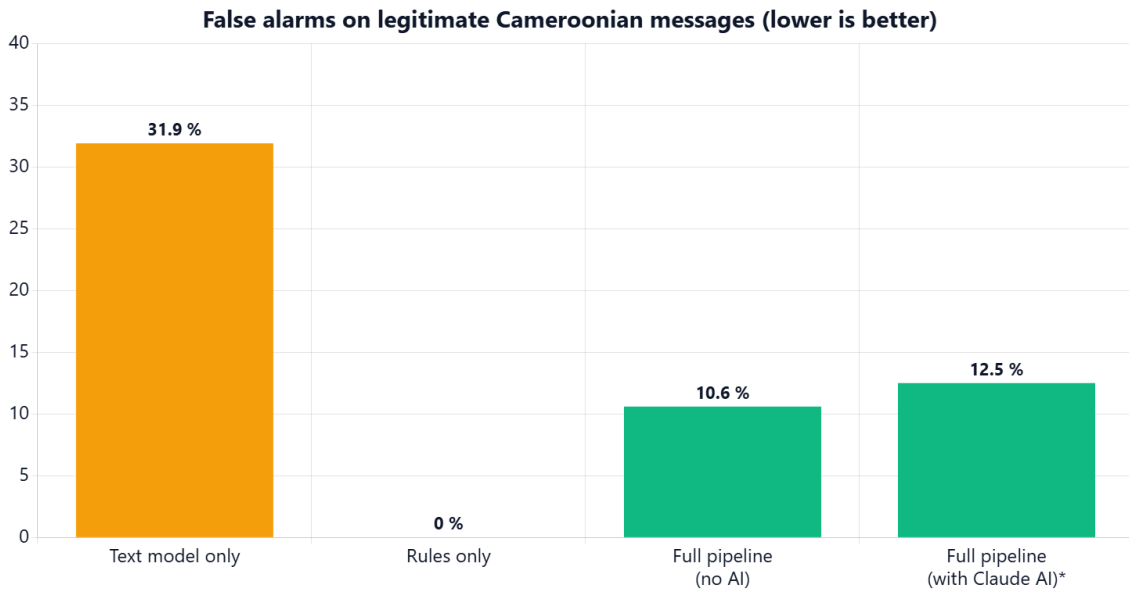


Figure 4.43: False-positive rate per configuration

The text model alone catches almost every scam (recall 0.991) but flags almost one legitimate message in three (false-positive rate 31.9 %). The rules alone never raise a false alarm but catch only 6.4 % of scams. Fusing the signals without AI reduces the false-positive rate by a factor of three (to 10.6 %) with precision 0.899, at the cost of lower recall (0.809). Adding the Claude signal raises recall to 0.958 and F1 to 0.939 on the 40-message sub-sample, while keeping the false-positive rate at 12.5 % (2 of 16 legitimate messages).

On the hand-written challenge set (24 messages), the pipeline without AI reaches F1 0.70 (precision 0.88, recall 0.58), showing that subtle scams without links or typical keywords remain the hardest cases for the offline signals. On the held-out part of the public UCI SMS corpus (297 messages), the pipeline without AI reaches precision 1.00, recall 0.94 and F1 0.97, with no false positives.

4.6.2 Response time

Configuration	Median (p50)	95th percentile
Pipeline without AI (offline signals)	10 ms	12 ms
Pipeline with Claude AI	7.2 s	—

Table 4.14: Measured analysis latency

4.6.3 Achievement of the specific objectives

Specific objective	Evidence	Status
SO1 — Hybrid engine, $F1 \geq 0.85$ and $FPR < 15\%$ on held-out local messages	F1 0.852 / FPR 10.6 % without AI (n = 204); F1 0.939 / FPR 12.5 % with AI (n = 40)	Achieved
SO2 — Multi-channel platform, median response < 10 s	Text, link, OCR, .eml, forward, Gmail/Outlook, PWA share target; median 7.2 s with AI, 10 ms without	Achieved
SO3 — Awareness component	Highlighted cues, actions, read-aloud, quiz with scoring, privacy page	Achieved (impact on users not yet measured)
SO4 — Secured administration and SOC	Approval, triage, incidents, radar, origin tracing, audit, sudo; 12 security tests passed	Achieved

Table 4.15: Achievement of the specific objectives

4.6.4 Results per test source

Configuration	Templates (n = 178)	Challenge (n = 24)	UCI SMS (n = 297)	All local (n = 204)
Text model only	0.87	0.88	1.00	0.88
Rules only	0.12	0.00	0.06	0.12
Pipeline without AI	0.87	0.70	0.97	0.85

Table 4.16: F1-score of each configuration per test source (flag threshold 40)

4.6.5 Error analysis

To understand where the offline pipeline fails, every misclassified Cameroonian test message was grouped by family (Table 4.17).

Family	Type	Errors	Typical score and cause
Fake relative	Missed scam	9 of 14	Scores 37–38: no link, no brand, no secret requested; only urgency and a money request.
PIN request via fake agent	Missed scam	5 of 12	Scores 35–40: the word "code" appears in a friendly customer-care tone.
Challenge set	Missed scam	5 of 12	Subtle, hand-written scams with no typical keyword.
Utility bill / admin label	Missed scam	2	Borderline scores just under 40.
Official link	False alarm	8 of 20	Operator adverts linking to another institution's domain (e.g. "Orange ... on impots.cm"), which the brand check treats as impersonation.
Bank information / challenge	False alarm	2	Genuine statements with account vocabulary, score 41.

Table 4.17: Errors of the pipeline without AI on Cameroonian test messages, by family

Two patterns emerge. **Missed scams** are concentrated in families without any link, where the URL signal is absent and the rule set has no specific pattern; their scores cluster just below the threshold (35–40). These are exactly the cases where the AI signal adds the most: on the AI sub-sample, recall rises from 0.81 to 0.96. **False alarms** come mostly from a generator artefact — adverts from one brand pointing to another brand's official domain — which a human would also find odd; in real traffic such cross-brand links are rare. No false alarm was observed on genuine mobile-money receipts, OTP messages, appointments or personal messages.

4.6.6 Limitations observed

- The Cameroonian part of the corpus is built from templates written from real patterns, not from a large collection of real messages; results on real traffic may differ.
- The AI configuration was evaluated on 40 messages only, for cost reasons.
- The user-acceptance study (Appendix A) has not yet been conducted, so the effect of the awareness module on user behaviour is not yet measured.
- Sender-origin tracing is impossible when the attacker uses a large webmail provider, which hides the client IP; geolocation is approximate (city level at best).

- The platform has not been load-tested for many simultaneous users.

4.7 Summary

This chapter presented the analysis, design, implementation and testing of PhishGuard-AI and the results of its evaluation. All functional requirements in scope were implemented, all functional and security test cases passed, and the four specific objectives were achieved on the technical evaluation. The discussion of these findings, the conclusions and the recommendations are the subject of Chapter 5.

References

- Almeida, T. A., Gómez Hidalgo, J. M., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. In *Proceedings of the 11th ACM Symposium on Document Engineering* (pp. 259–262). ACM. <https://doi.org/10.1145/2034691.2034742>
- Anti-Phishing Working Group. (2024). *Phishing activity trends report* (1st–4th quarters 2024). APWG. <https://apwg.org/trendsreports/>
- Cavoukian, A. (2009). *Privacy by design: The 7 foundational principles*. Information and Privacy Commissioner of Ontario.
- Cialdini, R. B. (2006). *Influence: The psychology of persuasion* (Rev. ed.). Harper Business.
- Crocker, D., Hansen, T., & Kucherawy, M. (Eds.). (2011). *DomainKeys Identified Mail (DKIM) signatures* (RFC 6376). IETF. <https://doi.org/10.17487/RFC6376>
- Dietterich, T. G. (2000). Ensemble methods in machine learning. In *Multiple Classifier Systems: First International Workshop, MCS 2000* (Lecture Notes in Computer Science, Vol. 1857, pp. 1–15). Springer. https://doi.org/10.1007/3-540-45014-9_1
- Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*.
- Fette, I., Sadeh, N., & Tomasic, A. (2007). Learning to detect phishing emails. In *Proceedings of the 16th International Conference on World Wide Web* (pp. 649–656). ACM. <https://doi.org/10.1145/1242572.1242660>
- Hardt, D. (Ed.). (2012). *The OAuth 2.0 authorization framework* (RFC 6749). IETF. <https://doi.org/10.17487/RFC6749>
- Heiding, F., Schneier, B., Vishwanath, A., Bernstein, J., & Park, P. S. (2024). Devising and detecting phishing emails using large language models. *IEEE Access*, 12, 42131–42146. <https://doi.org/10.1109/ACCESS.2024.3375882>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>
- INTERPOL. (2023). *African cyberthreat assessment report 2023*. INTERPOL Cybercrime Directorate.
- Jakobsson, M., & Myers, S. (Eds.). (2007). *Phishing and countermeasures: Understanding the increasing problem of electronic identity theft*. Wiley.
- Kahneman, D. (2011). *Thinking, fast and slow*. Farrar, Straus and Giroux.
- Kitterman, S. (2014). *Sender Policy Framework (SPF) for authorizing use of domains in email, version 1* (RFC 7208). IETF. <https://doi.org/10.17487/RFC7208>

- Koide, T., Fukushi, N., Nakano, H., & Chiba, D. (2024). ChatSpamDetector: Leveraging large language models for effective phishing email detection. *arXiv preprint arXiv:2402.18093*.
- Kucherawy, M., & Zwicky, E. (Eds.). (2015). *Domain-based Message Authentication, Reporting, and Conformance (DMARC) (RFC 7489)*. IETF. <https://doi.org/10.17487/RFC7489>
- Kumaraguru, P., Sheng, S., Acquisti, A., Cranor, L. F., & Hong, J. (2010). Teaching Johnny not to fall for phish. *ACM Transactions on Internet Technology*, 10(2), Article 7. <https://doi.org/10.1145/1754393.1754396>
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.
- M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). *TOTP: Time-based one-time password algorithm (RFC 6238)*. IETF. <https://doi.org/10.17487/RFC6238>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77. <https://doi.org/10.2753/MIS0742-1222240302>
- Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. In *Proceedings of the 1999 USENIX Annual Technical Conference* (pp. 81–91). USENIX Association.
- Rogers, R. W. (1975). A protection motivation theory of fear appeals and attitude change. *The Journal of Psychology*, 91(1), 93–114. <https://doi.org/10.1080/00223980.1975.9915803>
- Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. *Expert Systems with Applications*, 117, 345–357. <https://doi.org/10.1016/j.eswa.2018.09.029>
- Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308. <https://doi.org/10.1109/PROC.1975.9939>
- Sheng, S., Holbrook, M., Kumaraguru, P., Cranor, L. F., & Downs, J. (2010). Who falls for phish? A demographic analysis of phishing susceptibility and effectiveness of interventions. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 373–382). ACM. <https://doi.org/10.1145/1753326.1753383>
- Sheng, S., Wardman, B., Warner, G., Cranor, L. F., Hong, J., & Zhang, C. (2009). An empirical analysis of phishing blacklists. In *Proceedings of the Sixth Conference on Email and Anti-Spam (CEAS)*.

- Verizon. (2024). *2024 data breach investigations report*. Verizon Business.
- Vishwanath, A., Herath, T., Chen, R., Wang, J., & Rao, H. R. (2011). Why do people get phished? Testing individual differences in phishing vulnerability within an integrated, information processing model. *Decision Support Systems*, 51(3), 576–586. <https://doi.org/10.1016/j.dss.2011.03.002>
- Zhang, Y., Hong, J. I., & Cranor, L. F. (2007). CANTINA: A content-based approach to detecting phishing web sites. In *Proceedings of the 16th International Conference on World Wide Web* (pp. 639–648). ACM. <https://doi.org/10.1145/1242572.1242659>

Appendix A: User Acceptance Questionnaire

This instrument will be administered to at least 30 participants after a 15-minute session in which each participant checks five prepared messages (three scams, two genuine) and completes the quiz. Items 1–10 use a five-point Likert scale (1 = strongly disagree, 5 = strongly agree). Items 1–8 are adapted from the System Usability Scale and the Technology Acceptance Model; items 9–10 measure perceived awareness.

Part 1 — Profile

- Age group: 18–24 / 25–34 / 35–49 / 50+
- Main device: Android / iPhone / computer
- Uses mobile money: yes / no
- Has already received a scam message: yes / no / not sure
- Has already lost money or an account to a scam: yes / no

Part 2 — Perception (Likert 1–5)

1. PhishGuard-AI is easy to use.
2. I could check a message without help.
3. The result was fast enough.
4. I understand why the message was classified as dangerous or safe.
5. The highlighted parts of the message helped me understand the verdict.
6. The recommended actions are clear and I could follow them.
7. I trust the verdicts given by PhishGuard-AI.
8. I would use PhishGuard-AI again when I receive a suspicious message.
9. After using the platform, I feel more able to recognise a scam on my own.
10. I would recommend PhishGuard-AI to my relatives.

Part 3 — Performance measures (observed)

- Number of the five test messages correctly classified by the participant before and after using the platform.
- Quiz score (percentage of correct answers).

- Time taken to obtain the first result.

Part 4 — Open questions

- What did you like most?
- What was difficult or confusing?
- Which feature would you add?